

Basics of Rust Concurrency

Long before multi-core processors were commonplace, operating systems allowed for a single computer to run many programs concurrently. This is achieved by rapidly switching between processes, allowing each to repeatedly make a little bit of progress, one by one. Nowadays, virtually all our computers and even our phones and watches have processors with multiple cores, which can truly execute multiple processes in parallel.

Operating systems isolate processes from each other as much as possible, allowing a program to do its thing while completely unaware of what any other processes are doing. For example, a process cannot normally access the memory of another process, or communicate with it in any way, without asking the operating system's kernel first.

However, a program can spawn extra *threads of execution*, as part of the same *process*. Threads within the same process are not isolated from each other. Threads share memory and can interact with each other through that memory.

This chapter will explain how threads are spawned in Rust, and all the basic concepts around them, such as how to safely share data between multiple threads. The concepts explained in this chapter are foundational to the rest of the book.



If you're already familiar with these parts of Rust, feel free to skip ahead. However, before you continue to the next chapters, make sure you have a good understanding of threads, interior mutability, Send and Sync, and know what a mutex, a condition variable, and thread parking are.

Rust 并发基础

早在多核处理器普及之前，操作系统就已经能让单台计算机同时运行多个程序了。这是通过在进程之间快速切换来实现的，让每个进程依次反复地取得一点进展。如今，几乎所有的计算机，甚至我们的手机和手表，都配备了拥有多个核心的处理器，能够真正地并行执行多个进程。

操作系统会尽可能地将进程彼此隔离，让每个程序都能专注于自己的任务，而完全不知道其他进程正在做什么。例如，一个进程通常无法访问另一个进程的内存，也无法以任何方式与之通信，除非首先请求操作系统的内核。

不过，一个程序可以在同一个进程内生成额外的执行线程。同一进程内的线程之间并不相互隔离。它们共享内存，并可以通过该内存相互交互。

本章将解释在 Rust 中如何生成线程，以及围绕线程的所有基本概念，例如如何在多个线程之间安全地共享数据。本章所阐述的概念是本书其余部分的基础。



如果您已经熟悉 Rust 的这些部分，可以随意跳过。不过，在继续阅读接下来的章节之前，请确保您对线程、内部可变性 Send 和 Sync 有深入的理解，并了解什么是互斥锁、条件变量以及线程挂起。

Threads in Rust

Every program starts with exactly one thread: the main thread. This thread will execute your `main` function and can be used to spawn more threads if necessary.

In Rust, new threads are spawned using the `std::thread::spawn` function from the standard library. It takes a single argument: the function the new thread will execute. The thread stops once this function returns.

Let's take a look at an example:

```
use std::thread;

fn main() {
    thread::spawn(f);
    thread::spawn(f);

    println!("Hello from the main thread.");
}

fn f() {
    println!("Hello from another thread!");

    let id = thread::current().id();
    println!("This is my thread id: {id:?}");
}
```

We spawn two threads that will both execute `f` as their main function. Both of these threads will print a message and show their *thread id*, while the main thread will also print its own message.

Thread ID

The Rust standard library assigns every thread a unique identifier. This identifier is accessible through `Thread::id()` and is of the type `ThreadId`. There's not much you can do with a `ThreadId` other than copying it around and checking for equality. There is no guarantee that these IDs will be assigned consecutively, only that they will be different for each thread.

If you run our example program above several times, you might notice the output varies between runs. This is the output I got on my machine during one particular run:

```
Hello from the main thread.
Hello from another thread!
This is my thread id:
```

Surprisingly, part of the output seems to be missing.

Rust 中的线程

每个程序都恰好由一个线程开始：主线程。该线程将执行您的 `main` 函数，并在必要时用于生成更多线程。

在Rust 中，可以使用标准库中的 `std::thread::spawn` 函数来生成新的线程。它接收一个参数：新线程将要执行的函数。一旦该函数返回，线程就会停止。

让我们来看一个示例：

```
use std::thread;

fn main() {
    thread::spawn(f);
    thread::spawn(f);

    println!("Hello from the main thread.");
}

fn f() {
    println!("Hello from another thread!");

    let id = thread::current().id();
    println!("This is my thread id: {id:?}");
}
```

我们生成两个线程，它们都将执行 `f` 作为其主函数。这两个线程都将打印一条消息并显示它们的线程 *ID*，同时主线程也会打印自己的消息。

线程 ID

Rust标准库为每个线程分配一个唯一的标识符。该标识符可以通过`Thread::id()`访问，其类型为 `ThreadId`。对于 `ThreadId`，除了对其进行复制和检查是否相等之外，你能做的事情并不多。这些 ID 不保证会被连续分配，仅保证每个线程的 ID 都不同。

如果你多次运行上面的示例程序，你可能会发现每次运行的输出结果都不同。这是我在自己的机器上运行一次所得到的输出：

```
Hello from the main thread.
Hello from another thread!
This is my thread id:
```

令人惊讶的是，部分输出似乎丢失了。

What happened here is that the main thread finished executing the `main` function before the newly spawned threads finished executing their functions.

Returning from `main` will exit the entire program, even if other threads are still running.

In this particular example, one of the newly spawned threads had just enough time to get to halfway through the second message, before the program was shut down by the main thread.

If we want to make sure the threads are finished before we return from `main`, we can wait for them by *joining* them. To do so, we have to use the `JoinHandle` returned by the `spawn` function:

```
fn main() {
    let t1 = thread::spawn(f);
    let t2 = thread::spawn(f);

    println!("Hello from the main thread.");

    t1.join().unwrap();
    t2.join().unwrap();
}
```

The `.join()` method waits until the thread has finished executing and returns a `std::thread::Result`. If the thread did not successfully finish its function because it panicked, this will contain the panic message. We could attempt to handle that situation, or just call `.unwrap()` to panic when joining a panicked thread.

Running this version of our program will no longer result in truncated output:

```
Hello from the main thread.
Hello from another thread!
This is my thread id: ThreadId(3)
Hello from another thread!
This is my thread id: ThreadId(2)
```

The only thing that still changes between runs is the order in which the messages are printed:

```
Hello from the main thread.
Hello from another thread!
Hello from another thread!
This is my thread id: ThreadId(2)
This is my thread id: ThreadId(3)
```

这里发生的情况是，主线程在新生成的线程完成其函数执行之前，就已经执行完了 `main` 函数。

从 `main` 返回将退出整个程序，即使其他线程仍在运行。

在这个具体的例子中，其中一个新 `spawn` 的线程仅有足够的时间运行到第二条消息的一半，随后程序就被主线程关闭了。

如果我们想确保在从 `main` 返回之前线程已经运行完毕，可以通过**等待 (join)** 它们来实现。为此，我们需要使用 `spawn` 函数返回的 `JoinHandle`：

```
fn main() {
    let t1 = thread::spawn(f);
    let t2 = thread::spawn(f);

    println!("Hello from the main thread.");

    t1.join().unwrap();
    t2.join().unwrap();
}
```

`.join()` 方法会等待直到线程执行完毕并返回一个 `std::thread::Result`。如果线程因为 `panic` 而未能成功完成其函数，该 `Result` 将包含 `panic` 消息。我们可以尝试处理这种情况，或者在等待一个已 `panic` 的线程时直接调用 `.unwrap()` 来触发 `panic`。

运行这个版本的程序将不再导致输出被截断：

```
Hello from the main thread.
Hello from another thread!
This is my thread id: ThreadId(3)
Hello from another thread!
This is my thread id: ThreadId(2)
```

在不同次运行之间唯一仍然会发生变化的是消息打印的顺序：

```
Hello from the main thread.
Hello from another thread!
Hello from another thread!
This is my thread id: ThreadId(2)
This is my thread id: ThreadId(3)
```

Output Locking

The `println` macro uses `std::io::Stdout::lock()` to make sure its output does not get interrupted. A `println!()` expression will wait until any concurrently running one is finished before writing any output. If this was not the case, we could've gotten more interleaved output such as:

```
Hello fromHello from another thread!
  another This is my thretheadHello fromthread id: ThreadId!
( the main thread.
2)This is my thread
id: ThreadId(3)
```

Rather than passing the name of a function to `std::thread::spawn`, as in our example above, it's far more common to pass it a *closure*. This allows us to capture values to move into the new thread:

```
let numbers = vec![1, 2, 3];

thread::spawn(move || {
    for n in numbers {
        println!("{n}");
    }
}).join().unwrap();
```

Here, ownership of `numbers` is transferred to the newly spawned thread, since we used a `move` closure. If we had not used the `move` keyword, the closure would have captured `numbers` by reference. This would have resulted in a compiler error, since the new thread might outlive that variable.

Since a thread might run until the very end of the program's execution, the `spawn` function has a 'static lifetime bound on its argument type. In other words, it only accepts functions that may be kept around forever. A closure capturing a local variable by reference may not be kept around forever, since that reference would become invalid the moment the local variable ceases to exist.

Getting a value back out of the thread is done by returning it from the closure. This return value can be obtained from the `Result` returned by the `join` method:

```
let numbers = Vec::from_iter(0..=1000);

let t = thread::spawn(move || {
    let len = numbers.len();
    let sum = numbers.into_iter().sum::<usize>();
    sum / len ❶
});

let average = t.join().unwrap(); ❷
```

输出锁定

`println`宏使用`std::io::Stdout::lock()`来确保其输出不会被中断。`println!()`表达式会在写入任何输出之前，等待任何并发运行的表达式执行完毕。如果不是这样，我们可能会看到更多交织的输出，例如：

```
Hello fromHello from another thread!
  another This is my thretheadHello fromthread id: ThreadId!
( the main thread.
2)This is my thread
id: ThreadId(3)
```

与我们上面的示例不同，向 `std::thread::spawn` 传递一个**闭包**而不是函数名称要常见得多。这使我们能够捕获值并将其移动到新线程中：

```
let numbers = vec![1, 2, 3];

thread::spawn(move || {
    for n in numbers {
        println!("{n}");
    }
}).join().unwrap();
```

这里，由于我们使用了 `move` 闭包，`numbers` 的所有权被转移到了新生成的线程中。如果我们没有使用 `move` 关键字，闭包将会通过引用捕获 `numbers`。这将导致编译器错误，因为新线程的生命周期可能会超过该变量。

由于线程可能会一直运行到程序执行结束，因此 `spawn` 函数对其参数类型有一个 'static 生命周期限制。换句话说，它只接受可以永久保留的函数。通过引用捕获局部变量的闭包不能被永久保留，因为一旦局部变量停止存在，该引用就会变得无效。

从线程中获取值的做法是从闭包中将其返回。该返回值可以通过 `join` 方法返回的 `Result` 来获取：

```
let numbers = Vec::from_iter(0..=1000);

let t = thread::spawn(move || {
    let len = numbers.len();
    let sum = numbers.into_iter().sum::<usize>();
    sum / len ❶
});

let average = t.join().unwrap(); ❷
```

```
println!("average: {average}");
```

Here, the value returned by the thread's closure (❶) is sent back to the main thread through the join method (❷).

If `numbers` had been empty, the thread would've panicked while trying to divide by zero (❶), and `join` would've returned that panic message instead, causing the main thread to panic too because of `unwrap` (❷).

Thread Builder

The `std::thread::spawn` function is actually just a convenient shorthand for `std::thread::Builder::new().spawn().unwrap()`.

A `std::thread::Builder` allows you to set some settings for the new thread before spawning it. You can use it to configure the stack size for the new thread and to give the new thread a name. The name of a thread is available through `std::thread::current().name()`, will be used in panic messages, and will be visible in monitoring and debugging tools on most platforms.

Additionally, Builder's `spawn` function returns an `std::io::Result`, allowing you to handle situations where spawning a new thread fails. This might happen if the operating system runs out of memory, or if resource limits have been applied to your program. The `std::thread::spawn` function simply panics if it is unable to spawn a new thread.

Scoped Threads

If we know for sure that a spawned thread will definitely not outlive a certain scope, that thread could safely borrow things that do not live forever, such as local variables, as long as they outlive that scope.

The Rust standard library provides the `std::thread::scope` function to spawn such *scoped threads*. It allows us to spawn threads that cannot outlive the scope of the closure we pass to that function, making it possible to safely borrow local variables.

How it works is best shown with an example:

```
let numbers = vec![1, 2, 3];

thread::scope(|s| { ❶
    s.spawn(|| { ❷
        println!("length: {}", numbers.len());
    });
    s.spawn(|| { ❷
        for n in &numbers {
```

```
println!("average: {average}");
```

这里，线程闭包 (❶) 返回的值通过 `join` 方法 (❷) 发回给主线程。

如果 `numbers` 为空，该线程在尝试除以零 (❶) 时将会 panic，而 `join` 将会返回该 panic 消息，并由于 `unwrap` (❷) 而导致主线程也随之 panic。

线程构建器

`std::thread::spawn` 函数实际上只是 `std::thread::Builder::new().spawn().unwrap()` 的一个便捷缩写。

`std::thread::Builder` 允许你在生成新线程之前为其设置一些配置。你可以用它来配置新线程的栈大小，以及为新线程指定一个名称。要获取线程的名称，可以通过调用方法 `std::thread::current().name()`，它会出现 panic 信息中，并且在大多数平台的监控和调试工具中都是可见的。

此外，Builder 的 `spawn` 函数返回一个 `std::io::Result`，允许你处理创建新线程失败的情况。这种情况可能发生在操作系统内存不足，或者你的程序被应用了资源限制时。而 `std::thread::spawn` 函数在无法创建新线程时则会直接 panic。

作用域线程

如果我们能确定一个 `spawn` 的线程绝对不会比某个作用域存活得更久，那么只要相关内容比该作用域存活得久，该线程就可以安全地借用非永久存活的对象，例如局部变量。

Rust 标准库提供了 `std::thread::scope` 函数来生成这样的作用域线程。它允许我们生成的线程不会比传递给该函数的闭包的作用域存活得更久，从而使得安全地借用局部变量成为可能。

其工作原理通过一个示例可以最清晰地展示：

```
let numbers = vec![1, 2, 3];

thread::scope(|s| { ❶
    s.spawn(|| { ❷
        println!("length: {}", numbers.len());
    });
    s.spawn(|| { ❷
        for n in &numbers {
```

```

        println!("{n}");
    }
}); ❸

```

- ❶ We call the `std::thread::scope` function with a closure. Our closure is directly executed and gets an argument, `s`, representing the scope.
- ❷ We use `s` to spawn threads. The closures can borrow local variables like `numbers`.
- ❸ When the scope ends, all threads that haven't been joined yet are automatically joined.

This pattern guarantees that none of the threads spawned in the scope can outlive the scope. Because of that, this scoped spawn method does not have a 'static bound on its argument type, allowing us to reference anything as long as it outlives the scope, such as `numbers`.

In the example above, both of the new threads are concurrently accessing `numbers`. This is fine, because neither of them (nor the main thread) modifies it. If we were to change the first thread to modify `numbers`, as shown below, the compiler wouldn't allow us to spawn another thread that also uses `numbers`:

```

let mut numbers = vec![1, 2, 3];

thread::scope(|s| {
    s.spawn(|| {
        numbers.push(1);
    });
    s.spawn(|| {
        numbers.push(2); // Error!
    });
});

```

The exact error message depends on the version of the Rust compiler, since it's often improved to produce better diagnostics, but attempting to compile the code above will result in something like this:

```

error[E0499]: cannot borrow `numbers` as mutable more than once at a time
--> example.rs:7:13
   |
 4 |     s.spawn(|| {
   |               -- first mutable borrow occurs here
 5 |         numbers.push(1);
   |         ^^^^^ first borrow occurs due to use of `numbers` in closure
   |
 7 |     s.spawn(|| {
   |               ^^ second mutable borrow occurs here
 8 |         numbers.push(2);
   |         ^^^^^ second borrow occurs due to use of `numbers` in closure

```

```

        println!("{n}");
    }
});

```

```
}); ❸
```

- ❶ 我们调用 `std::thread::scope` 函数并传入一个闭包。我们的闭包会被直接执行，并接收一个代表作用域的参数 `s`。
- ❷ 我们使用 `s` 来 spawn 线程。这些闭包可以借用本地变量，例如 `numbers`。
- ❸ 作用域结束时，所有尚未被手动等待的线程都会被自动等待完成。

这种模式保证了在作用域中 spawn 的任何线程都不会超出该作用域的生命周期。正因如此，这个 `scoped spawn` 方法不需要在其参数类型上设置 'static 约束，从而允许我们引用任何比该作用域生命周期更长的内容，例如 `numbers`。

在上面的示例中，两个新线程都在并发地访问 `numbers`。这是没问题的，因为它们（以及主线程）都没有修改它。如果我们将第一个线程改为修改 `numbers`（如下所示），编译器将不允许我们 spawn 另一个同样使用 `numbers` 的线程：

```

let mut numbers = vec![1, 2, 3];

thread::scope(|s| {
    s.spawn(|| {
        numbers.push(1);
    });
    s.spawn(|| {
        numbers.push(2); // Error!
    });
});

```

具体的错误信息取决于 Rust 编译器的版本，因为编译器经常会进行改进以提供更好的诊断信息，但尝试编译上述代码会导致类似下面的结果：

```

error[E0499]: cannot borrow `numbers` as mutable more than once at a time
--> example.rs:7:13
   |
 4 |     s.spawn(|| {
   |               -- first mutable borrow occurs here
 5 |         numbers.push(1);
   |         ^^^^^ first borrow occurs due to use of `numbers` in closure
   |
 7 |     s.spawn(|| {
   |               ^^ second mutable borrow occurs here
 8 |         numbers.push(2);
   |         ^^^^^ second borrow occurs due to use of `numbers` in closure

```


The Leakpocalypse

Before Rust 1.0, the standard library had a function named `std::thread::scoped` that would directly spawn a thread, just like `std::thread::spawn`. It allowed non-`'static` captures, because instead of a `JoinHandle`, it returned a `JoinGuard` which joined the thread when dropped. Any borrowed data only needed to outlive this `JoinGuard`. This seemed safe, as long as the `JoinGuard` got dropped at some point.

Just before the release of Rust 1.0, it slowly became clear that it's not possible to guarantee that something will be dropped. There are many ways, such as creating a cycle of reference-counted nodes, that make it possible to forget about something, or *leak* it, without dropping it.

Eventually, in what some people refer to as “The Leakpocalypse,” the conclusion was made that the design of a (safe) interface cannot rely on the assumption that objects will always be dropped at the end of their lifetime. Leaking an object might reasonably result in leaking more objects (e.g., leaking a `Vec` will also leak its elements), but it may not result in undefined behavior. Because of this conclusion, `std::thread::scoped` was no longer deemed safe and was removed from the standard library. Additionally, `std::mem::forget` was upgraded from an unsafe function to a *safe* function, to emphasize that forgetting (or leaking) is always a possibility.

Only much later, in Rust 1.63, a new `std::thread::scoped` function was added with a new design that does not rely on `Drop` for correctness.

Shared Ownership and Reference Counting

So far we've looked at transferring ownership of a value to a thread using a move closure (“[Threads in Rust](#)” on page 2) and borrowing data from longer-living parent threads (“[Scoped Threads](#)” on page 5). When sharing data between two threads where neither thread is guaranteed to outlive the other, neither of them can be the owner of that data. Any data shared between them will need to live as long as the longest living thread.

Statics

There are several ways to create something that's not owned by a single thread. The simplest one is a `static` value, which is “owned” by the entire program, instead of an individual thread. In the following example, both threads can access `X`, but neither of them owns it:

```
static X: [i32; 3] = [1, 2, 3];
```

Leakpocalypse

在 Rust 1.0 之前，标准库中有一个名为 `std::thread::scoped` 的函数，它可以像 `std::thread::spawn` 一样直接 spawn 一个线程。它允许非 `'static` 捕获，因为它返回的不是 `JoinHandle`，而是一个在 `Drop` 时会汇合线程的 `JoinGuard`。任何借用数据只需要比这个 `JoinGuard` 存活时间更长即可。只要 `JoinGuard` 在某个时刻被 `Drop`，这看起来就是安全的。

就在 Rust 1.0 发布之前，人们逐渐意识到，无法保证某个对象一定会被 `Drop`。有很多方式（例如创建引用计数节点的循环）会导致人们遗忘某个对象，或者在不执行 `Drop` 的情况下使其 *leak*（内存泄漏）。

最终，在一些人称之为“Leakpocalypse”的事件中，人们得出了这样一个结论：(安全)接口的设计不能依赖于“对象在生命周期结束时一定会被 `Drop`”这一假设。一个对象的内存泄漏可能会合理地导致更多对象的内存泄漏(例如，leak 一个 `Vec` 也会导致其元素 leak)，但这并不一定会导致未定义行为。基于这一结论，`std::thread::scoped` 不再被认为是安全的，并从标准库中移除。此外，`std::mem::forget` 从一个 `unsafe` 函数升级为了 安全 函数，以强调遗忘（或内存泄漏）始终是一种可能性。

直到很久之后的 Rust 1.63 版本，才增加了一个具有新设计的 `std::thread::scoped` 函数，该函数不再依赖于 `Drop` 来保证正确性。

共享所有权与引用计数

到目前为止，我们已经研究了如何使用 `move` 闭包将值的所有权转移到线程（见第 2 页的“[Threads in Rust](#)”），以及如何从生命周期更长的父线程中借用数据（见第 5 页的“[作用域线程](#)”）。当在两个线程之间共享数据，且无法保证其中任何一个线程比另一个存活时间更长时，这两个线程都不能作为该数据的所有者。任何在它们之间共享的数据都需要与存活时间最长的线程一样长地存在。

静态变量

有几种方法可以创建不被单线程拥有的对象。最简单的一种是 `static` 值，它由整个程序而非单个线程“拥有”。在接下来的示例中，两个线程都可以访问 `x`，但它们都不拥有它：

```
static X: [i32; 3] = [1, 2, 3];
```

```
thread::spawn(|| dbg!(&X));
thread::spawn(|| dbg!(&X));
```

A `static` item has a constant initializer, is never dropped, and already exists before the main function of the program even starts. Every thread can borrow it, since it's guaranteed to always exist.

Leaking

Another way to share ownership is by *leaking* an allocation. Using `Box::leak`, one can release ownership of a `Box`, promising to never drop it. From that point on, the `Box` will live forever, without an owner, allowing it to be borrowed by any thread for as long as the program runs.

```
let x: &'static [i32; 3] = Box::leak(Box::new([1, 2, 3]));
```

```
thread::spawn(move || dbg!(x));
thread::spawn(move || dbg!(x));
```

The `move` closure might make it look like we're moving ownership into the threads, but a closer look at the type of `x` reveals that we're only giving the threads a *reference* to the data.



References are Copy, meaning that when you “move” them, the original still exists, just like with an integer or boolean.

Note how the `'static` lifetime doesn't mean that the value lived since the start of the program, but only that it lives to the end of the program. The past is simply not relevant.

The downside of leaking a `Box` is that we're *leaking memory*. We allocate something, but never drop and deallocate it. This can be fine if it happens only a limited number of times. But if we keep doing this, the program will slowly run out of memory.

Reference Counting

To make sure that shared data gets dropped and deallocated, we can't completely give up its ownership. Instead, we can *share ownership*. By keeping track of the number of owners, we can make sure the value is dropped only when there are no owners left.

The Rust standard library provides this functionality through the `std::rc::Rc` type, short for “reference counted.” It is very similar to a `Box`, except cloning it will not allocate anything new, but instead increment a counter stored next to the contained

```
thread::spawn(|| dbg!(&X));
thread::spawn(|| dbg!(&X));
```

`static` 项具有常量初始化程序，永远不会被 Drop，并且在程序的主函数启动之前就已经存在。由于它保证始终存在，因此每个线程都可以借用它。

内存泄漏

另一种共享所有权的方法是对内存分配进行*leak*。使用 `Box::leak`，人们可以释放 `Box` 的所有权，并承诺永远不将其 Drop。从那时起，该 `Box` 将在没有所有者的情况下永久存在，从而允许任何线程在程序运行期间对其进行借用。

```
let x: &'static [i32; 3] = Box::leak(Box::new([1, 2, 3]));
```

```
thread::spawn(move || dbg!(x));
thread::spawn(move || dbg!(x));
```

`move` 闭包可能看起来像是我们将所有权移入了线程，但仔细观察 `x` 的类型就会发现，我们只是向线程提供了一个指向数据的引用。



引用是 Copy 的，这意味着当你“移动”它们时，原始引用仍然存在，就像整数或布尔值一样。

请注意，`'static` 生命周期并不意味着该值自程序启动以来就一直存在，而仅是指它会一直存续到程序结束。过去的情况根本不相关。

`leak Box` 的缺点是我们正在造成内存泄漏。我们分配了某些内容，但从未将其 Drop 并释放内存。如果这种情况只发生有限的次数，那是没问题的。但如果我们持续这样做，程序将逐渐耗尽内存。

引用计数

为了确保共享数据能够被 Drop 并释放，我们不能完全放弃其所有权。相反，我们可以共享所有权。通过追踪所有者的数量，我们可以确保只有在没有所有者剩余时，该值才会被 Drop。

Rust 标准库通过 `std::rc::Rc` 类型提供此功能，它是“reference counted” (引用计数) 的缩写。它与 `Box` 非常相似，不同之处在于 Clone 它不会进行新的内存分配，而是增加存储在所包含值旁边的计数器。原始的 `Rc` 和 Clone 后的 `Rc` 都

value. Both the original and cloned Rc will refer to the same allocation; they *share ownership*.

```
use std::rc::Rc;

let a = Rc::new([1, 2, 3]);
let b = a.clone();

assert_eq!(a.as_ptr(), b.as_ptr()); // Same allocation!
```

Dropping an Rc will decrement the counter. Only the last Rc, which will see the counter drop to zero, will be the one dropping and deallocating the contained data.

If we were to try to send an Rc to another thread, however, we would run into the following compiler error:

```
error[E0277]: `Rc` cannot be sent between threads safely
  |
8  |         thread::spawn(move || dbg!(b));
  |                        ^^^^^^^^^^^^^^^^^
```

As it turns out, Rc is not *thread safe* (more on that in “[Thread Safety: Send and Sync](#)” on page 16). If multiple threads had an Rc to the same allocation, they might try to modify the reference counter at the same time, which can give unpredictable results.

Instead, we can use `std::sync::Arc`, which stands for “atomically reference counted.” It’s identical to Rc, except it guarantees that modifications to the reference counter are indivisible *atomic* operations, making it safe to use it with multiple threads. (More on that in [Chapter 2](#).)

```
use std::sync::Arc;

let a = Arc::new([1, 2, 3]); ❶
let b = a.clone(); ❷

thread::spawn(move || dbg!(a)); ❸
thread::spawn(move || dbg!(b)); ❸
```

- ❶ We put an array in a new allocation together with a reference counter, which starts at one.
- ❷ Cloning the Arc increments the reference count to two and provides us with a second Arc to the same allocation.
- ❸ Both threads get their own Arc through which they can access the shared array. Both decrement the reference counter when they drop their Arc. The last thread to drop its Arc will see the counter drop to zero and will be the one to drop and deallocate the array.

将指向同一个内存分配；它们共享所有权。

```
use std::rc::Rc;

let a = Rc::new([1, 2, 3]);
let b = a.clone();

assert_eq!(a.as_ptr(), b.as_ptr()); // Same allocation!
```

Drop 一个 Rc 将使计数器递减。只有最后一个 Rc（它会看到计数器降至零）才会执行 Drop 并释放所包含的数据的内存分配。

然而，如果我们尝试将 Rc 发送到另一个线程，我们将遇到以下编译器错误：

```
error[E0277]: `Rc` cannot be sent between threads safely
  |
8  |         thread::spawn(move || dbg!(b));
  |                        ^^^^^^^^^^^^^^^^^
```

事实证明，Rc 不是线程安全的（详见第 16 页的“[线程安全：Send 与 Sync](#)”）。如果多个线程拥有同一个内存分配的 Rc，它们可能会尝试同时修改引用计数器，这可能会导致不可预测的结果。

相反，我们可以使用 `std::sync::Arc`，它代表“原子引用计数”。它与 Rc 完全相同，不同之处在于它保证对引用计数器的修改是不可分割的原子操作，使其可以在多个线程中安全地使用。（详见第 2 章。）

```
use std::sync::Arc;

let a = Arc::new([1, 2, 3]); ❶
let b = a.clone(); ❷

thread::spawn(move || dbg!(a)); ❸
thread::spawn(move || dbg!(b)); ❸
```

- ❶ 我们将一个数组放在一块新的内存分配中，并附带一个引用计数器，该计数器初始值为 1。
- ❷ 克隆 Arc 会将引用计数增加到 2，并为我们提供指向同一个内存分配的第二个 Arc。
- ❸ 两个线程都拿到了各自的 Arc，通过它们可以访问共享的数组。当两个线程各自丢弃（drop）其 Arc 时，引用计数都会减少。最后一个丢弃其 Arc 的线程会看到引用计数降为 0，并且它将负责丢弃并释放（deallocate）这个数组。

Naming Clones

Having to give every clone of an `Arc` a different name can quickly make the code quite cluttered and hard to follow. While every clone of an `Arc` is a separate object, each clone represents the same shared value, which is not well reflected by naming each one differently.

Rust allows (and encourages) you to *shadow* variables by defining a new variable with the same name. If you do that in the same scope, the original variable cannot be named anymore. But by opening a new scope, a statement like `let a = a.clone();` can be used to reuse the same name within that scope, while leaving the original variable available outside the scope.

By wrapping a closure in a new scope (with `{}`), we can clone variables before moving them into the closure, without having to rename them.

<pre>let a = Arc::new([1, 2, 3]); let b = a.clone(); thread::spawn(move { dbg!(b); }); dbg!(a);</pre>	<pre>let a = Arc::new([1, 2, 3]); thread::spawn({ let a = a.clone(); move { dbg!(a); } }); dbg!(a);</pre>
The clone of the <code>Arc</code> lives in the same scope. Each thread gets its own clone with a different name.	The clone of the <code>Arc</code> lives in a different scope. We can use the same name in each thread.

Because ownership is shared, reference counting pointers (`Rc<T>` and `Arc<T>`) have the same restrictions as shared references (`&T`). They do not give you mutable access to their contained value, since the value might be borrowed by other code at the same time.

For example, if we were to try to sort the slice of integers in an `Arc<[i32]>`, the compiler would stop us from doing so, telling us that we're not allowed to mutate the data:

```
error[E0596]: cannot borrow data in an `Arc` as mutable
  |
6 |     a.sort();
  |     ^^^^^^^
```

Clone 的命名

如果必须给 `Arc` 的每一个 `Clone` 实例起不同的名称，很快就会导致代码变得非常凌乱且难以理解。虽然 `Arc` 的每个 `Clone` 实例都是一个独立的对象，但每个 `Clone` 都代表同一个共享值，而为每个实例起不同的名称并不能很好地反映这一点。

Rust 允许（并且鼓励）你通过定义一个同名的新变量来遮蔽 (*shadow*) 变量。如果你在同一个作用域中这样做，原始变量就无法再被访问。但通过开启一个新的作用域，像 `let a = a.clone();`；这样的语句可以在该作用域内重复使用相同的名称，同时在作用域之外保留原始变量。

通过将闭包包裹在一个新的作用域中（使用 `{}`），我们可以在将变量移动到闭包之前对其进行 `Clone`，而无需对其重新命名。

<pre>let a = Arc::new([1, 2, 3]); let b = a.clone(); thread::spawn(move { dbg!(b); }); dbg!(a);</pre>	<pre>let a = Arc::new([1, 2, 3]); thread::spawn({ let a = a.clone(); move { dbg!(a); } }); dbg!(a);</pre>
<code>Arc</code> 的 <code>Clone</code> 存在于同一个作用域中。每个线程都会获得一个具有不同名称的独立 <code>Clone</code> 。	<code>Arc</code> 的 <code>Clone</code> 存在于一个不同的作用域中。我们在每个线程中可以使用相同的名称。

由于所有权是共享的，引用计数指针 (`Rc<T>` and `Arc<T>`) 具有与共享引用 (`&T`) 相同的限制。它们不会向您提供对其包含值的可变访问，因为该值可能同时被其他代码借用。

例如，如果我们尝试对 `Arc<[i32]>` 中的整数切片进行排序，编译器会阻止我们这样做，并告知我们不允许修改该数据：

```
error[E0596]: cannot borrow data in an `Arc` as mutable
  |
6 |     a.sort();
  |     ^^^^^^^
```

Borrowing and Data Races

In Rust, values can be borrowed in two ways:

Immutable borrowing

Borrowing something with `&` gives an *immutable reference*. Such a reference can be copied. Access to the data it references is shared between all copies of such a reference. As the name implies, the compiler doesn't normally allow you to *mutate* something through such a reference, since that might affect other code that's currently borrowing the same data.

Mutable borrowing

Borrowing something with `&mut` gives a *mutable reference*. A mutable borrow guarantees it's the only active borrow of that data. This ensures that mutating the data will not change anything that other code is currently looking at.

These two concepts together fully prevent *data races*: situations where one thread is mutating data while another is concurrently accessing it. Data races are generally *undefined behavior*, which means the compiler does not need to take these situations into account. It will simply assume they do not happen.

To clarify what that means, let's take a look at an example where the compiler can make a useful assumption using the borrowing rules:

```
fn f(a: &i32, b: &mut i32) {  
    let before = *a;  
    *b += 1;  
    let after = *a;  
    if before != after {  
        x(); // never happens  
    }  
}
```

Here, we get an immutable reference to an integer, and store the value of the integer both before and after incrementing the integer that `b` refers to. The compiler is free to assume that the fundamental rules about borrowing and data races are upheld, which means that `b` can't possibly refer to the same integer as `a` does. In fact, nothing in the entire program can mutably borrow the integer that `a` refers to as long as `a` is borrowing it. Therefore, the compiler can easily conclude that `*a` will not change and the condition of the `if` statement will never be true, and can completely remove the call to `x` from the program as an optimization.

It's impossible to write a Rust program that breaks the compiler's assumptions, other than by using an `unsafe` block to disable some of the compiler's safety checks.

借用与数据竞态

在 Rust 中，值可以通过两种方式被借用：

不可变借用

使用 `&` 借用某个对象会产生一个不可变引用。这种引用可以被复制。对该引用所指向数据的访问在所有此类引用的副本之间是共享的。顾名思义，编译器通常不允许你通过这种引用来修改某个对象，因为这可能会影响到当前正在借用相同数据的其他代码。

可变借用

使用 `&mut` 借用某个对象会产生一个可变引用。可变借用保证它是该数据的唯一活跃借用。这确保了修改数据不会改变其他代码当前正在查看的内容。

这两个概念共同地完全防止了数据竞态：即一个线程在修改数据而另一个线程在并发访问该数据的场景。数据竞态通常是未定义行为，这意味着编译器不需要考虑这些情况，而会简单地假设它们不会发生。

为了明确这意味着什么，让我们来看一个编译器可以利用借用规则做出有用假设的示例：

```
fn f(a: &i32, b: &mut i32) {  
    let before = *a;  
    *b += 1;  
    let after = *a;  
    if before != after {  
        x(); // never happens  
    }  
}
```

在这里，我们获取了一个整数的不可变引用，并在对 `b` 所指的整数进行递增之前和之后，分别存储该整数的值。编译器可以自由地假设关于借用与数据竞态的基本规则得到了维护，这意味着 `b` 不可能与 `a` 指向同一个整数。事实上，只要 `a` 正在借用该整数，整个程序中的任何部分都不能对 `a` 所指的整数进行可变借用。因此，编译器可以轻易地得出结论，即 `*a` 不会改变，且 `if` 语句的条件永远不会为真，并可以将对 `x` 的调用作为优化而从程序中完全删除。

除了使用 `unsafe` 块来禁用编译器的一些安全检查之外，不可能编写一个能够破坏编译器假设的 Rust 程序。

Undefined Behavior

Languages like C, C++, and Rust have a set of rules that need to be followed to avoid something called *undefined behavior*. For example, one of Rust’s rules is that there may never be more than one mutable reference to any object.

In Rust, it’s only possible to break any of these rules when using `unsafe` code. “Unsafe” doesn’t mean that the code is incorrect or never safe to use, but rather that the compiler is not validating for you that the code is safe. If the code does violate these rules, it is called *unsound*.

The compiler is allowed to assume, without checking, that these rules are never broken. When broken, this results in something called *undefined behavior*, which we need to avoid at all costs. If we allow the compiler to make an assumption that is not actually true, it can easily result in more wrong conclusions about different parts of your code, affecting your whole program.

As a concrete example, let’s take a look at a small snippet that uses the `get_unchecked` method on a slice:

```
let a = [123, 456, 789];
let b = unsafe { a.get_unchecked(index) };
```

The `get_unchecked` method gives us an element of the slice given its index, just like `a[index]`, but allows the compiler to assume the index is always within bounds, without any checks.

This mean that in this code snippet, because `a` is of length 3, the compiler may assume that `index` is less than three. It’s up to us to make sure its assumption holds.

If we break this assumption, for example if we run this with `index` equal to 3, anything might happen. It might result in reading from memory whatever was stored in the bytes right after `a`. It might cause the program to crash. It might end up executing some entirely unrelated part of the program. It can cause all kinds of havoc.

Perhaps surprisingly, undefined behavior can even “travel back in time,” causing problems in code that precedes it. To understand how that can happen, imagine we had a `match` statement before our previous snippet, as follows:

```
match index {
  0 => x(),
  1 => y(),
  _ => z(index),
}

let a = [123, 456, 789];
let b = unsafe { a.get_unchecked(index) };
```

Because of the `unsafe` code, the compiler is allowed to assume `index` is only ever 0, 1, or 2. It may logically conclude that the last arm of our `match` statement will only

未定义行为

像 C 语言、C++ 和 Rust 这样的语言有一套需要遵守的规则，以避免所谓的 未定义 行为。例如，Rust 的其中一条规则是：对于任何对象，绝不能拥有超过一个可变引用。

在Rust 中，只有在使用 `unsafe` 代码时才有可能破坏这些规则。“Unsafe”并不意味着代码是不正确的或永远不能安全使用，而是意味着编译器不再为你验证代码是否安全。如果代码违反了这些规则，则被称为不健全的。

编译器被允许在不检查的情况下假设这些规则从未被破坏。一旦被破坏，就会导致所谓的未定义行为，这是我们需要不惜一切代价避免的。如果我们允许编译器做出一个实际上并不成立的假设，它很容易会对代码的不同部分产生更多错误的结论，从而影响你的整个程序。

作为一个具体的例子，让我们来看一段使用切片 `get_unchecked` 方法的短代码：

```
let a = [123, 456, 789];
let b = unsafe { a.get_unchecked(index) };
```

`get_unchecked` 方法根据索引为我们提供切片中的一个元素，就像 `a[index]`一样，但它允许编译器在不进行任何检查的情况下，假设索引始终在边界范围内。

这意味着在这段代码片段中，因为 `a` 的长度为 3，编译器可能会假设 `index` 小于 3。确保这个假设成立是由我们负责的。

如果我们打破这个假设，例如在 `index` 等于 3 的情况下运行，可能会发生任何事情。这可能会导致从 `a` 之后的字节中读取存储在内存中的任何内容。它可能会导致程序崩溃。它最终可能会执行程序中某些完全不相关的部分。它可以造成各种破坏。

也许令人惊讶的是，未定义行为甚至可以“穿越回过去”，导致在其之前的代码中出现问题。为了理解这是如何发生的，假设我们在之前的代码片段之前有一条 `match` 语句，如下所示：

```
match index {
  0 => x(),
  1 => y(),
  _ => z(index),
}

let a = [123, 456, 789];
let b = unsafe { a.get_unchecked(index) };
```

由于存在 `unsafe` 代码，编译器可以假设 `index` 只可能取值为 0、1 或 2。它可能会在逻辑上推断，`match` 语句的最后一个分支只会匹配到 2，因此 `z` 只会被以

ever match a 2, and thus that `z` is only ever called as `z(2)`. That conclusion might be used not only to optimize the `match`, but also to optimize `z` itself. This can include throwing out unused parts of the code.

If we execute this with an `index` of 3, our program might attempt to execute parts that have been optimized away, resulting in completely unpredictable behavior, long before we get to the `unsafe` block on the last line. Just like that, undefined behavior can propagate through a whole program, both backwards and forwards, in often very unexpected ways.

When calling any `unsafe` function, read its documentation carefully and make sure you fully understand its *safety requirements*: the assumptions you need to uphold, as the caller, to avoid undefined behavior.

Interior Mutability

The borrowing rules as introduced in the previous section are simple, but can be quite limiting—especially when multiple threads are involved. Following these rules makes communication between threads extremely limited and almost impossible, since no data that’s accessible by multiple threads can be mutated.

Luckily, there is an escape hatch: *interior mutability*. A data type with interior mutability slightly bends the borrowing rules. Under certain conditions, those types can allow mutation through an “immutable” reference.

In “[Reference Counting](#)” on page 8, we’ve already seen one subtle example involving interior mutability. Both `Rc` and `Arc` mutate a reference counter, even though there might be multiple clones all using the same reference counter.

As soon as interior mutable types are involved, calling a reference “immutable” or “mutable” becomes confusing and inaccurate, since some things can be mutated through both. The more accurate terms are “shared” and “exclusive”: a *shared reference* (`&T`) can be copied and shared with others, while an *exclusive reference* (`&mut T`) guarantees it’s the only *exclusive borrowing* of that `T`. For most types, shared references do not allow mutation, but there are exceptions. Since in this book we will mostly be working with these exceptions, we’ll use the more accurate terms in the rest of this book.



Keep in mind that interior mutability only bends the rules of shared borrowing to allow mutation when shared. It does not change anything about exclusive borrowing. Exclusive borrowing still guarantees that there are no other active borrows. Unsafe code that results in more than one active exclusive reference to something always invokes undefined behavior, regardless of interior mutability.

`z(2)` 的形式调用。这一结论不仅可能被用于优化 `match`，还可能被用于优化 `z` 本身。这其中可能包括剔除不使用的代码部分。

如果我们使用 `index 3` 来执行，我们的程序可能会尝试执行已被优化掉的部分，从而导致完全不可预测的行为，而这一切在到达最后一行的 `unsafe` 块之前就已经发生了。就这样，未定义行为可以以通常非常出人意料的方式，在整个程序中向前或向后传播。

在调用任何 `unsafe` 函数时，请仔细阅读其文档并确保你完全理解其**安全要求**：即作为调用者，为了避免未定义行为而需要遵守的假设。

内部可变性

前一节中介绍的借用规则很简单，但可能会非常具有限制性——尤其是当涉及多个线程时。遵循这些规则使得线程之间的通信极其受限且几乎不可能，因为任何可被多个线程访问的数据都不能被变异。

幸运的是，有一个逃生口：内部可变性。具有内部可变性的数据类型稍微弯曲了借用规则。在某些条件下，这些类型允许通过一个“不可变”引用进行变异。

在[第 8 页](#)的“引用计数”部分中，我们已经看到一个涉及内部可变性的微妙示例。和 `Arc` 都会对引用计数器进行变异，尽管可能存在多个 `Clone` 都在使用同一个引用计数器。

一旦涉及内部可变类型，将引用称为“不可变”或“可变”就会变得混乱且不准确，因为某些内容可以通过两者进行变异。更准确的术语是“共享”和“独占”：共享引用 (`&T`) 可以被复制并与其他人共享，而独占引用 (`&mut T`) 则保证它是该 `T` 的唯一独占借用。对于大多数类型，共享引用不允许变异，但也有例外。由于本书将主要处理这些例外，因此在接下来的内容中，我们将使用更准确的术语。



请记住，内部可变性仅仅是通过弯曲共享借用的规则，从而允许在共享时进行变异。它并没有改变关于独占借用的任何内容。独占借用仍然保证没有其他活跃的借用。无论是否具有内部可变性，导致对某个对象存在多个活跃独占引用的不安全代码始终会引发未定义行为。

Let’s take a look at a few types with interior mutability and how they can allow mutation through shared references without causing undefined behavior.

Cell

A `std::cell::Cell<T>` simply wraps a `T`, but allows mutations through a shared reference. To avoid undefined behavior, it only allows you to copy the value out (if `T` is `Copy`), or replace it with another value as a whole. In addition, it can only be used within a single thread.

Let’s take a look at an example similar to the one in the previous section, but this time using `Cell<i32>` instead of `i32`:

```
use std::cell::Cell;

fn f(a: &Cell<i32>, b: &Cell<i32>) {
    let before = a.get();
    b.set(b.get() + 1);
    let after = a.get();
    if before != after {
        x(); // might happen
    }
}
```

Unlike last time, it is now possible for the `if` condition to be true. Because a `Cell<i32>` has interior mutability, the compiler can no longer assume its value won’t change as long as we have a shared reference to it. Both `a` and `b` might refer to the same value, such that mutating through `b` might affect `a` as well. It may still assume, however, that no other threads are accessing the cells concurrently.

The restrictions on a `Cell` are not always easy to work with. Since it can’t directly let us borrow the value it holds, we need to move a value out (leaving something in its place), modify it, then put it back, to mutate its contents:

```
fn f(v: &Cell<Vec<i32>>) {
    let mut v2 = v.take(); // Replaces the contents of the Cell with an empty Vec
    v2.push(1);
    v.set(v2); // Put the modified Vec back
}
```

RefCell

Unlike a regular `Cell`, a `std::cell::RefCell` does allow you to borrow its contents, at a small runtime cost. A `RefCell<T>` does not only hold a `T`, but also holds a counter that keeps track of any outstanding borrows. If you try to borrow it while it is already mutably borrowed (or vice-versa), it will panic, which avoids undefined behavior. Just like a `Cell`, a `RefCell` can only be used within a single thread.

让我们来看看几个具有内部可变性的类型，以及它们如何允许通过共享引用进行变异而不会导致未定义行为。

Cell

`std::cell::Cell<T>` 简单地封装了一个 `T`，但允许通过共享引用进行变异。为了避免未定义行为，它仅允许你将值复制出来（如果 `T` 实现了 `Copy`），或者将其整体替换为另一个值。此外，它只能在单线程中使用。

让我们看一个与前一节类似的示例，但这次使用 `Cell<i32>` 而不是 `i32`:

```
use std::cell::Cell;

fn f(a: &Cell<i32>, b: &Cell<i32>) {
    let before = a.get();
    b.set(b.get() + 1);
    let after = a.get();
    if before != after {
        x(); // might happen
    }
}
```

与上次不同，现在 `if` 条件有可能为真。因为 `Cell<i32>` 具有内部可变性，编译器不再能假设只要我们持有它的共享引用，其值就不会改变。`a` 和 `b` 可能指向同一个值，因此通过 `b` 进行变异可能会同时影响 `a`。不过，编译器仍然可以假设没有其他线程在并发访问这些 `Cell`。

`Cell` 的限制并不总是那么容易处理。由于它不能直接让我们借用其持有的值，为了变更其内容，我们需要将一个值移出（在原处留下一个替代值），对其进行修改，然后将其放回：

```
fn f(v: &Cell<Vec<i32>>) {
    let mut v2 = v.take(); // Replaces the contents of the Cell with an empty Vec
    v2.push(1);
    v.set(v2); // Put the modified Vec back
}
```

RefCell

与普通的 `Cell` 不同，`std::cell::RefCell` 允许你借用其内容，但会产生少量的运行时开销。`RefCell<T>` 不仅持有 `T`，还持有一个用于跟踪所有未归还借用的计数器。如果你在它已经被可变借用时尝试再次借用（反之亦然），它会 panic，从而避免未定义行为。与 `Cell` 一样，`RefCell` 只能在单线程中使用。

Borrowing the contents of `RefCell` is done by calling `borrow` or `borrow_mut`:

```
use std::cell::RefCell;

fn f(v: &RefCell<Vec<i32>>) {
    v.borrow_mut().push(1); // We can modify the `Vec` directly.
}
```

While `Cell` and `RefCell` can be very useful, they become rather useless when we need to do something with multiple threads. So let's move on to the types that are relevant for concurrency.

Mutex and RwLock

An `RwLock` or *reader-writer lock* is the concurrent version of a `RefCell`. An `RwLock<T>` holds a `T` and tracks any outstanding borrows. However, unlike a `RefCell`, it does not panic on conflicting borrows. Instead, it blocks the current thread—putting it to sleep—while waiting for conflicting borrows to disappear. We'll just have to patiently wait for our turn with the data, after the other threads are done with it.

Borrowing the contents of an `RwLock` is called *locking*. By *locking* it we temporarily block concurrent conflicting borrows, allowing us to borrow it without causing data races.

A `Mutex` is very similar, but conceptually slightly simpler. Instead of keeping track of the number of shared and exclusive borrows like an `RwLock`, it only allows exclusive borrows.

We'll go more into detail on these types in “[Locking: Mutexes and RwLocks](#)” on page 18.

Atomics

The atomic types represent the concurrent version of a `Cell`, and are the main topic of Chapters 2 and 3. Like a `Cell`, they avoid undefined behavior by making us copy values in and out as a whole, without letting us borrow the contents directly.

Unlike a `Cell`, though, they cannot be of arbitrary size. Because of this, there is no generic `Atomic<T>` type for any `T`, but there are only specific atomic types such as `AtomicU32` and `AtomicPtr<T>`. Which ones are available depends on the platform, since they require support from the processor to avoid data races. (We'll dive into that in [Chapter 7](#).)

Since they are so limited in size, atomics often don't directly contain the information that needs to be shared between threads. Instead, they are often used as a tool to make it possible to share other—often bigger—things between threads. When

B借用 `RefCell` 的内容是通过调用 `borrow` 或 `borrow_mut` 来实现的:

```
use std::cell::RefCell;

fn f(v: &RefCell<Vec<i32>>) {
    v.borrow_mut().push(1); // We can modify the `Vec` directly.
}
```

虽然 `Cell` 和 `RefCell` 非常有用, 但当我们处理多个线程时, 它们就显得相当无能为力了。所以, 让我们转向与并发相关的类型。

互斥锁 and 读写锁

`RwLock` 或 读写锁 是 `RefCell` 的并发版本。`RwLock<T>` 持有一个 `T` 并跟踪任何尚未结束的借用。然而, 与 `RefCell` 不同的是, 它在出现冲突借用时不会 panic。相反, 它会阻塞当前线程 (使其进入睡眠状态), 直到冲突的借用消失。我们只需要耐心地等待其他线程完成对数据的操作, 然后轮到我们。

借用 `RwLock` 的内容被称为 加锁。通过对其进行 锁定, 我们可以暂时阻止并发的冲突借用, 从而在不引起数据竞争的情况下对其进行借用。

`Mutex` 非常相似, 但在概念上稍微简单一些。与 `RwLock` 不同, 它不需要跟踪共享借用和独占借用的数量, 而仅允许独占借用。

我们将在第 18 页的 “[加锁: 互斥锁和读写锁](#)” 中详细讨论这些类型。

原子操作

原子类型代表了 的并发版本, 并且是第 2 章和第 3 章的主要内容。与 一样, 它们通过强制我们将值整体地复制进出, 而不允许我们直接借用其内容, 从而避免了未定义行为。

不过, 与 不同的是, 它们不能具有任意的大小。因此, 并没有针对任何 的泛型 类型, 而只有特定的原子类型, 例如 和 。具体有哪些可用类型取决于平台, 因为它们需要处理器的支持才能避免数据竞争。(我们将在第 7 章中深入探讨这一点。)

由于原子操作的大小非常有限, 它们通常不直接包含需要在线程之间共享的信息。相反, 它们经常被用作一种工具, 从而使在线程之间共享其他 (通常是更大的) 内容成为可能。当

atomics are used to say something about other data, things can get surprisingly complicated.

UnsafeCell

An `UnsafeCell` is the primitive building block for interior mutability.

An `UnsafeCell<T>` wraps a `T`, but does not come with any conditions or restrictions to avoid undefined behavior. Instead, its `get()` method just gives a raw pointer to the value it wraps, which can only be meaningfully used in `unsafe` blocks. It leaves it up to the user to use it in a way that does not cause any undefined behavior.

Most commonly, an `UnsafeCell` is not used directly, but wrapped in another type that provides safety through a limited interface, such as `Cell` or `Mutex`. All types with interior mutability—including all types discussed above—are built on top of `UnsafeCell`.

Thread Safety: Send and Sync

In this chapter, we’ve seen several types that are not *thread safe*, types that can only be used on a single thread, such as `Rc`, `Cell`, and others. Since that restriction is needed to avoid undefined behavior, it’s something the compiler needs to understand and check for you, so you can use these types without having to use `unsafe` blocks.

The language uses two special traits to keep track of which types can be safely used across threads:

Send

A type is `Send` if it can be sent to another thread. In other words, if ownership of a value of that type can be transferred to another thread. For example, `Arc<i32>` is `Send`, but `Rc<i32>` is not.

Sync

A type is `Sync` if it can be shared with another thread. In other words, a type `T` is `Sync` if and only if a shared reference to that type, `&T`, is `Send`. For example, an `i32` is `Sync`, but a `Cell<i32>` is not. (A `Cell<i32>` is `Send`, however.)

All primitive types such as `i32`, `bool`, and `str` are both `Send` and `Sync`.

Both of these traits are *auto traits*, which means that they are automatically implemented for your types based on their fields. A `struct` with fields that are all `Send` and `Sync`, is itself also `Send` and `Sync`.

The way to opt out of either of these is to add a field to your type that does not implement the trait. For that purpose, the special `std::marker::PhantomData<T>`

的) 内容成为可能。当原子操作被用来描述其他数据时, 情况可能会变得出人意料地复杂。

UnsafeCell

`UnsafeCell` 是实现内部可变性的基础构建块。

`UnsafeCell<T>` 封装了一个 `T`, 但它没有提供任何条件或限制来避免未定义行为。相反, 它的 `get()` 方法仅提供该封装值的原始指针, 而该指针只能在 `unsafe` 块中有意义地使用。它将如何使用且不导致任何未定义行为的任务交给了用户。

最常见的情况是, `UnsafeCell` 不被直接使用, 而是被封装在另一个通过有限接口提供安全保障的类型中, 例如 `Cell` 或 `Mutex`。所有具有内部可变性的类型——包括上述讨论的所有类型——都是构建在 `UnsafeCell` 之上的。

线程安全: Send 与 Sync

在本章中, 我们已经看到了几种不是 线程安全 的类型, 这些类型只能在单线程中使用, 例如 `Rc`、`Cell` 等。由于这种限制对于避免未定义行为是必要的, 因此编译器需要理解并为你检查这一点, 这样你就可以在无需使用 `unsafe` 块的情况下使用这些类型。

该语言使用两个特殊的 trait 来跟踪哪些类型可以在线程之间安全地使用:

Send

如果一个类型可以被发送到另一个线程, 那么它就是 `Send`。换句话说, 如果该类型的值的所有权可以转移到另一个线程, 它就是 `Send`。例如, `Arc<i32>` 是 `Send`, 但 `Rc<i32>` 则不是。

Sync

如果一个类型可以与另一个线程共享, 那么它就是 `Sync`。换句话说, 一个类型 `t` 是 `Sync`, 当且仅当该类型的共享引用 `&t` 是 `Send`。例如, `i32` 是 `Sync`, 但 `Cell<i32>` 则不是。(不过, `Cell<i32>` 是 `Send`。)

所有原始类型(如 `i32`、`bool` 和 `str`) 既是 `Send` 也是 `Sync`。

这两个 trait 都是 自动 *trait*, 这意味着它们会根据类型的字段自动为你的类型实现。如果一个 `struct` 的所有字段都是 `Send` 且是 `Sync`, 那么它本身也是 `Send` 且是 `Sync`。

要选择不使用其中任何一种的方法, 是在你的类型中添加一个没有实现该 trait 的字段。为此, 特殊的 `std::marker::PhantomData<T>` 类型通常会派上用场。该

type often comes in handy. That type is treated by the compiler as a `T`, except it doesn't actually exist at runtime. It's a zero-sized type, taking no space.

Let's take a look at the following struct:

```
use std::marker::PhantomData;

struct X {
    handle: i32,
    _not_sync: PhantomData<Cell<()>>,
}
```

In this example, `X` would be both `Send` and `Sync` if `handle` was its only field. However, we added a zero-sized `PhantomData<Cell<()>>` field, which is treated as if it were a `Cell<()>`. Since a `Cell<()>` is not `Sync`, neither is `X`. It is still `Send`, however, since all its fields implement `Send`.

Raw pointers (`*const T` and `*mut T`) are neither `Send` nor `Sync`, since the compiler doesn't know much about what they represent.

The way to opt in to either of the traits is the same as with any other trait; use an `impl` block to implement the trait for your type:

```
struct X {
    p: *mut i32,
}

unsafe impl Send for X {}
unsafe impl Sync for X {}
```

Note how implementing these traits requires the `unsafe` keyword, since the compiler cannot check for you if it's correct. It's a promise you make to the compiler, which it will just have to trust.

If you try to move something into another thread which is not `Send`, the compiler will politely stop you from doing that. Here is a small example to demonstrate that:

```
fn main() {
    let a = Rc::new(123);
    thread::spawn(move || { // Error!
        dbg!(a);
    });
}
```

Here, we try to send an `Rc<i32>` to a new thread, but `Rc<i32>`, unlike `Arc<i32>`, does not implement `Send`.

类型被编译器视为 `T`，但它在运行时实际上并不存在。它是一个零大小类型，不占用空间。

让我们来看看下面的 struct:

```
use std::marker::PhantomData;

struct X {
    handle: i32,
    _not_sync: PhantomData<Cell<()>>,
}
```

在这个例子中，如果 `handle` 是其唯一的字段，那么 `x` 将同时是 `Send` 和 `Sync`。然而，我们添加了一个零大小的 `PhantomData<Cell<()>>` 字段，该字段被视为一个 `Cell<()>`。由于 `Cell<()>` 不是 `Sync`，因此 `x` 也不具备该特性。不过，由于其所有字段都实现了 `Send`，它仍然是 `Send`。

原始指针 (`*const T` 和 `*mut T`) 既不是 `Send` 也不是 `Sync`，因为编译器并不太了解它们代表的内容。

选择实现这两个 trait 中的任何一个的方法与实现其他任何 trait 相同；使用 `impl` 块为您的类型实现该 trait:

```
struct X {
    p: *mut i32,
}

unsafe impl Send for X {}
unsafe impl Sync for X {}
```

请注意，实现这些 trait 需要使用 `unsafe` 关键字，因为编译器无法为您检查其正确性。这是您向编译器做出的承诺，而编译器只能选择信任它。

如果你尝试将一个并非 `Send` 的对象移动到另一个线程中，编译器会礼貌地阻止你这样做。这里有一个简单的示例来演示这一点:

```
fn main() {
    let a = Rc::new(123);
    thread::spawn(move || { // Error!
        dbg!(a);
    });
}
```

在这里，我们尝试将 `Rc<i32>` 发送到一个新的线程，但与 `Arc<i32>` 不同，`Rc<i32>` 没有实现 `Send`。

If we try to compile the example above, we’re faced with an error that looks something like this:

```
error[E0277]: `Rc<i32>` cannot be sent between threads safely
--> src/main.rs:3:5
  |
3 | |     thread::spawn(move || {
  | |         ^^^^^^^^^^^^^^^ `Rc<i32>` cannot be sent between threads safely
  | |
  | = help: within `[closure]`, the trait `Send` is not implemented for `Rc<i32>`
note: required because it's used within this closure
--> src/main.rs:3:19
  |
3 | |     thread::spawn(move || {
  | |         ^^^^^^^
  |
note: required by a bound in `spawn`
```

The `thread::spawn` function requires its argument to be `Send`, and a closure is only `Send` if all of its captures are. If we try to capture something that’s not `Send`, our mistake is caught, protecting us from undefined behavior.

Locking: Mutexes and RwLocks

The most commonly used tool for sharing (mutable) data between threads is a *mutex*, which is short for “mutual exclusion.” The job of a mutex is to ensure threads have exclusive access to some data by temporarily blocking other threads that try to access it at the same time.

Conceptually, a mutex has only two states: locked and unlocked. When a thread locks an unlocked mutex, the mutex is marked as locked and the thread can immediately continue. When a thread then attempts to lock an already locked mutex, that operation will *block*. The thread is put to sleep while it waits for the mutex to be unlocked. Unlocking is only possible on a locked mutex, and should be done by the same thread that locked it. If other threads are waiting to lock the mutex, unlocking will cause one of those threads to be woken up, so it can try to lock the mutex again and continue its course.

Protecting data with a mutex is simply the agreement between all threads that they will only access the data when they have the mutex locked. That way, no two threads can ever access that data concurrently and cause a data race.

Rust’s Mutex

The Rust standard library provides this functionality through `std::sync::Mutex<T>`. It is generic over a type `T`, which is the type of the data the mutex is protecting. By making this `T` part of the mutex, the data can only be accessed through the mutex, allowing for a safe interface that can guarantee all threads will uphold the agreement.

如果我们尝试编译上面的示例，将会遇到一个类似这样的错误：

```
error[E0277]: `Rc<i32>` cannot be sent between threads safely
--> src/main.rs:3:5
  |
3 | |     thread::spawn(move || {
  | |         ^^^^^^^^^^^^^^^ `Rc<i32>` cannot be sent between threads safely
  | |
  | = help: within `[closure]`, the trait `Send` is not implemented for `Rc<i32>`
note: required because it's used within this closure
--> src/main.rs:3:19
  |
3 | |     thread::spawn(move || {
  | |         ^^^^^^^
  |
note: required by a bound in `spawn`
```

`thread::spawn` 函数要求其参数必须是 `Send`，且只有在其所有捕获项都是 `Send` 时，闭包才是 `Send`。如果我们尝试捕获 `Send` 的项，我们的错误将被捕捉，从而保护我们免于未定义行为。

加锁：互斥锁 与 读写锁

在线程之间共享(可变)数据最常用的工具是**互斥锁**，它是 "mutual exclusion" 的缩写。互斥锁的作用是通过暂时阻塞尝试同时访问数据的其他线程，来确保线程对某些数据拥有独占访问权限。

从概念上讲，互斥锁只有两种状态：锁定和解锁。当一个线程锁定一个未锁定的互斥锁时，该互斥锁被标记为锁定状态，且该线程可以立即继续执行。当一个线程随后尝试锁定一个已经锁定的互斥锁时，该操作将会阻塞。该线程将进入睡眠状态，直到互斥锁被解锁。只有在互斥锁被锁定时才能执行解锁操作，并且应该由锁定它的同一个线程来执行。如果其他线程正在等待锁定该互斥锁，解锁操作将导致其中一个线程被唤醒，从而使其能够再次尝试锁定互斥锁并继续执行。

使用互斥锁保护数据的本质，就是所有线程之间达成的一项协议：只有在锁定互斥锁的情况下才能访问数据。这样，就没有两个线程能并发地访问该数据，从而避免产生数据竞态。

Rust 的 Mutex

Rust 标准库通过 `std::sync::Mutex<T>` 提供了这一功能。它是针对类型 `T` 的泛型，而 `T` 即为互斥锁所保护的数据类型。通过将 `T` 作为互斥锁的一部分，数据只能通过互斥锁进行访问，从而提供一个能够确保所有线程都遵守协议的安全接口。

To ensure a locked mutex can only be unlocked by the thread that locked it, it does not have an `unlock()` method. Instead, its `lock()` method returns a special type called a `MutexGuard`. This guard represents the guarantee that we have locked the mutex. It behaves like an exclusive reference through the `DerefMut` trait, giving us exclusive access to the data the mutex protects. Unlocking the mutex is done by dropping the guard. When we drop the guard, we give up our ability to access the data, and the `Drop` implementation of the guard will unlock the mutex.

Let's take a look at an example to see a mutex in practice:

```
use std::sync::Mutex;

fn main() {
    let n = Mutex::new(0);
    thread::scope(|s| {
        for _ in 0..10 {
            s.spawn(|| {
                let mut guard = n.lock().unwrap();
                for _ in 0..100 {
                    *guard += 1;
                }
            });
        }
    });
    assert_eq!(n.into_inner().unwrap(), 1000);
}
```

Here, we have a `Mutex<i32>`, a mutex protecting an integer, and we spawn ten threads to each increment the integer one hundred times. Each thread will first lock the mutex to obtain a `MutexGuard`, and then use that guard to access the integer and modify it. The guard is implicitly dropped right after, when that variable goes out of scope.

After the threads are done, we can safely remove the protection from the integer through `into_inner()`. The `into_inner` method takes ownership of the mutex, which guarantees that nothing else can have a reference to the mutex anymore, making locking unnecessary.

Even though the increments happen in steps of one, a thread observing the integer would only ever see multiples of 100, since it can only look at the integer when the mutex is unlocked. Effectively, thanks to the mutex, the one hundred increments together are now a single indivisible—atomic—operation.

为了确保被锁定的互斥锁只能由锁定它的线程解锁，它没有 `unlock()` 方法。相反，其 `lock()` 方法会返回一个名为 `MutexGuard` 的特殊类型。这个守护对象代表了我们已锁定互斥锁的保证。它通过 `DerefMut` trait 表现得像一个独占引用，赋予我们对互斥锁所保护数据的独占访问权。解锁互斥锁是通过 `Drop` 守护对象来实现的。当我们 `Drop` 守护对象时，我们便放弃了访问数据的能力，而守护对象的 `Drop` 实现将会解锁该互斥锁。

让我们看一个例子，看看互斥锁在实际中是如何应用的：

```
use std::sync::Mutex;

fn main() {
    let n = Mutex::new(0);
    thread::scope(|s| {
        for _ in 0..10 {
            s.spawn(|| {
                let mut guard = n.lock().unwrap();
                for _ in 0..100 {
                    *guard += 1;
                }
            });
        }
    });
    assert_eq!(n.into_inner().unwrap(), 1000);
}
```

在这里，我们有一个 `Mutex<i32>`，一个保护整数的互斥锁，并且我们 spawn 了十个线程，每个线程将该整数增加一百次。每个线程首先会锁定互斥锁以获取一个 `MutexGuard`，然后使用该 `guard` 来访问并修改该整数。当该变量超出作用域时，`guard` 会在随后被隐式丢弃。

在线程执行完毕后，我们可以通过 `into_inner()` 安全地移除对该整数的保护。`into_inner` 方法会获取互斥锁的所有权，这保证了没有任何其他内容能再持有该互斥锁的引用，从而使加锁变得不再必要。

尽管递增是以 1 为步长进行的，但观察该整数的线程将永远只能看到 100 的倍数，因为它只能在互斥锁解锁时查看该整数。实际上，多亏了互斥锁，这一百次递增现在共同成为了一个单一的不可分割的——原子的——操作。

To clearly see the effect of the mutex, we can make each thread wait a second before unlocking the mutex:

```
use std::time::Duration;

fn main() {
    let n = Mutex::new(0);
    thread::scope(|s| {
        for _ in 0..10 {
            s.spawn(|| {
                let mut guard = n.lock().unwrap();
                for _ in 0..100 {
                    *guard += 1;
                }
                thread::sleep(Duration::from_secs(1)); // New!
            });
        }
    });
    assert_eq!(n.into_inner().unwrap(), 1000);
}
```

When you run the program now, you will see that it takes about 10 seconds to complete. Each thread only waits for one second, but the mutex ensures that only one thread at a time can do so.

If we drop the guard—and therefore unlock the mutex—before sleeping one second, we will see it happen in parallel instead:

```
fn main() {
    let n = Mutex::new(0);
    thread::scope(|s| {
        for _ in 0..10 {
            s.spawn(|| {
                let mut guard = n.lock().unwrap();
                for _ in 0..100 {
                    *guard += 1;
                }
                drop(guard); // New: drop the guard before sleeping!
                thread::sleep(Duration::from_secs(1));
            });
        }
    });
    assert_eq!(n.into_inner().unwrap(), 1000);
}
```

With this change, this program takes only about one second, since now the 10 threads can execute their one-second sleep at the same time. This shows the importance of keeping the amount of time a mutex is locked as short as possible. Keeping a mutex locked longer than necessary can completely nullify any benefits of parallelism, effectively forcing everything to happen serially instead.

为了清晰地看到互斥锁的效果，我们可以让每个线程在解锁互斥锁之前等待一秒：

```
use std::time::Duration;

fn main() {
    let n = Mutex::new(0);
    thread::scope(|s| {
        for _ in 0..10 {
            s.spawn(|| {
                let mut guard = n.lock().unwrap();
                for _ in 0..100 {
                    *guard += 1;
                }
                thread::sleep(Duration::from_secs(1)); // New!
            });
        }
    });
    assert_eq!(n.into_inner().unwrap(), 1000);
}
```

现在运行该程序，你会发现它大约需要 10 秒才能完成。虽然每个线程只等待一秒，但互斥锁确保了同一时间只有一个线程可以执行此操作。

如果在睡眠一秒之前 Drop 守护对象——从而解锁互斥锁——我们将看到它们改为并行执行：

```
fn main() {
    let n = Mutex::new(0);
    thread::scope(|s| {
        for _ in 0..10 {
            s.spawn(|| {
                let mut guard = n.lock().unwrap();
                for _ in 0..100 {
                    *guard += 1;
                }
                drop(guard); // New: drop the guard before sleeping!
                thread::sleep(Duration::from_secs(1));
            });
        }
    });
    assert_eq!(n.into_inner().unwrap(), 1000);
}
```

经过这次修改，该程序仅需大约一秒钟即可运行完毕，因为现在 10 个线程可以同时执行一秒钟的休眠。这表明了将互斥锁锁定时间尽可能缩短的重要性。如果互斥锁锁定时间过长，可能会完全抵消并行化带来的任何好处，实际上会迫使所有操作串行地执行。

Lock Poisoning

The `unwrap()` calls in the examples above relate to *lock poisoning*.

A `Mutex` in Rust gets marked as *poisoned* when a thread panics while holding the lock. When that happens, the `Mutex` will no longer be locked, but calling its `lock` method will result in an `Err` to indicate it has been poisoned.

This is a mechanism to protect against leaving the data that’s protected by a `mutex` in an inconsistent state. In our example above, if a thread would panic after incrementing the integer fewer than 100 times, the `mutex` would unlock and the integer would be left in an unexpected state where it is no longer a multiple of 100, possibly breaking assumptions made by other threads. Automatically marking the `mutex` as poisoned in that case forces the user to handle this possibility.

Calling `lock()` on a poisoned `mutex` still locks the `mutex`. The `Err` returned by `lock()` contains the `MutexGuard`, allowing us to correct an inconsistent state if necessary.

While lock poisoning might seem like a powerful mechanism, recovering from a potentially inconsistent state is not often done in practice. Most code either disregards poison or uses `unwrap()` to panic if the lock was poisoned, effectively propagating panics to all users of the `mutex`.

Lifetime of the MutexGuard

While it’s convenient that implicitly dropping a guard unlocks the `mutex`, it can sometimes lead to subtle surprises. If we assign the guard a name with a `let` statement (as in our examples above), it’s relatively straightforward to see when it will be dropped, since local variables are dropped at the end of the scope they are defined in. Still, not explicitly dropping a guard might lead to keeping the `mutex` locked for longer than necessary, as demonstrated in the examples above.

Using a guard *without* assigning it a name is also possible, and can be very convenient at times. Since a `MutexGuard` behaves like an exclusive reference to the protected data, we can directly use it without assigning a name to the guard first. For example, if you have a `Mutex<Vec<i32>>`, you can lock the `mutex`, push an item into the `Vec`, and unlock the `mutex` again, in a single statement:

```
list.lock().unwrap().push(1);
```

Any temporaries produced within a larger expression, such as the guard returned by `lock()`, will be dropped at the end of the statement. While this might seem obvious and reasonable, it leads to a common pitfall that usually involves a `match`, `if let`, or `while let` statement. Here is an example that runs into this pitfall:

锁中毒

上述示例中的 `unwrap()` 调用与**锁中毒**相关。

在Rust中，当持有锁的线程发生panic时，`Mutex` 会被标记为**中毒 (poisoned)**。在这种情况下，`Mutex` 将不再被锁定，但调用其 `lock` 方法将返回一个 `Err`，以表明它已被毒化。

这是一种防止互斥锁保护的数据处于不一致状态的机制。在我们上面的示例中，如果一个线程在将整数增加不到 100 次后发生 panic，互斥锁将会解锁，而该整数将处于一个意外的状态（不再是 100 的倍数），这可能会破坏其他线程所做的假设。在这种情况下，自动将互斥锁标记为中毒，强制用户处理这种可能性。

对中毒的互斥锁调用 `lock()` 仍然会锁定该互斥锁。`lock()` 返回的 `Err` 包含了 `MutexGuard`，使我们能够在必要时修正不一致的状态。

虽然锁中毒看起来是一个强大的机制，但在实践中，从潜在的不一致状态中恢复的情况并不常见。大多数代码要么忽略毒化，要么使用 `unwrap()` 在锁被毒化时触发 panic，从而有效地将 panic 传播给互斥锁的所有使用者。

MutexGuard 的生命周期

虽然隐式地 `Drop` 一个守护对象可以解锁互斥锁，这很方便，但有时会导致一些微妙的意外。如果我们使用`let`语句为守护对象命名(如上面的示例所示)，那么相对很容易看出它何时被 `Drop`，因为局部变量在它们定义的作用域结束时会被 `Drop`。尽管如此，如上面的示例所示，不显式地 `Drop` 守护对象可能会导致互斥锁被锁定的时间比必要的更长。

在不为其命名的情况下使用守护对象也是可能的，而且有时会非常方便。由于 `MutexGuard` 的行为类似于指向受保护数据的独占引用，我们可以直接使用它，而无需先为守护对象命名。例如，如果你有一个 `Mutex<Vec<i32>>`，你可以在单个语句中锁定互斥锁，向 `Vec` 压入一个项，然后再次解锁互斥锁：

```
list.lock().unwrap().push(1);
```

在较大的表达式中产生的任何临时变量（例如由 `lock()` 返回的守护对象）都会在语句结束时被 `Drop`。虽然这看起来显而易见且合理，但它会导致一个常见的陷阱，通常涉及 `match`、`if let` 或 `while let` 语句。以下是一个陷入此陷阱的示例：

```
if let Some(item) = list.lock().unwrap().pop() {
    process_item(item);
}
```

If our intention was to lock the list, pop an item, unlock the list, and *then* process the item after the list is unlocked, we made a subtle but important mistake here. The temporary guard is not dropped until the end of the entire `if let` statement, meaning we needlessly hold on to the lock while processing the item.

Perhaps surprisingly, this does not happen for a similar `if` statement, such as in this example:

```
if list.lock().unwrap().pop() == Some(1) {
    do_something();
}
```

Here, the temporary guard does get dropped before the body of the `if` statement is executed. The reason is that the condition of a regular `if` statement is always a plain boolean, which cannot borrow anything. There is no reason to extend the lifetime of temporaries from the condition to the end of the statement. For an `if let` statement, however, that might not be the case. If we had used `front()` rather than `pop()`, for example, `item` would be borrowing from the list, making it necessary to keep the guard around. Since the borrow checker is only really a check and does *not* influence when or in what order things are dropped, the same happens when we use `pop()`, even though that wouldn't have been necessary.

We can avoid this by moving the pop operation to a separate `let` statement. Then the guard is dropped at the end of that statement, before the `if let`:

```
let item = list.lock().unwrap().pop();
if let Some(item) = item {
    process_item(item);
}
```

Reader-Writer Lock

A mutex is only concerned with exclusive access. The `MutexGuard` will provide us an exclusive reference (`&mut T`) to the protected data, even if we only wanted to look at the data and a shared reference (`&T`) would have sufficed.

A reader-writer lock is a slightly more complicated version of a mutex that understands the difference between exclusive and shared access, and can provide either. It has three states: unlocked, locked by a single *writer* (for exclusive access), and locked by any number of *readers* (for shared access). It is commonly used for data that is often read by multiple threads, but only updated once in a while.

The Rust standard library provides this lock through the `std::sync::RwLock<T>` type. It works similarly to the standard `Mutex`, except its interface is mostly split in

```
if let Some(item) = list.lock().unwrap().pop() {
    process_item(item);
}
```

如果我们的意图是锁定列表，弹出项，解锁列表，然后在列表解锁后处理该项，那么我们在这里犯了一个微妙但重要的错误。临时守护对象直到整个 `if let` 语句结束时才会被 Drop，这意味着我们在处理该项时毫无必要地持有锁定。

或许令人惊讶的是，对于类似的 `if` 语句，这种情况并不会发生，例如在本例中：

```
if list.lock().unwrap().pop() == Some(1) {
    do_something();
}
```

在这里，临时守护对象在 `if` 语句的正文执行之前确实被 Drop 了。原因是常规 `if` 语句的条件始终是一个简单的布尔值，它无法借用任何内容。因此没有理由将条件中的临时变量的生命周期延长到语句结束。然而，对于 `if let` 语句，情况可能并非如此。例如，如果我们使用 `front()` 而不是 `pop()`，`item` 将会从列表中进行借用，从而使得保留守护对象变得必要。由于借用检查器仅仅是一个检查过程，并*not*影响对象被 Drop 的时间或顺序，因此当我们使用 `pop()` 时也会发生同样的情况，尽管这其实并非必要。

我们可以通过将 pop 操作移到一个单独的 `let` 语句来避免这种情况。这样 `guard` 将在该语句结束时、在 `if let` 之前被 Drop：

```
let item = list.lock().unwrap().pop();
if let Some(item) = item {
    process_item(item);
}
```

读写锁

互斥锁仅关注独占访问。即使我们只想查看数据，且共享引用 (`&T`) 就足够了，`MutexGuard` 仍会为我们提供一个指向受保护数据的独占引用 (`&mut T`)。

读写锁是互斥锁的一个稍复杂版本，它能够区分独占访问和共享访问，并能提供其中任意一种。它有三种状态：解锁状态、被单个写入者锁定（用于独占访问），以及被任意数量的读者锁定（用于共享访问）。它通常用于那些经常被多个线程读取、但仅偶尔被更新的数据。

Rust 标准库通过 `std::sync::RwLock<T>` 类型提供了这种锁定。它的工作方式与标准的 `Mutex` 类似，不同之处在于其接口主要分为两部分。它没有单一的

two parts. Instead of a single `lock()` method, it has a `read()` and `write()` method for locking as either a reader or a writer. It comes with two guard types, one for readers and one for writers: `RwLockReadGuard` and `RwLockWriteGuard`. The former only implements `Deref` to behave like a shared reference to the protected data, while the latter also implements `DerefMut` to behave like an exclusive reference.

It is effectively the multi-threaded version of `RefCell`, dynamically tracking the number of references to ensure the borrow rules are upheld.

Both `Mutex<T>` and `RwLock<T>` require `T` to be `Send`, because they can be used to send a `T` to another thread. An `RwLock<T>` additionally requires `T` to also implement `Sync`, because it allows multiple threads to hold a shared reference (`&T`) to the protected data. (Strictly speaking, you can create a lock for a `T` that doesn't fulfill these requirements, but you wouldn't be able to share it between threads as the lock itself won't implement `Sync`.)

The Rust standard library provides only one general purpose `RwLock` type, but its implementation depends on the operating system. There are many subtle variations between reader-writer lock implementations. Most implementations will block new readers when there is a writer waiting, even when the lock is already read-locked. This is done to prevent *writer starvation*, a situation where many readers collectively keep the lock from ever unlocking, never allowing any writer to update the data.

Mutexes in Other Languages

Rust's standard `Mutex` and `RwLock` types look a bit different than those you find in other languages like C or C++.

The biggest difference is that Rust's `Mutex<T>` *contains* the data it is protecting. In C++, for example, `std::mutex` does not contain the data it protects, nor does it even know what it is protecting. This means that it is the responsibility of the user to remember which data is protected and by which mutex, and ensure the right mutex is locked every time “protected” data is accessed. This is useful to keep in mind when reading code involving mutexes in other languages, or when communicating with programmers who are not familiar with Rust. A Rust programmer might talk about “the data inside the mutex,” or say things like “wrap it in a mutex,” which can be confusing to those only familiar with mutexes in other languages.

If you really need a stand-alone mutex that doesn't contain anything, for example to protect some external hardware, you can use `Mutex<()>`. But even in a case like that, you are probably better off defining a (possibly zero-sized) type to interface with that hardware and wrapping that in a `Mutex` instead. That way, you are still forced to lock the mutex before you can interact with the hardware.

`lock()` 方法，而是拥有 `read()` 和 `write()` 方法，以便作为读者或写者进行加锁。它带有两种守护类型，一种用于读者，另一种用于写者：`RwLockReadGuard` 和 `RwLockWriteGuard`。前者仅实现了 `Deref`，以表现得像对受保护数据的共享引用，而后者还实现了 `DerefMut`，以表现得像独占引用。

它实际上是 `RefCell` 的多线程版本，通过动态追踪引用数量来确保借用规则得到遵守。

`Mutex<T>` 和 `RwLock<T>` 都要求 `T` 必须是 `Send`，因为它们可以被用于将 `T` 发送到另一个线程。`RwLock<T>` 还额外要求 `T` 必须实现 `Sync`，因为它允许多个线程持有指向受保护数据的共享引用(`&T`)。(严格来说，你可以为一个不满足这些要求的 `T` 创建锁，但你将无法在线程之间共享它，因为锁本身不会实现 `Sync`。)

Rust 标准库仅提供了一种通用目的的 `RwLock` 类型，但其具体实现取决于操作系统。不同的读写锁实现之间存在许多细微的差别。大多数实现在有写者等待时会阻塞新的读者，即使此时锁已被读锁锁定。这样做是为了防止写者饥饿，即许多读者共同导致锁永远无法解锁，从而使任何写者都无法更新数据的情况。

其他语言中的互斥锁

Rust 的标准 `Mutex` and `RwLock` 类型看起来与你在 C语言 或 C++ 等其他语言中看到的略有不同。

最大的区别在于 Rust 的 `Mutex<T>` 包含它所保护的数据。例如在 C++ 中，`std::mutex` not 包含它保护的数据，甚至不知道自己正在保护什么。这意味着用户有责任记住哪些数据被哪个互斥锁保护，并确保在每次访问“受保护”数据时都锁定正确的互斥锁。在阅读涉及其他语言互斥锁的代码，或者与不熟悉 Rust 的程序员交流时，这一点非常有用。Rust 程序员可能会谈论“互斥锁内部的数据”，或者说“用互斥锁将其包裹起来”，这对于只熟悉其他语言互斥锁的人来说可能会感到困惑。

如果你确实需要一个不包含任何内容的独立互斥锁，例如用来保护某些外部硬件，你可以使用 `Mutex<()>`。但即便在这种情况下，定义一个（可能是零大小的）类型来与该硬件交互，并将其封装在 `Mutex` 中，可能仍然是更好的选择。这样一来，在与硬件交互之前，你仍然会被强制要求锁定互斥锁。

Waiting: Parking and Condition Variables

When data is mutated by multiple threads, there are many situations where they would need to wait for some event, for some condition about the data to become true. For example, if we have a mutex protecting a Vec, we might want to wait until it contains anything.

While a mutex does allow threads to wait until it becomes unlocked, it does not provide functionality for waiting for any other conditions. If a mutex was all we had, we'd have to keep locking the mutex to repeatedly check if there's anything in the Vec yet.

Thread Parking

One way to wait for a notification from another thread is called *thread parking*. A thread can *park* itself, which puts it to sleep, stopping it from consuming any CPU cycles. Another thread can then *unpark* the parked thread, waking it up from its nap.

Thread parking is available through the `std::thread::park()` function. For unparking, you call the `unpark()` method on a Thread object representing the thread that you want to unpark. Such an object can be obtained from the join handle returned by `spawn`, or by the thread itself through `std::thread::current()`.

Let's dive into an example that uses a mutex to share a queue between two threads. In the following example, a newly spawned thread will consume items from the queue, while the main thread will insert a new item into the queue every second. Thread parking is used to make the consuming thread wait when the queue is empty.

```
use std::collections::VecDeque;

fn main() {
    let queue = Mutex::new(VecDeque::new());

    thread::scope(|s| {
        // Consuming thread
        let t = s.spawn(|| loop {
            let item = queue.lock().unwrap().pop_front();
            if let Some(item) = item {
                dbg!(item);
            } else {
                thread::park();
            }
        });

        // Producing thread
        for i in 0.. {
            queue.lock().unwrap().push_back(i);
            t.thread().unpark();
            thread::sleep(Duration::from_secs(1));
        }
    });
}
```

等待：线程挂起与条件变量

当数据被多个线程修改时，在许多情况下，它们需要等待某个事件，或者等待数据满足某个特定条件。例如，如果我们有一个互斥锁来保护一个 Vec，我们可能希望等待直到其中包含内容。

虽然互斥锁允许线程等待直到其解锁，但它并不提供等待任何其他条件的机制。如果只有互斥锁，我们就必须不断地锁定互斥锁，以重复检查 Vec 中是否已经有了内容。

线程挂起

一种等待另一个线程通知的方法被称为**线程挂起 (thread parking)**。一个线程可以使自己 *park*（挂起），这会使其进入睡眠状态，从而停止消耗任何 CPU 周期。随后，另一个线程可以 *unpark*（唤醒）该挂起的线程，将其从睡眠中唤醒。

可以通过 `std::thread::park()` 函数实现线程挂起。对于 unpark 操作，您需要对代表要唤醒线程的 Thread 对象调用 `unpark()` 方法。这样的对象可以通过 `spawn` 返回的连接句柄获取，或者由线程自身通过 `std::thread::current()` 获取。

让我们来看一个使用互斥锁在两个线程之间共享队列的示例。在接下来的示例中，一个新 spawn 的线程将从队列中 Consume 元素，而主线程将每秒向队列中插入一个新元素。线程挂起被用于在队列为空时让消费线程等待。

```
use std::collections::VecDeque;

fn main() {
    let queue = Mutex::new(VecDeque::new());

    thread::scope(|s| {
        // Consuming thread
        let t = s.spawn(|| loop {
            let item = queue.lock().unwrap().pop_front();
            if let Some(item) = item {
                dbg!(item);
            } else {
                thread::park();
            }
        });

        // Producing thread
        for i in 0.. {
            queue.lock().unwrap().push_back(i);
            t.thread().unpark();
            thread::sleep(Duration::from_secs(1));
        }
    });
}
```

```
    }  
    });  
}
```

The consuming thread runs an infinite loop in which it pops items out of the queue to display them using the `dbg` macro. When the queue is empty, it stops and goes to sleep using the `park()` function. If it gets unparked, the `park()` call returns, and the loop continues, popping items from the queue again until it is empty. And so on.

The producing thread produces a new number every second by pushing it into the queue. Every time it adds an item, it uses the `unpark()` method on the `Thread` object that refers to the consuming thread to unpark it. That way, the consuming thread gets woken up to process the new element.

An important observation to make here is that this program would still be theoretically correct, although inefficient, if we remove parking. This is important, because `park()` does not guarantee that it will only return because of a matching `unpark()`. While somewhat rare, it might have *spurious wake-ups*. Our example deals with that just fine, because the consuming thread will lock the queue, see that it is empty, and directly unlock it and park itself again.

An important property of thread parking is that a call to `unpark()` *before* the thread parks itself does not get lost. The request to unpark is still recorded, and the next time the thread tries to park itself, it clears that request and directly continues without actually going to sleep. To see why that is critical for correct operation, let's go through a possible ordering of the steps executed by both threads:

1. The consuming thread—let's call it C—locks the queue.
2. C tries to pop an item from the queue, but it is empty, resulting in `None`.
3. C unlocks the queue.
4. The producing thread, which we'll call P, locks the queue.
5. P pushes a new item onto the queue.
6. P unlocks the queue again.
7. P calls `unpark()` to notify C that there are new items.
8. C calls `park()` to go to sleep, to wait for more items.

While there is most likely only a very brief moment between releasing the queue in step 3 and parking in step 8, steps 4 through 7 could potentially happen in that moment before the thread parks itself. If `unpark()` would do nothing if the thread wasn't parked, the notification would be lost. The consuming thread would still be waiting, even if there were an item in the queue. Thanks to unpark requests getting saved for a future call to `park()`, we don't have to worry about this.

```
    }  
    });  
}
```

消费线程运行一个无限循环，从中弹出队列中的项目，并使用 `dbg` 宏将其显示。当队列为空时，它会停止运行并使用 `park()` 函数进入睡眠状态。如果它被 unpark，则 `park()` 调用将返回，`loop` 将继续执行，再次从队列中弹出项目，直到队列为空。如此往复。

生产线程每秒通过将一个新数字推入队列来产生该数字。每次添加项目时，它都会调用指向消费线程的 `Thread` 对象上的 `unpark()` 方法来将其 unpark。这样，消费线程就会被唤醒以处理新元素。

这里需要注意的一个重要观察是，如果我们移除线程挂起，这个程序在理论上仍然是正确的，尽管效率较低。这一点很重要，因为 `park()` 不能保证它仅因为匹配的 `unpark()` 而返回。虽然这种情况较为罕见，但它可能会出现 虚假唤醒。我们的示例很好地处理了这个问题，因为消费线程会锁定队列，发现它是空的，然后直接解锁并再次将自己挂起。

线程挂起的一个重要属性是：在线程将自己挂起之前对 `unpark()` 的调用不会丢失。unpark 请求仍然会被记录，当下一次线程尝试将自己挂起时，它会清除该请求并直接继续执行，而不会真正进入睡眠状态。为了了解为什么这对于正确运行至关重要，让我们来看看两个线程执行步骤的一种可能内存顺序：

1. 消费线程——我们称之为 C——锁定队列。
2. C 尝试从队列中弹出项，但队列为空，导致 `None`。
3. C 解锁队列。
4. 生产线程——我们称之为 P——锁定队列。
5. P 将新项推入队列。
6. P 再次解锁队列。
7. P 调用 `unpark()` 来通知 C 有新项。
8. C 调用 `park()` 进入睡眠状态，以等待更多项。

虽然在第 3 步释放队列和第 8 步线程挂起之间可能只有极短的瞬间，但第 4 到第 7 步可能会在这个瞬间、即线程挂起之前发生。如果 `unpark()` 在线程未挂起时不做任何操作，那么通知将会丢失。即使队列中已有项，消费线程仍然会在等待。得益于 unpark 请求会被保存以便在未来的 `park()` 调用中使用，我们无需担心这个问题。

However, `unpark()` requests don't stack up. Calling `unpark()` two times and then calling `park()` two times afterwards still results in the thread going to sleep. The first `park()` clears the request and returns directly, but the second one goes to sleep as usual.

This means that in our example above it's important that we only park the thread if we've seen the queue is empty, rather than park it after every processed item. While it's extremely unlikely to happen in this example because of the huge (one second) sleep, it's possible for multiple `unpark()` calls to wake up only a single `park()` call.

Unfortunately, this does mean that if `unpark()` is called right after `park()` returns, but before the queue gets locked and emptied out, the `unpark()` call was unnecessary but still causes the next `park()` call to instantly return. This results in the (empty) queue getting locked and unlocked an extra time. While this doesn't affect the correctness of the program, it does affect its efficiency and performance.

This mechanism works well for simple situations like in our example, but quickly breaks down when things get more complicated. For example, if we had multiple consumer threads taking items from the same queue, the producer thread would have no way of knowing which of the consumers is actually waiting and should be woken up. The producer will have to know exactly when a consumer is waiting, and what condition it is waiting for.

Condition Variables

Condition variables are a more commonly used option for waiting for something to happen to data protected by a mutex. They have two basic operations: *wait* and *notify*. Threads can wait on a condition variable, after which they can be woken up when another thread notifies that same condition variable. Multiple threads can wait on the same condition variable, and notifications can either be sent to one waiting thread, or to all of them.

This means that we can create a condition variable for specific events or conditions we're interested in, such as the queue being non-empty, and wait on that condition. Any thread that causes that event or condition to happen then notifies the condition variable, without having to know which or how many threads are interested in that notification.

To avoid the issue of missing notifications in the brief moment between unlocking a mutex and waiting for a condition variable, condition variables provide a way to *atomically* unlock the mutex and start waiting. This means there is simply no possible moment for notifications to get lost.

The Rust standard library provides a condition variable as `std::sync::Condvar`. Its `wait` method takes a `MutexGuard` that proves we've locked the mutex. It first unlocks

然而, `unpark` 请求不会堆叠。连续调用两次 `unpark()` 然后再调用两次 `park()`, 结果仍然是线程进入睡眠状态。第一次 `park()` 会清除请求并直接返回, 但第二次则会像往常一样进入睡眠。

这意味着在上面的示例中, 关键在于我们仅在发现队列为空时才让线程进入等待状态, 而不是在处理每个项目后都让其等待。虽然在这个示例中由于有很长的(一秒)睡眠时间, 这种情况极不可能发生, 但确实有可能多个 `unpark()` 调用仅唤醒单个 `park()` 调用。

不幸的是, 这意味着如果 `unpark()` 在 `park()` 返回后立即被调用, 但在队列被锁定并清空之前, 那么此次 `unpark()` 调用虽然是不必要的, 但仍会导致下一个 `park()` 调用立即返回。这导致(空的)队列被多余地锁定和解锁了一次。虽然这不会影响程序的正确性, 但会影响其效率和性能。

这种机制在像我们示例中的简单场景下运行良好, 但当情况变得复杂时会迅速失效。例如, 如果我们有多个消费者线程从同一个队列中提取项目, 生产者线程将无法得知哪个消费者实际上在等待并应该被唤醒。生产者必须准确地知道消费者何时在等待, 以及它在等待什么条件。

条件变量

条件变量是等待被互斥锁保护的数据发生某种事件时的更常用的选择。它们有两种基本操作: 等待 (*wait*) 和 通知 (*notify*)。线程可以在条件变量上等待, 随后当另一个线程通知该相同的条件变量时, 它们可以被唤醒。多个线程可以等待同一个条件变量, 且通知可以发送给一个等待中的线程, 或者发送给所有等待中的线程。

这意味着我们可以为我们感兴趣的特定事件或条件创建条件变量, 例如队列非空, 并在此条件下等待。任何导致该事件或条件发生的线程随后通知该条件变量, 而无需知道哪些线程或有多少个线程对该通知感兴趣。

为了避免在解锁互斥锁与等待条件变量之间极短的时间内丢失通知的问题, 条件变量提供了一种原子的方式来解锁互斥锁并开始等待。这意味着根本不存在丢失通知的可能性。

Rust 标准库提供了条件变量 `std::sync::Condvar`。其 `wait` 方法接收一个 `MutexGuard`, 用以证明我们已经锁定了互斥锁。该方法首先解锁互斥锁并进入

the mutex and goes to sleep. Later, when woken up, it relocks the mutex and returns a new `MutexGuard` (which proves that the mutex is locked again).

It has two notify functions: `notify_one` to wake up just one waiting thread (if any), and `notify_all` to wake them all up.

Let's modify the example we used for thread parking to use `Condvar` instead:

```
use std::sync::Condvar;

let queue = Mutex::new(VecDeque::new());
let not_empty = Condvar::new();

thread::scope(|s| {
    s.spawn(|| {
        loop {
            let mut q = queue.lock().unwrap();
            let item = loop {
                if let Some(item) = q.pop_front() {
                    break item;
                } else {
                    q = not_empty.wait(q).unwrap();
                }
            };
            drop(q);
            dbg!(item);
        }
    });

    for i in 0.. {
        queue.lock().unwrap().push_back(i);
        not_empty.notify_one();
        thread::sleep(Duration::from_secs(1));
    }
});
```

We had to change a few things:

- We now not only have a `Mutex` containing the queue, but also a `Condvar` to communicate the “not empty” condition.
- We no longer need to know which thread to wake up, so we don't store the return value from `spawn` anymore. Instead, we notify the consumer through the condition variable with the `notify_one` method.
- Unlocking, waiting, and relocking is all done by the `wait` method. We had to restructure the control flow a bit to be able to pass the guard to the `wait` method, while still dropping it before processing an item.

睡眠状态。随后，在被唤醒时它会重新锁定互斥锁并返回一个新的 `MutexGuard` (用以证明互斥锁已再次被锁定)。

它有两个通知函数：`notify_one` 用于唤醒一个等待中的线程（如果有的话），以及 `notify_all` 用于唤醒所有线程。

让我们修改之前在线程挂起示例中使用的代码，改为使用 `Condvar`：

```
use std::sync::Condvar;

let queue = Mutex::new(VecDeque::new());
let not_empty = Condvar::new();

thread::scope(|s| {
    s.spawn(|| {
        loop {
            let mut q = queue.lock().unwrap();
            let item = loop {
                if let Some(item) = q.pop_front() {
                    break item;
                } else {
                    q = not_empty.wait(q).unwrap();
                }
            };
            drop(q);
            dbg!(item);
        }
    });

    for i in 0.. {
        queue.lock().unwrap().push_back(i);
        not_empty.notify_one();
        thread::sleep(Duration::from_secs(1));
    }
});
```

我们需要更改几处内容：

- 现在我们不仅有一个包含队列的 `Mutex`，还有一个用于传递“非空”条件的 `Condvar`。
- 我们不再需要知道唤醒哪个线程，因此不再存储 `spawn` 的返回值。相反，我们通过条件变量使用 `notify_one` 方法来通知 `Consume` 端。
- 解锁、等待和重新加锁全部由 `wait` 方法完成。我们不得不稍微调整控制流，以便将守护对象传递给 `wait` 方法，同时在处理项目之前将其 `Drop`。

Now we can spawn as many consuming threads as we like, and even spawn more later, without having to change anything. The condition variable takes care of delivering the notifications to whichever thread is interested.

If we had a more complicated system with threads that are interested in different conditions, we could define a `Condvar` for each condition. For example, we could define one to indicate the queue is non-empty and another one to indicate it is empty. Then each thread would wait for whichever condition is relevant to what they are doing.

Normally, a `Condvar` is only ever used together with a single `Mutex`. If two threads try to concurrently `wait` on a condition variable using two different mutexes, it might cause a panic.

A downside of a `Condvar` is that it only works when used together with a `Mutex`, but for most use cases that is perfectly fine, as that's exactly what's already used to protect the data anyway.

Both `thread::park()` and `Condvar::wait()` also have a variant with a time limit: `thread::park_timeout()` and `Condvar::wait_timeout()`. These take a `Duration` as an extra argument, which is the time after which it should give up waiting for a notification and unconditionally wake up.

现在我们可以根据需要 spawn 任意数量的消费线程，甚至在之后 spawn 更多线程，而无需更改任何内容。条件变量会负责将通知传递给任何感兴趣的线程。

如果我们有一个更复杂的系统，其中线程关注不同的条件，我们可以为每个条件定义一个 `Condvar`。例如，我们可以定义一个用于表示队列非空，另一个用于表示队列为空。然后，每个线程将等待与其执行操作相关的条件。

通常情况下，`Condvar` 仅与单个 `Mutex` 配合使用。如果两个线程尝试使用两个不同的互斥锁并发地在条件变量上执行 `wait`，可能会导致 panic。

`Condvar` 的一个缺点是它必须与 `Mutex` 配合使用才有效，但对于大多数用例来说这完全没问题，因为无论如何，保护数据时已经使用了互斥锁。

`thread::park()` 和 `Condvar::wait()` 也都具有带有时间限制的变体：

`thread::park_timeout()` 和 `Condvar::wait_timeout()`。它们接收一个 `Duration` 作为额外参数，该参数是指在多久之后应该放弃等待通知并无条件唤醒。

Summary

- Multiple threads can run concurrently within the same program and can be spawned at any time.
- When the main thread ends, the entire program ends.
- Data races are undefined behavior, which is fully prevented (in safe code) by Rust's type system.
- Data that is Send can be sent to other threads, and data that is Sync can be shared between threads.
- Regular threads might run as long as the program does, and thus can only borrow 'static data such as statics and leaked allocations.
- Reference counting (Arc) can be used to share ownership to make sure data lives as long as at least one thread is using it.
- Scoped threads are useful to limit the lifetime of a thread to allow it to borrow non-'static data, such as local variables.
- &T is a *shared* reference. &mut T is an *exclusive* reference. Regular types do not allow mutation through a shared reference.
- Some types have interior mutability, thanks to UnsafeCell, which allows for mutation through shared references.
- Cell and RefCell are the standard types for single-threaded interior mutability. Atomics, Mutex, and RwLock are their multi-threaded equivalents.
- Cell and atomics only allow replacing the value as a whole, while RefCell, Mutex, and RwLock allow you to mutate the value directly by dynamically enforcing access rules.
- Thread parking can be a convenient way to wait for some condition.
- When a condition is about data protected by a Mutex, using a Condvar is more convenient, and can be more efficient, than thread parking.

总结

- 多个线程可以在同一个程序内并发运行，并且可以在任何时候被生成（spawn）。
- 当主线程结束时，整个程序随之结束。
- 数据竞争（data race）是未定义行为（undefined behavior），在安全代码（safe code）中，Rust 的类型系统可以完全阻止这种情况。
- 实现了 Send 的数据可以发送到其他线程，实现了 Sync 的数据可以在线程间共享。
- 普通线程（指 std::thread::spawn 生成的线程）的生命周期可能与程序一样长，因此只能借用 'static 数据，例如静态变量（statics）和泄漏的分配内存（leaked allocations）。
- 引用计数（Arc）可用于共享所有权，以确保数据只要至少有一个线程在使用它，就能持续存活。
- 作用域线程（scoped threads）有助于限制线程的生命周期，使其能够借用非 'static 的数据，例如局部变量。
- &T是共享引用(shared reference)，&mut T是独占引用(exclusive reference)。常规类型不允许通过共享引用进行修改。
- 某些类型具有内部可变性（interior mutability），这得益于UnsafeCell，它允许通过共享引用进行修改。
- Cell和RefCell是用于单线程内部可变性的标准类型；原子类型（Atomics）、Mutex 和 RwLock 则是它们在多线程环境中的对应物。
- Cell 和原子类型只允许整体替换值，而 RefCell、Mutex 和 RwLock 则通过动态强制访问规则，允许直接修改值。
- 线程暂停（thread parking）可能是等待某个条件的一种便捷方式。
- 当条件涉及受 Mutex 保护的数据时，使用 Condvar（条件变量）比使用线程暂停更便捷，并且可能更高效。

CHAPTER 2

Atomics

The word *atomic* comes from the Greek word *ἄτομος*, meaning *indivisible*, something that cannot be cut into smaller pieces. In computer science, it is used to describe an operation that is indivisible: it is either fully completed, or it didn't happen yet.

As mentioned in “[Borrowing and Data Races](#)” on page 11, multiple threads concurrently reading and modifying the same variable normally results in undefined behavior. However, atomic operations do allow for different threads to safely read and modify the same variable. Since such an operation is indivisible, it either happens completely before or completely after another operation, avoiding undefined behavior. Later, in [Chapter 7](#), we'll see how this works at the hardware level.

Atomic operations are the main building block for anything involving multiple threads. All the other concurrency primitives, such as mutexes and condition variables, are implemented using atomic operations.

In Rust, atomic operations are available as methods on the standard atomic types that live in `std::sync::atomic`. They all have names starting with `Atomic`, such as `AtomicI32` or `AtomicUsize`. Which ones are available depends on the hardware architecture and sometimes operating system, but almost all platforms provide at least all atomic types up to the size of a pointer.

Unlike most types, they allow modification through a shared reference (e.g., `&AtomicU8`). This is possible thanks to interior mutability, as discussed in “[Interior Mutability](#)” on page 13.

Each of the available atomic types has the same interface with methods for storing and loading, methods for atomic “fetch-and-modify” operations, and some more advanced “compare-and-exchange” methods. We'll discuss them in detail in the rest of this chapter.

第 2 章

原子操作

atomic（原子）这个词源自希腊语 *ἄτομος*，意思是不可分割的，即不能被切成更小碎片的東西。在计算机科学中，它被用来描述一个不可分割的操作：该操作要么完全完成，要么尚未发生。

正如第 11 页的“[借用与数据竞态](#)”中所提到的，多个线程并发地读取和修改同一个变量通常会导致未定义行为。然而，原子操作允许不同的线程安全地读取和修改同一个变量。由于这种操作是不可分割的，它要么在另一个操作之前完全发生，要么在另一个操作之后完全发生，从而避免了未定义行为。稍后在 [第 7 章](#)中，我们将看到这在硬件层级是如何运作的。

原子操作是所有涉及多线程功能的主要构建块。所有其他的并发原语，例如互斥锁和条件变量，都是使用原子操作实现的。

在 Rust 中，原子操作作为 `std::sync::atomic` 中标准原子类型的方法提供。它们的所有名称都以 `Atomic` 开头，例如 `AtomicI32` 或 `AtomicUsize`。具体可以使用哪些类型取决于硬件架构，有时还取决于操作系统，但几乎所有平台至少都提供了大小直到指针大小的所有原子类型。

与大多数类型不同，它们允许通过共享引用（例如 `&AtomicU8`）进行修改。这得益于内部可变性（interior mutability），正如 [第 13 页](#)“[内部可变性](#)”一节中所讨论的那样。

每种可用的原子类型都具有相同的接口，包含用于存储和加载的方法、用于原子“获取并修改”操作的方法，以及一些更高级的“比较并交换”方法。我们将在本章接下来的内容中详细讨论它们。

But, before we can dive into the different atomic operations, we briefly need to touch upon a concept called *memory ordering*:

Every atomic operation takes an argument of type `std::sync::atomic::Ordering`, which determines what guarantees we get about the relative ordering of operations. The simplest variant with the fewest guarantees is `Relaxed`. `Relaxed` still guarantees consistency on a single atomic variable, but does not promise anything about the relative order of operations between different variables.

What this means is that two threads might see operations on different variables happen in a different order. For example, if one thread writes to one variable first and then to a second variable very quickly afterwards, another thread might see that happen in the opposite order.

In this chapter we'll only look at use cases where this is not a problem and simply use `Relaxed` everywhere without going more into detail. We'll discuss all the details of memory ordering and the other available memory orderings in [Chapter 3](#).

Atomic Load and Store Operations

The first two atomic operations we'll look at are the most basic ones: load and store. Their function signatures are as follows, using `AtomicI32` as an example:

```
impl AtomicI32 {
    pub fn load(&self, ordering: Ordering) -> i32;
    pub fn store(&self, value: i32, ordering: Ordering);
}
```

The `load` method atomically loads the value stored in the atomic variable, and the `store` method atomically stores a new value in it. Note how the `store` method takes a shared reference (`&T`) rather than an exclusive reference (`&mut T`), even though it modifies the value.

Let's take a look at some realistic use cases for these two methods.

Example: Stop Flag

The first example uses an `AtomicBool` for a *stop flag*. Such a flag is used to inform other threads to stop running.

```
use std::sync::atomic::AtomicBool;
use std::sync::atomic::Ordering::Relaxed;

fn main() {
    static STOP: AtomicBool = AtomicBool::new(false);

    // Spawn a thread to do the work.
    let background_thread = thread::spawn(|| {
```

但在深入探讨不同的原子操作之前，我们需要简要触及一个名为 `内存顺序` 的概念：

每个原子操作都接收一个类型为 `std::sync::atomic::Ordering` 的参数，它决定了我们能获得关于操作相对内存顺序的哪些保证。保证最少的最简单变体是 `Relaxed`。`Relaxed` 仍然保证单个原子变量的一致性，但不对不同变量之间操作的相对顺序做出任何承诺。

这意味着两个线程可能会看到不同变量上的操作以不同的顺序发生。例如，如果一个线程先写入一个变量，然后很快地写入第二个变量，另一个线程可能会看到这两个操作以相反的顺序发生。

在本章中，我们仅讨论这种情况不构成问题的使用场景，并简单地各处使用 `Relaxed`，而不再深入探讨细节。我们将在 [第 3 章](#) 中讨论内存排序的所有细节以及其他可用的内存顺序。

原子加载与存储操作

我们将探讨的前两个原子操作是最基础的：`load`和`store`。以 `AtomicI32` 为例，它们的函数签名如下：

```
impl AtomicI32 {
    pub fn load(&self, ordering: Ordering) -> i32;
    pub fn store(&self, value: i32, ordering: Ordering);
}
```

`load` 方法原子的加载存储在原子变量中的值，而 `store` 方法则原子的地将一个新值存储其中。请注意，尽管 `store` 方法修改了该值，但它接收的是一个共享引用 (`&T`) 而非独占引用 (`&mut T`)。

让我们来看看这两个方法的几个实际使用场景。

示例：停止标志

第一个示例使用 `AtomicBool` 来实现一个停止标志。这种标志用于通知其他线程停止运行。

```
use std::sync::atomic::AtomicBool;
use std::sync::atomic::Ordering::Relaxed;

fn main() {
    static STOP: AtomicBool = AtomicBool::new(false);

    // Spawn a thread to do the work.
    let background_thread = thread::spawn(|| {
```

```

        while !STOP.load(Relaxed) {
            some_work();
        }
    });

    // Use the main thread to listen for user input.
    for line in std::io::stdin().lines() {
        match line.unwrap().as_str() {
            "help" => println!("commands: help, stop"),
            "stop" => break,
            cmd => println!("unknown command: {cmd:?}"),
        }
    }

    // Inform the background thread it needs to stop.
    STOP.store(true, Relaxed);

    // Wait until the background thread finishes.
    background_thread.join().unwrap();
}

```

In this example, the background thread is repeatedly running `some_work()`, while the main thread allows the user to enter some commands to interact with the program. In this simple example, the only useful command is `stop` to make the program stop.

To make the background thread stop, the atomic `STOP` boolean is used to communicate this condition to the background thread. When the foreground thread reads the `stop` command, it sets the flag to `true`, which is checked by the background thread before each new iteration. The main thread waits until the background thread is finished with its current iteration using the `join` method.

This simple solution works great as long as the flag is regularly checked by the background thread. If it gets stuck in `some_work()` for a long time, that can result in an unacceptable delay between the `stop` command and the program quitting.

Example: Progress Reporting

In our next example, we process 100 items one by one on a background thread, while the main thread gives the user regular updates on the progress:

```

use std::sync::atomic::AtomicUsize;

fn main() {
    let num_done = AtomicUsize::new(0);

    thread::scope(|s| {
        // A background thread to process all 100 items.
        s.spawn(|| {
            for i in 0..100 {
                process_item(i); // Assuming this takes some time.
            }
        });
    });
}

```

```

        while !STOP.load(Relaxed) {
            some_work();
        }
    });

    // Use the main thread to listen for user input.
    for line in std::io::stdin().lines() {
        match line.unwrap().as_str() {
            "help" => println!("commands: help, stop"),
            "stop" => break,
            cmd => println!("unknown command: {cmd:?}"),
        }
    }

    // Inform the background thread it needs to stop.
    STOP.store(true, Relaxed);

    // Wait until the background thread finishes.
    background_thread.join().unwrap();
}

```

在这个示例中，后台线程重复运行 `some_work()`，而主线程则允许用户输入某些命令与程序进行交互。在这个简单的示例中，唯一有用的命令是 `stop`，用于让程序停止。

为了让后台线程停止，使用了原子的 `STOP` 布尔值将这一条件传达给后台线程。当前台线程读取到 `stop` 命令时，它会将标志设为 `true`，后台线程在每次进行新迭代之前都会检查该标志。主线程使用 `join` 方法等待后台线程完成当前的迭代。

只要后台线程定期检查该标志，这个简单的解决方案就能很好地工作。但如果它在 `some_work()` 中卡住很长时间，可能会导致 `stop` 命令与程序退出之间出现不可接受的延迟。

示例：进度报告

在接下来的示例中，我们在后台线程中逐个处理 100 个项目，而主线程则向用户定期更新进度：

```

use std::sync::atomic::AtomicUsize;

fn main() {
    let num_done = AtomicUsize::new(0);

    thread::scope(|s| {
        // A background thread to process all 100 items.
        s.spawn(|| {
            for i in 0..100 {
                process_item(i); // Assuming this takes some time.
            }
        });
    });
}

```



```

        num_done.store(i + 1, Relaxed);
    }
});

// The main thread shows status updates, every second.
loop {
    let n = num_done.load(Relaxed);
    if n == 100 { break; }
    println!("Working.. {n}/100 done");
    thread::sleep(Duration::from_secs(1));
}

println!("Done!");
}

```

This time, we use a scoped thread ([“Scoped Threads” on page 5](#)), which will automatically handle the joining of the thread for us, and also allow us to borrow local variables.

Every time the background thread finishes processing an item, it stores the number of processed items in an `AtomicUsize`. Meanwhile, the main thread shows that number to the user to inform them of the progress, about once per second. Once the main thread sees that all 100 items have been processed, it exits the scope, which implicitly joins the background thread, and informs the user that everything is done.

Synchronization

Once the last item is processed, it might take up to one whole second for the main thread to know, introducing an unnecessary delay at the end. To solve this, we can use thread parking ([“Thread Parking” on page 24](#)) to wake the main thread from its sleep whenever there is new information it might be interested in.

Here’s the same example, but now using `thread::park_timeout` rather than `thread::sleep`:

```

fn main() {
    let num_done = AtomicUsize::new(0);

    let main_thread = thread::current();

    thread::scope(|s| {
        // A background thread to process all 100 items.
        s.spawn(|| {
            for i in 0..100 {
                process_item(i); // Assuming this takes some time.
                num_done.store(i + 1, Relaxed);
                main_thread.unpark(); // Wake up the main thread.
            }
        });
    });
}

```

```

        num_done.store(i + 1, Relaxed);
    }
});

// The main thread shows status updates, every second.
loop {
    let n = num_done.load(Relaxed);
    if n == 100 { break; }
    println!("Working.. {n}/100 done");
    thread::sleep(Duration::from_secs(1));
}

println!("Done!");
}

```

这次我们使用一个作用域线程（见第 5 页的 [“Scoped Threads”](#)），它会自动为我们处理线程的合并线程，并且允许我们借用局部变量。

每当后台线程完成一个项目的处理时，它会将已处理项目的数量存储在一个 `AtomicUsize` 中。与此同时，主线程大约每秒向用户显示一次该数字，以告知处理进度。一旦主线程看到所有 100 个项目都已处理完毕，它将退出作用域，从而隐式地汇合后台线程，并告知用户所有工作已完成。

同步

一旦最后一个项目被处理，主线程可能需要长达一整秒才能得知，这在最后引入了不必要的延迟。为了解决这个问题，我们可以使用线程挂起（见第 24 页的 [“Thread Parking”](#)），以便在有其感兴趣的新信息时唤醒处于睡眠状态的主线程。

这里是同一个示例，但现在使用 `thread::park_timeout` 而不是 `thread::sleep`：

```

fn main() {
    let num_done = AtomicUsize::new(0);

    let main_thread = thread::current();

    thread::scope(|s| {
        // A background thread to process all 100 items.
        s.spawn(|| {
            for i in 0..100 {
                process_item(i); // Assuming this takes some time.
                num_done.store(i + 1, Relaxed);
                main_thread.unpark(); // Wake up the main thread.
            }
        });
    });
}

```

```

    // The main thread shows status updates.
    loop {
        let n = num_done.load(Relaxed);
        if n == 100 { break; }
        println!("Working.. {n}/100 done");
        thread::park_timeout(Duration::from_secs(1));
    }
});

println!("Done!");
}

```

Not much has changed. We've obtained a handle to the main thread through `thread::current()`, which is now used by the background thread to unpark the main thread after every status update. The main thread now uses `park_timeout` rather than `sleep`, such that it can be interrupted.

Now, any status updates are immediately reported to the user, while still repeating the last update every second to show that the program is still running.

Example: Lazy Initialization

The last example before we move on to more advanced atomic operations is about *lazy initialization*.

Imagine there is a value `x`, which we are reading from a file, obtaining from the operating system, or calculating in some other way, that we expect to be constant during a run of the program. Maybe `x` is the version of the operating system, or the total amount of memory, or the 400th digit of `tau`. It doesn't really matter for this example.

Since we don't expect it to change, we can request or calculate it only the first time we need it, and remember the result. The first thread that needs it will have to calculate the value, but it can store it in an atomic `static` to make it available for all threads, including itself if it needs it again later.

Let's take a look at an example of this. To keep things simple, we'll assume `x` is never zero, so that we can use zero as a placeholder before it has been calculated.

```

use std::sync::atomic::AtomicU64;

fn get_x() -> u64 {
    static X: AtomicU64 = AtomicU64::new(0);
    let mut x = X.load(Relaxed);
    if x == 0 {
        x = calculate_x();
        X.store(x, Relaxed);
    }
    x
}

```

```

    // The main thread shows status updates.
    loop {
        let n = num_done.load(Relaxed);
        if n == 100 { break; }
        println!("Working.. {n}/100 done");
        thread::park_timeout(Duration::from_secs(1));
    }
});

println!("Done!");
}

```

并没有太多的变化。我们通过 `thread::current()` 获取了主线程的句柄，现在后台线程在每次状态更新后都会使用它来 unpark 主线程。主线程现在使用 `park_timeout` 而不是 `sleep`，以便它可以被中断。

现在，任何状态更新都会立即报告给用户，同时每秒仍然重复最后一次更新，以表明程序仍在运行。

示例：延迟初始化

在进入更高级的原子操作之前，最后一个示例是关于 延迟初始化 的。

假设有一个值 `x`，我们是从文件中读取、从操作系统获取或通过其他方式计算出来的，并且我们预期它在程序运行期间保持不变。也许 `x` 是操作系统的版本，或者内存的总量，或者是 `tau` 的第 400 位数字。对于这个示例来说，这并不重要。

既然我们不认为它会改变，我们可以在第一次需要它时才请求或计算它，并记住结果。第一个需要该值的线程必须计算该值，但它可以将其存储在一个原子的 `static` 中，使其对所有线程可用，包括如果它以后再次需要该值的话。

让我们来看一个相关的示例。为了简单起见，我们假设 `x` 永远不为零，这样我们就可以在它被计算出来之前将零作为占位符。

```

use std::sync::atomic::AtomicU64;

fn get_x() -> u64 {
    static X: AtomicU64 = AtomicU64::new(0);
    let mut x = X.load(Relaxed);
    if x == 0 {
        x = calculate_x();
        X.store(x, Relaxed);
    }
    x
}

```

The first thread to call `get_x()` will check the static `X` and see it is still zero, calculate its value, and store the result back in the static to make it available for future use. Later, any call to `get_x()` will see that the value in the static is nonzero, and return it immediately without calculating it again.

However, if a second thread calls `get_x()` while the first one is still calculating `x`, the second thread will also see a zero and also calculate `x` in parallel. One of the threads will end up overwriting the result of the other, depending on which one finishes first. This is called a *race*. Not a *data race*, which is undefined behavior and impossible in Rust without using `unsafe`, but still a race with an unpredictable winner.

Since we expect `x` to be constant, it doesn't matter who wins the race, as the result will be the same regardless. Depending on how much time we expect `calculate_x()` to take, this might be a very good or very bad strategy.

If `calculate_x()` is expected to take a long time, it's better if threads wait while the first thread is still initializing `X`, to avoid unnecessarily wasting processor time. You could implement this using a condition variable or thread parking ([“Waiting: Parking and Condition Variables” on page 24](#)), but that quickly gets too complicated for a small example. The Rust standard library provides exactly this functionality through `std::sync::Once` and `std::sync::OnceLock`, so there's usually no need to implement these yourself.

Fetch-and-Modify Operations

Now that we've seen a few use cases for the basic `load` and `store` operations, let's move on to more interesting operations: the *fetch-and-modify* operations. These operations modify the atomic variable, but also load (fetch) the original value, as a single atomic operation.

The most commonly used ones are `fetch_add` and `fetch_sub`, which perform addition and subtraction, respectively. Some of the other available operations are `fetch_or` and `fetch_and` for bitwise operations, and `fetch_max` and `fetch_min` which can be used to keep a running maximum or minimum.

第一个调用 `get_x()` 的线程将检查静态变量 `x` 并发现其仍然为零，随后计算其值，并将结果存储回静态变量中，以便将来使用。之后，任何对 `get_x()` 的调用都会发现静态变量中的值非零，并立即将其返回，而无需再次计算。

然而，如果当第一个线程仍在计算 `x` 时，第二个线程调用了 `get_x()`，那么第二个线程也将看到零，并并行地计算 `x`。根据哪个线程先完成，其中一个线程最终会覆盖另一个线程的结果。这被称为竞态。这并非数据竞态（数据竞态是未定义行为，且在不使用 `unsafe` 的情况下在 Rust 中是不可能的），但仍然是一个胜负不可预测的竞态。

由于我们预期 `x` 是常数，因此无论谁赢得竞态都无关紧要，因为结果无论如何都是相同的。根据我们预期的 `calculate_x()` 所需的时间，这可能是一个非常好或非常糟糕的策略。

如果 `calculate_x()` 预计需要较长时间，那么最好让其他线程在第一个线程仍在初始化 `x` 时等待，以避免不必要地浪费处理器时间。你可以使用条件变量或线程挂起来实现这一点（见第 24 页的 [“等待：线程挂起与条件变量”](#)），但对于一个简单的示例来说，这样做很快就会变得过于复杂。Rust 标准库通过 `std::sync::Once` 和 `std::sync::OnceLock` 正好提供了这项功能，因此通常无需自行实现。

fetch-and-modify 操作

既然我们已经了解了一些基础的 `load` 和 `store` 操作的用例，现在让我们转向更有趣的操作：*fetch-and-modify* 操作。这些操作在修改原子变量的同时，还会将原始值加载（获取）出来，且整个过程是一个单一的原子操作。

最常用的是 `fetch_add` 和 `fetch_sub`，它们分别执行加法和减法。其他可用操作还包括用于位运算的 `fetch_or` 和 `fetch_and`，以及可用于维护运行最大值或最小值的 `fetch_max` 和 `fetch_min`。

Their function signatures are as follows, using AtomicI32 as an example:

```
impl AtomicI32 {
    pub fn fetch_add(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_sub(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_or(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_and(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_nand(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_xor(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_max(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_min(&self, v: i32, ordering: Ordering) -> i32;
    pub fn swap(&self, v: i32, ordering: Ordering) -> i32; // "fetch_store"
}
```

The one outlier is the operation that simply stores a new value, regardless of the old value. Instead of `fetch_store`, it has been called `swap`.

Here’s a quick demonstration showing how `fetch_add` returns the value before the operation:

```
use std::sync::atomic::AtomicI32;

let a = AtomicI32::new(100);
let b = a.fetch_add(23, Relaxed);
let c = a.load(Relaxed);

assert_eq!(b, 100);
assert_eq!(c, 123);
```

The `fetch_add` operation incremented `a` from 100 to 123, but returned to us the old value of 100. Any next operation will see the value of 123.

The return value from these operations is not always relevant. If you only need the operation to be applied to the atomic value, but are not interested in the value itself, it’s perfectly fine to simply ignore the return value.

An important thing to keep in mind is that `fetch_add` and `fetch_sub` implement *wrapping* behavior for overflows. Incrementing a value past the maximum representable value will wrap around and result in the minimum representable value. This is different than the behavior of the plus and minus operators on regular integers, which will panic in debug mode on overflow.

In “Compare-and-Exchange Operations” on page 42, we’ll see how to do atomic addition with overflow checking.

But first, let’s see some real-world use cases of these methods.

以 AtomicI32 为例，它们的函数签名如下：

```
impl AtomicI32 {
    pub fn fetch_add(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_sub(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_or(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_and(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_nand(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_xor(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_max(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_min(&self, v: i32, ordering: Ordering) -> i32;
    pub fn swap(&self, v: i32, ordering: Ordering) -> i32; // "fetch_store"
}
```

唯一的例外是仅存储新值的操作，而不考虑旧值。它没有被地称为 `fetch_store`，而被地称为 `swap`。

这里有一个简单的演示，展示了 `fetch_add` 如何返回操作之前的值：

```
use std::sync::atomic::AtomicI32;

let a = AtomicI32::new(100);
let b = a.fetch_add(23, Relaxed);
let c = a.load(Relaxed);

assert_eq!(b, 100);
assert_eq!(c, 123);
```

`fetch_add` 操作将 `a` 从 100 增加到了 123，但向我们返回了旧值 100。随后的任何操作都将看到 123 这个值。

这些操作的返回值并不总是相关的。如果您只需要将操作应用于原子的值，而对值本身不感兴趣，那么直接忽略返回值是完全没有问题的。

需要记住的一点是，`fetch_add` 和 `fetch_sub` 对溢出实现了回绕(*wrapping*)行为。将值增加到超过最大可表示值时，它将回绕并导致最小可表示值。这与常规整数上的加法和减法运算符的行为不同，后者在调试模式下发生溢出时会 panic。

在第 42 页的“比较并交换操作”中，我们将看到如何进行带有溢出检查的原子加法。

但首先，让我们来看看这些方法的实际使用场景。

Example: Progress Reporting from Multiple Threads

In “[Example: Progress Reporting](#)” on page 33, we used an `AtomicUsize` to report the progress of a background thread. If we had split the work over, for example, four threads with each processing 25 items, we’d need to know the progress from all four threads.

We could use a separate `AtomicUsize` for each thread and load them all in the main thread and sum them up, but an easier solution is to use a single `AtomicUsize` to track the total number of processed items over all threads.

To make that work, we can no longer use the `store` method, as that would overwrite the progress from other threads. Instead, we can use an atomic add operation to increment the counter after every processed item.

Let’s update the example from “[Example: Progress Reporting](#)” on page 33 to split the work over four threads:

```
fn main() {
    let num_done = &AtomicUsize::new(0);

    thread::scope(|s| {
        // Four background threads to process all 100 items, 25 each.
        for t in 0..4 {
            s.spawn(move || {
                for i in 0..25 {
                    process_item(t * 25 + i); // Assuming this takes some time.
                    num_done.fetch_add(1, Relaxed);
                }
            });
        }

        // The main thread shows status updates, every second.
        loop {
            let n = num_done.load(Relaxed);
            if n == 100 { break; }
            println!("Working.. {n}/100 done");
            thread::sleep(Duration::from_secs(1));
        }
    });

    println!("Done!");
}
```

A few things have changed. Most importantly, we now spawn four background threads rather than one, and use `fetch_add` instead of `store` to modify the `num_done` atomic variable.

More subtly, we now use a `move` closure for the background threads, and `num_done` is now a reference. This is not related to our use of `fetch_add`, but rather to how

示例：来自多个线程的进度报告

在第 33 页的“[示例：进度报告](#)”中，我们使用了一个 `AtomicUsize` 来报告后台线程的进度。如果我们将工作分摊到例如四个线程上，每个线程处理 25 个项目，那么我们就需要知道所有四个线程的进度。

我们可以为每个线程使用一个单独的 `AtomicUsize`，然后在主线程中将它们全部加载并求和，但更简单的解决方案是使用一个单一的 `AtomicUsize` 来跟踪所有线程中已处理项目的总数。

为了实现这一点，我们不能再使用 `store` 方法，因为这会覆盖来自其他线程的进度。相反，我们可以使用原子加法操作在每个项目处理完成后递增计数器。

让我们更新 第 33 页的“[示例：进度报告](#)”中的示例，将工作分配给四个线程：

```
fn main() {
    let num_done = &AtomicUsize::new(0);

    thread::scope(|s| {
        // Four background threads to process all 100 items, 25 each.
        for t in 0..4 {
            s.spawn(move || {
                for i in 0..25 {
                    process_item(t * 25 + i); // Assuming this takes some time.
                    num_done.fetch_add(1, Relaxed);
                }
            });
        }

        // The main thread shows status updates, every second.
        loop {
            let n = num_done.load(Relaxed);
            if n == 100 { break; }
            println!("Working.. {n}/100 done");
            thread::sleep(Duration::from_secs(1));
        }
    });

    println!("Done!");
}
```

有几处发生了变化。最重要的一点是，我们现在 spawn 四个后台线程而不是一个，并且使用 `fetch_add` 而不是 `store` 来修改 `num_done` 原子变量。

更微妙的是，我们现在为后台线程使用了一个 `move` 闭包，并且 `num_done` 现在是一个引用。这与我们对 `fetch_add` 的使用无关，而是因为

we spawn four threads in a loop. This closure captures `t` to know which of the four threads it is, and thus whether to start at item 0, 25, 50, or 75. Without the `move` keyword, the closure would try to capture `t` by reference. That isn't allowed, as it only exists briefly during the loop.

As a `move` closure, it moves (or copies) its captures rather than borrowing them, giving it a copy of `t`. Because it also captures `num_done`, we've changed that variable to be a reference, since we still want to borrow that same `AtomicUsize`. Note that the atomic types do not implement the `Copy` trait, so we'd have gotten an error if we had tried to move one into more than one thread.

Closure capture subtleties aside, the change to use `fetch_add` here is very simple. We don't know in which order the threads will increment `num_done`, but as the addition is atomic, we don't have to worry about anything and can be sure it will be exactly 100 when all threads are done.

Example: Statistics

Continuing with this concept of reporting what other threads are doing through atomics, let's extend our example to also collect and report some statistics on the time it takes to process an item.

Next to `num_done`, we're adding two atomic variables, `total_time` and `max_time`, to keep track of the amount of time spent processing items. We'll use these to report the average and peak processing times.

```
fn main() {
    let num_done = &AtomicUsize::new(0);
    let total_time = &AtomicU64::new(0);
    let max_time = &AtomicU64::new(0);

    thread::scope(|s| {
        // Four background threads to process all 100 items, 25 each.
        for t in 0..4 {
            s.spawn(move || {
                for i in 0..25 {
                    let start = Instant::now();
                    process_item(t * 25 + i); // Assuming this takes some time.
                    let time_taken = start.elapsed().as_micros() as u64;
                    num_done.fetch_add(1, Relaxed);
                    total_time.fetch_add(time_taken, Relaxed);
                    max_time.fetch_max(time_taken, Relaxed);
                }
            });
        }

        // The main thread shows status updates, every second.
        loop {
            let total_time = Duration::from_micros(total_time.load(Relaxed));
```

我们在循环中 `spawn` 了四个线程。这个闭包捕获了 `t` 以确定它是四个线程中的哪一个，从而决定是从第 0、25、50 还是 75 个项目开始。如果没有 `move` 关键字，闭包将尝试通过引用捕获 `t`。这是不允许的，因为它仅在循环期间短暂存在。

作为一个 `move` 闭包，它会移动（或复制）其捕获的内容而非借用它们，从而使其获得 `t` 的一个副本。由于它还捕获了 `num_done`，我们将该变量改为引用，因为我们仍然希望借用同一个 `AtomicUsize`。请注意，原子类型没有实现 `Copy` trait，因此如果我们尝试将一个原子变量移动到多个线程中，将会报错。

撇开闭包捕获的微妙之处不谈，这里改为使用 `fetch_add` 非常简单。我们不知道线程将以什么样的顺序增加 `num_done`，但由于加法是原子的，我们无需担心任何问题，并且可以确定当所有线程执行完毕时，它的值将恰好为 100。

示例：统计数据

继续使用通过原子操作报告其他线程在做什么这一概念，让我们扩展示例，使其还能收集并报告处理单个项目所需时间的统计数据。

在 `num_done` 旁边，我们添加了两个原子变量 `total_time` 和 `max_time`，用于跟踪处理项目所花费的时间。我们将使用这些变量来报告平均处理时间和峰值处理时间。

```
fn main() {
    let num_done = &AtomicUsize::new(0);
    let total_time = &AtomicU64::new(0);
    let max_time = &AtomicU64::new(0);

    thread::scope(|s| {
        // Four background threads to process all 100 items, 25 each.
        for t in 0..4 {
            s.spawn(move || {
                for i in 0..25 {
                    let start = Instant::now();
                    process_item(t * 25 + i); // Assuming this takes some time.
                    let time_taken = start.elapsed().as_micros() as u64;
                    num_done.fetch_add(1, Relaxed);
                    total_time.fetch_add(time_taken, Relaxed);
                    max_time.fetch_max(time_taken, Relaxed);
                }
            });
        }

        // The main thread shows status updates, every second.
        loop {
            let total_time = Duration::from_micros(total_time.load(Relaxed));
```



```

    let max_time = Duration::from_micros(max_time.load(Relaxed));
    let n = num_done.load(Relaxed);
    if n == 100 { break; }
    if n == 0 {
        println!("Working.. nothing done yet.");
    } else {
        println!(
            "Working.. {n}/100 done, {:?} average, {:?} peak",
            total_time / n as u32,
            max_time,
        );
    }
    thread::sleep(Duration::from_secs(1));
}
});

println!("Done!");
}

```

The background threads now use `Instant::now()` and `Instant::elapsed()` to measure the time they spend in `process_item()`. An atomic add operation is used to add the number of microseconds to `total_time`, and an atomic max operation is used to keep track of the highest measurement in `max_time`.

The main thread divides the total time by the number of processed items to obtain the average processing time, which it then reports together with the peak time from `max_time`.

Since the three atomic variables are updated separately, it is possible for the main thread to load the values after a thread has incremented `num_done`, but before it has updated `total_time`, resulting in an underestimate of the average. More subtly, because the `Relaxed` memory ordering gives no guarantees about the relative order of operations as seen from another thread, it might even briefly see a new updated value of `total_time`, while still seeing an old value of `num_done`, resulting in an overestimate of the average.

Neither of this is a big issue in our example. The worst that can happen is that an inaccurate average is briefly reported to the user.

If we want to avoid this, we can put the three statistics inside a `Mutex`. Then we'd briefly lock the mutex while updating the three numbers, which no longer have to be atomic by themselves. This effectively turns the three updates into a single atomic operation, at the cost of locking and unlocking a mutex, and potentially temporarily blocking threads.

```

    let max_time = Duration::from_micros(max_time.load(Relaxed));
    let n = num_done.load(Relaxed);
    if n == 100 { break; }
    if n == 0 {
        println!("Working.. nothing done yet.");
    } else {
        println!(
            "Working.. {n}/100 done, {:?} average, {:?} peak",
            total_time / n as u32,
            max_time,
        );
    }
    thread::sleep(Duration::from_secs(1));
}
});

println!("Done!");
}

```

后台线程现在使用 `Instant::now()` 和 `Instant::elapsed()` 来衡量它们在 `process_item()` 中花费的时间。使用原子加法操作将微秒数累加到 `total_time`，并使用原子最大值操作来记录 `max_time` 中的最高测量值。

主线程将总时间除以处理的项目数以获得平均处理时间，然后将其与来自 `max_time` 的峰值时间一起报告。

由于这三个原子变量是分别更新的，主线程可能会在某个线程递增 `num_done` 之后但在其更新 `total_time` 之前加载这些值，从而导致对平均值的低估。更微妙的是，由于 `Relaxed` 内存顺序无法保证另一个线程看到的各种操作的相对顺序，它甚至可能会短暂地看到 `total_time` 的一个新更新值，而此时仍看到 `num_done` 的旧值，从而导致对平均值的高估。

在我们的示例中，这两者都不是大问题。最糟糕的情况就是向用户报告了一个短暂不准确的平均值。

如果我们想避免这种情况，可以将这三个统计数据放在一个 `Mutex` 中。这样我们在更新这三个数字时，只需短暂地锁定互斥锁，而这三个数字本身就不再需要是原子的。这实际上将三次更新变成了单个原子操作，代价是需要对互斥锁进行加锁和解锁，并可能暂时阻塞线程。

Example: ID Allocation

Let’s move on to a use case where we actually need the return value from `fetch_add`.

Suppose we need some function, `allocate_new_id()`, that gives a new unique number every time it is called. We might use these numbers to identify tasks or other things in our program; things that need to be uniquely identified by something small that can be easily stored and passed around between threads, such as an integer.

Implementing this function turns out to be trivial using `fetch_add`:

```
use std::sync::atomic::AtomicU32;

fn allocate_new_id() -> u32 {
    static NEXT_ID: AtomicU32 = AtomicU32::new(0);
    NEXT_ID.fetch_add(1, Relaxed)
}
```

We simply keep track of the *next* number to give out, and increment it every time we load it. The first caller will get a 0, the second a 1, and so on.

The only problem here is the wrapping behavior on overflow. The 4,294,967,296th call will overflow the 32-bit integer, such that the next call will return 0 again.

Whether this is a problem depends on the use case: how likely is it to be called this often, and what’s the worst that can happen if the numbers are not unique? While this might seem like a huge number, modern computers can easily execute our function that many times within seconds. If memory safety is dependent on these numbers being unique, our implementation above is not acceptable.

To solve this, we can attempt to make the function panic if it is called too many times, like this:

```
// This version is problematic.
fn allocate_new_id() -> u32 {
    static NEXT_ID: AtomicU32 = AtomicU32::new(0);
    let id = NEXT_ID.fetch_add(1, Relaxed);
    assert!(id < 1000, "too many IDs!");
    id
}
```

Now, the `assert` statement will panic after a thousand calls. However, this happens *after* the atomic add operation already happened, meaning that `NEXT_ID` has already been incremented to 1001 when we panic. If another thread then calls the function, it’ll increment it to 1002 before panicking, and so on. Although it might take significantly longer, we’ll run into the same problem after 4,294,966,296 panics when `NEXT_ID` will overflow to zero again.

示例：ID 分配

让我们 我们将探讨一个实际上需要 `fetch_add` 返回值的用例。

假设我们需要一个函数 `allocate_new_id()`，每次调用它时都会给出一个新的唯一数字。我们可能会使用这些数字来标识程序中的任务或其他事物；这些事物需要通过某种小型且易于存储并能在线程之间传递的东西（例如整数）来唯一标识。

使用 `fetch_add` 来实现这个函数其实非常简单：

```
use std::sync::atomic::AtomicU32;

fn allocate_new_id() -> u32 {
    static NEXT_ID: AtomicU32 = AtomicU32::new(0);
    NEXT_ID.fetch_add(1, Relaxed)
}
```

我们只需记录要分发的下一个数字，并在每次加载它时将其递增。第一个调用者将得到 0，第二个得到 1，依此类推。

这里唯一的问题是溢出时的回绕行为。第 4,294,967,296 次调用将导致 32 位整数溢出，使得下一次调用再次返回 0。

这是否是一个问题取决于具体的使用场景：它被如此频繁调用的可能性有多大，以及如果数字不唯一，最坏的情况会是什么？虽然这个数字看起来很大，但现代计算机在几秒钟内就能轻松执行我们函数这么多次。如果内存安全性依赖于这些数字的唯一性，那么我们上面的实现是不可接受的。

为了解决这个问题，我们可以尝试在函数被调用次数过多时使其 panic，如下所示：

```
// This version is problematic.
fn allocate_new_id() -> u32 {
    static NEXT_ID: AtomicU32 = AtomicU32::new(0);
    let id = NEXT_ID.fetch_add(1, Relaxed);
    assert!(id < 1000, "too many IDs!");
    id
}
```

现在，`assert` 语句在调用一千次后会 panic。然而，这发生在原子加法操作已经执行之后，这意味着当我们 panic 时，`NEXT_ID` 已经增加到了 1001。如果另一个线程随后调用该函数，它会在panic之前将其增加到 1002，以此类推。虽然这可能会花费更长的时间，但在经历 4,294,966,296 次panic后，我们会遇到同样的问题，届时 `NEXT_ID` 将再次溢出归零。

There are three common solutions to this problem. The first one is to not panic but instead completely abort the process on overflow. The `std::process::abort` function will abort the entire process, ruling out the possibility of anything continuing to call our function. While aborting the process might take a brief moment in which the function can still be called by other threads, the chance of that happening billions of times before the program is truly aborted is negligible.

This is, in fact, how the overflow check in `Arc::clone()` in the standard library is implemented, in case you somehow manage to clone it `usize::MAX` times. That'd take hundreds of years on a 64-bit computer, but is achievable in seconds if `usize` is only 32 bits.

A second way to deal with the overflow is to use `fetch_sub` to decrement the counter again before panicking, like this:

```
fn allocate_new_id() -> u32 {
    static NEXT_ID: AtomicU32 = AtomicU32::new(0);
    let id = NEXT_ID.fetch_add(1, Relaxed);
    if id >= 1000 {
        NEXT_ID.fetch_sub(1, Relaxed);
        panic!("too many IDs!");
    }
    id
}
```

It's still possible for the counter to very briefly be incremented beyond 1000 when multiple threads execute this function at the same time, but it is limited by the number of active threads. It's reasonable to assume there will never be billions of active threads at once, especially not all simultaneously executing the same function in the brief moment between `fetch_add` and `fetch_sub`.

This is how overflows are handled for the number of running threads in the standard library's `thread::scope` implementation.

The third way of handling overflows is arguably the only truly correct one, as it prevents the addition from happening at all if it would overflow. However, we cannot implement that with the atomic operations we've seen so far. For this, we'll need compare-and-exchange operations, which we'll explore next.

Compare-and-Exchange Operations

The most advanced and flexible atomic operation is the *compare-and-exchange* operation. This operation checks if the atomic value is equal to a given value, and only if that is the case does it replace it with a new value, all atomically as a single operation. It will return the previous value and tell us whether it replaced it or not.

解决这个问题有三种常见的方案。第一种是不进行 panic，而是在溢出时完全终止进程。`std::process::abort` 函数将终止整个进程，从而排除了任何内容继续调用我们函数的可能性。虽然终止进程可能需要极短的时间，在此期间函数仍可能被其他线程调用，但在程序真正终止之前发生数十亿次这种情况的概率微乎其微。

事实上，这就是标准库中 `Arc::clone()` 的溢出检查实现方式，以防您设法将其 Clone 了 `usize::MAX` 次。在 64 位计算机上这需要数百年的时间，但如果 `usize` 只有 32 位，则在几秒钟内即可实现。

处理溢出的第二种方法是使用 `fetch_sub` 在 panic 之前再次递减计数器，如下所示：

```
fn allocate_new_id() -> u32 {
    static NEXT_ID: AtomicU32 = AtomicU32::new(0);
    let id = NEXT_ID.fetch_add(1, Relaxed);
    if id >= 1000 {
        NEXT_ID.fetch_sub(1, Relaxed);
        panic!("too many IDs!");
    }
    id
}
```

当多个线程同时执行此函数时，计数器仍有可能在极短时间内增加到 1000 以上，但这受限于活动线程的数量。我们可以合理地认为，永远不会同时有数十亿个活动线程，尤其是不会在 `fetch_add` 和 `fetch_sub` 之间的短暂瞬间同时执行同一个函数。

这就是标准库 `thread::scope` 实现中处理运行线程数溢出的方式。

处理溢出的第三种方法可以说唯一真正正确的方法，因为它在加法会导致溢出时会直接阻止加法的发生。然而，我们无法使用目前看到的原子操作来实现这一点。为此，我们需要比较并交换操作，我们将在接下来的内容中探讨。

比较并交换操作

最先进且最灵活的原子操作是比较并交换（compare-and-exchange）操作。该操作会检查原子的值是否等于一个给定的值，并且只有在这种情况下，它才会将其替换为一个新值，所有这些都作为单个操作原子的地完成。它将返回之前的值，并告知我们是否执行了替换。

Its signature is a bit more complicated than the ones we've seen so far. Using `AtomicI32` as an example, it looks like this:

```
impl AtomicI32 {
    pub fn compare_exchange(
        &self,
        expected: i32,
        new: i32,
        success_order: Ordering,
        failure_order: Ordering
    ) -> Result<i32, i32>;
}
```

Ignoring memory ordering for a moment, it is basically identical to the following implementation, except it all happens as a single, indivisible atomic operation:

```
impl AtomicI32 {
    pub fn compare_exchange(&self, expected: i32, new: i32) -> Result<i32, i32> {
        // In reality, the load, comparison and store,
        // all happen as a single atomic operation.
        let v = self.load();
        if v == expected {
            // Value is as expected.
            // Replace it and report success.
            self.store(new);
            Ok(v)
        } else {
            // The value was not as expected.
            // Leave it untouched and report failure.
            Err(v)
        }
    }
}
```

Using this, we can load a value from an atomic variable, perform any calculation we like, and then only store the newly calculated value if the atomic variable didn't change in the meantime. If we put this in a loop to retry if it did change, we could use this to implement all the other atomic operations, making this the most general one.

To demonstrate, let's increment an `AtomicU32` by one without using `fetch_add`, just to see how `compare_exchange` is used in practice:

```
fn increment(a: &AtomicU32) {
    let mut current = a.load(Relaxed); ❶
    loop {
        let new = current + 1; ❷
        match a.compare_exchange(current, new, Relaxed, Relaxed) { ❸
            Ok(_) => return, ❹
            Err(v) => current = v, ❺
        }
    }
}
```

它的函数签名比我们目前看到的要复杂一些。以 `AtomicI32` 为例，其签名如下所示：

```
impl AtomicI32 {
    pub fn compare_exchange(
        &self,
        expected: i32,
        new: i32,
        success_order: Ordering,
        failure_order: Ordering
    ) -> Result<i32, i32>;
}
```

暂时忽略内存顺序，它基本上与以下实现相同，唯一的区别是所有操作都作为一个单一的、不可分割的原子操作完成：

```
impl AtomicI32 {
    pub fn compare_exchange(&self, expected: i32, new: i32) -> Result<i32, i32> {
        // In reality, the load, comparison and store,
        // all happen as a single atomic operation.
        let v = self.load();
        if v == expected {
            // Value is as expected.
            // Replace it and report success.
            self.store(new);
            Ok(v)
        } else {
            // The value was not as expected.
            // Leave it untouched and report failure.
            Err(v)
        }
    }
}
```

利用这一点，我们可以从一个原子变量中加载一个值，执行任何我们想要的计算，然后仅在原子变量在此期间没有发生变化时，才存储新计算出的值。如果我们将其放入一个循环中，在发生变化时进行重试，我们就可以利用它来实现所有其他原子操作，使其成为最通用的一种。

为了演示，让我们在不使用 `fetch_add` 的情况下将 `AtomicU32` 加一，以此来看看 `compare_exchange` 在实践中是如何使用的：

```
fn increment(a: &AtomicU32) {
    let mut current = a.load(Relaxed); ❶
    loop {
        let new = current + 1; ❷
        match a.compare_exchange(current, new, Relaxed, Relaxed) { ❸
            Ok(_) => return, ❹
            Err(v) => current = v, ❺
        }
    }
}
```

- ❶ First, we load the current value of `a`.
- ❷ We calculate the new value we want to store in `a`, not taking into account potential concurrent modifications of `a` by other threads.
- ❸ We use `compare_exchange` to update the value of `a`, but *only* if its value is still the same value we loaded before.
- ❹ If `a` was indeed still the same as before, it is now replaced by our new value and we are done.
- ❺ If `a` was not the same as before, another thread must've changed it in the brief moment since we loaded it. The `compare_exchange` operation gives us the changed value that `a` had, and we'll try again using that value instead. The brief moment between loading and updating is so short that it's unlikely for this to loop more than a few iterations.



If the atomic variable changes from some value `A` to `B` and then back to `A` after the load operation, but before the `compare_exchange` operation, it would still succeed, even though the atomic variable was changed (and changed back) in the meantime. In many cases, as with our `increment` example, this is not a problem. However, there are certain algorithms, often involving atomic pointers, for which this can be a problem. This is known as the *ABA problem*.

Next to `compare_exchange`, there is a similar method named `compare_exchange_weak`. The difference is that the weak version may still sometimes leave the value untouched and return an `Err`, even though the atomic value matched the expected value. On some platforms, this method can be implemented more efficiently and should be preferred in cases where the consequence of a spurious compare-and-exchange failure are insignificant, such as in our `increment` function above. In [Chapter 7](#), we'll dive into the low-level details to find out why the weak version can be more efficient.

Example: ID Allocation Without Overflow

Now, back to our overflow problem in `allocate_new_id()` from [“Example: ID Allocation” on page 41](#).

To stop incrementing `NEXT_ID` beyond a certain limit to prevent overflows, we can use `compare_exchange` to implement atomic addition with an upper bound. Using

- ❶ 首先，我们加载 `a` 的当前值。
- ❷ 我们计算想要存储在 `a` 中的新值，而不考虑其他线程对 `a` 可能进行的并发修改。
- ❸ 我们使用 `compare_exchange` 来更新 `a` 的值，但仅在其值仍为我们之前加载的值时才进行更新。
- ❹ 如果 `a` 确实与之前相同，它现在将被我们的新值替换，我们就完成了。
- ❺ 如果 `a` 与之前不同，那么在自加载以来的短暂时间内，必然有另一个线程修改了它。`compare_exchange` 操作会向我们提供 `a` 所持的修改后的值，然后我们将尝试使用该值再次运行。加载与更新之间的这段时间非常短，因此这种情况不太可能循环超过几次。



如果原子变量在 `load` 操作之后、`compare_exchange` 操作之前，从某个值 `A` 变为 `B`，然后又变回 `A`，那么 `compare_exchange` 操作仍然会成功，即使该原子变量在此期间确实发生了变化（并又变了回来）。在许多情况下，就像我们的 `increment` 示例一样，这不是一个问题。然而，对于某些算法（通常涉及原子指针）来说，这可能会成为一个问题。这被称为 *ABA 问题*。

在 `compare_exchange` 旁边，有一个类似的方法名为 `compare_exchange_weak`。区别在于，即使原子值与预期值匹配，Weak 版本有时仍可能保持值不变并返回 `Err`。在某些平台上，此方法的实现效率更高，在比较并交换失败后果并不严重的情况下（例如我们上面的 `increment` 函数），应当优先使用此方法。在 [第 7 章](#) 中，我们将深入研究底层细节，以找出为什么 Weak 版本会更高效。

示例：无溢出的 ID 分配

现在，回到我们在 `allocate_new_id()` 中遇到的溢出问题，该内容来自 [第 41 页](#) 的 [“示例：ID 分配”](#)。

为了阻止 `NEXT_ID` 无限递增以至于超出某个上限，从而防止溢出，我们可以使用 `compare_exchange` 来实现带有上限的原子加法。利用这个想法，让我们创

that idea, let’s make a version of `allocate_new_id` that always handles overflow correctly, even in practically impossible situations:

```
fn allocate_new_id() -> u32 {
    static NEXT_ID: AtomicU32 = AtomicU32::new(0);
    let mut id = NEXT_ID.load(Relaxed);
    loop {
        assert!(id < 1000, "too many IDs!");
        match NEXT_ID.compare_exchange_weak(id, id + 1, Relaxed, Relaxed) {
            Ok(_) => return id,
            Err(v) => id = v,
        }
    }
}
```

Now we check and panic *before* modifying `NEXT_ID`, guaranteeing it will never be incremented beyond 1000, making overflow impossible. We can now raise the upper limit from 1000 to `u32::MAX` if we want, without having to worry about edge cases in which it might get incremented beyond the limit.

Fetch-Update

The atomic types have a convenience method called `fetch_update` for the compare-and-exchange loop pattern. It’s equivalent to a load operation followed by a loop that repeats a calculation and `compare_exchange_weak`, just like what we did above.

Using it, we could implement our `allocate_new_id` function with a one-liner:

```
NEXT_ID.fetch_update(Relaxed, Relaxed,
    |n| n.checked_add(1)).expect("too many IDs!")
```

Check out the method’s documentation for details.

We’ll not use the `fetch_update` method in this book, so we can focus on the individual atomic operations.

Example: Lazy One-Time Initialization

In “[Example: Lazy Initialization](#)” on page 35, we looked at an example of lazy initialization of a constant value. We made a function that lazily initializes a value on the first call, but reuses it on later calls. When multiple threads run the function concurrently during the first call, more than one thread might execute the initialization, and they will overwrite each others’ result in an unpredictable order.

This is fine for values that we expect to be constant, or when we don’t care about changing values. However, there are also use cases where such a value gets initialized to a different value each time, even though we need every invocation of the function within a single run of the program to return the same value.

建一个 `allocate_new_id` 版本，使其即使在实际中几乎不可能出现的情况下也能正确处理溢出：

```
fn allocate_new_id() -> u32 {
    static NEXT_ID: AtomicU32 = AtomicU32::new(0);
    let mut id = NEXT_ID.load(Relaxed);
    loop {
        assert!(id < 1000, "too many IDs!");
        match NEXT_ID.compare_exchange_weak(id, id + 1, Relaxed, Relaxed) {
            Ok(_) => return id,
            Err(v) => id = v,
        }
    }
}
```

现在我们在修改 `NEXT_ID` 之前 进行检查并触发 panic，保证它永远不会被增加到 1000 以上，从而使溢出成为不可能。现在如果我们愿意，可以将上限从 1000 提高到 `u32::MAX`，而无需担心它可能会被增加到上限之外的边界情况。

取值-更新

原子类型提供了一个名为 `fetch_update` 的便捷方法，用于处理**比较并交换**（compare-and-exchange）的循环模式。它相当于一个 load 操作，随后是一个重复计算并执行 `compare_exchange_weak` 的循环，就像我们上面所做的一样。使用它，我们可以用一行代码来实现我们的 `allocate_new_id` 函数：

```
NEXT_ID.fetch_update(Relaxed, Relaxed,
    |n| n.checked_add(1)).expect("too many IDs!")
```

请查看该方法的文档以了解详细信息。

在本书中我们将不使用 `fetch_update` 方法，以便我们可以专注于单个的原子操作。

示例：延迟一次性初始化

在第 35 页的“[示例：延迟初始化](#)”中，我们查看了一个常量值延迟初始化的示例。我们编写了一个函数，在第一次调用时对值进行延迟初始化，但在随后的调用中重复使用该值。当多个线程在第一次调用期间并发运行该函数时，可能会有多个线程执行初始化，并且它们将以不可预测的顺序覆盖彼此的结果。

对于我们预期为常量的值，或者当我们不关心值是否变化时，这样做是没有问题的。然而，在某些使用场景中，即使我们需要在程序单次运行期间的所有函数调用中都返回相同的值，该值在每次运行时仍会被初始化为不同的值。

For example, imagine a function `get_key()` that returns a randomly generated key that's only generated once per run of the program. It might be an encryption key used for communication with the program, which needs to be unique every time the program is run, but stays constant within a process.

This means we cannot simply use a `store` operation after generating a key, since that might overwrite a key generated by another thread just moments ago, resulting in two threads using different keys. Instead, we can use `compare_exchange` to make sure we only store the key if no other thread has already done so, and otherwise throw our key away and use the stored key instead.

Here's an implementation of this idea:

```
fn get_key() -> u64 {
    static KEY: AtomicU64 = AtomicU64::new(0);
    let key = KEY.load(Relaxed);
    if key == 0 {
        let new_key = generate_random_key(); ❶
        match KEY.compare_exchange(0, new_key, Relaxed, Relaxed) { ❷
            Ok(_) => new_key, ❸
            Err(k) => k, ❹
        }
    } else {
        key
    }
}
```

- ❶ We only generate a new key if KEY was not yet initialized.
- ❷ We replace KEY with our newly generated key, but only if it is *still* zero.
- ❸ If we swapped the zero for our new key, we return our newly generated key. New invocations of `get_key()` will return the same new key that's now stored in KEY.
- ❹ If we lost the race to another thread that initialized KEY before we could, we forget our newly generated key and use the key from KEY instead.

This is a good example of a situation where `compare_exchange` is more appropriate than its weak variant. We don't run our compare-and-exchange operation in a loop, and we don't want to return zero if the operation spuriously fails.

As mentioned in “[Example: Lazy Initialization](#)” on page 35, if `generate_random_key()` takes a lot of time, it might make more sense to block threads during initialization, to avoid potentially spending time generating keys that will not be used. The Rust standard library provides such functionality through `std::sync::Once` and `std::sync::OnceLock`.

例如，想象一个函数 `get_key()`，它返回一个随机生成的密钥，且在程序每次运行期间仅生成一次。这可能是一个用于与程序通信的加密密钥，它在每次程序运行时都需要唯一，但在同一个进程中保持不变。

这意味着我们在生成密钥后不能简单地使用 `store` 操作，因为这可能会覆盖另一个线程刚刚生成的密钥，导致两个线程使用不同的密钥。相反，我们可以使用 `compare_exchange` 来确保只有在没有其他线程已经存储密钥的情况下才存储我们的密钥，否则则丢弃我们的密钥并使用已存储的密钥。

以下是该想法的一个实现：

```
fn get_key() -> u64 {
    static KEY: AtomicU64 = AtomicU64::new(0);
    let key = KEY.load(Relaxed);
    if key == 0 {
        let new_key = generate_random_key(); ❶
        match KEY.compare_exchange(0, new_key, Relaxed, Relaxed) { ❷
            Ok(_) => new_key, ❸
            Err(k) => k, ❹
        }
    } else {
        key
    }
}
```

- ❶ 只有在 KEY 尚未初始化时，我们才会生成一个新密钥。
- ❷ 我们将 KEY 替换为新生成的密钥，但前提是它仍然为零。
- ❸ 如果我们用新密钥替换了零，我们就返回这个新生成的密钥。后续调用 `get_key()` 将返回现在存储在 KEY 中的同一个新密钥。
- ❹ 如果我们在这个竞态中输给了另一个在之前就初始化了 KEY 的线程，我们就放弃新生成的密钥，转而使用来自 KEY 的密钥。

这是一个很好的例子，说明了在这种情况下 `compare_exchange` 比其 `Weak` 变体更合适。我们没有在循环中运行比较并交换操作，并且我们不希望在操作伪失败时返回零。

正如第 35 页的“[示例：延迟初始化](#)”中所述，如果 `generate_random_key()` 需要耗费大量时间，那么在初始化期间阻塞线程可能更有意义，以避免在生成可能不会被使用的密钥上浪费时间。Rust 标准库通过 `std::sync::Once` 和 `std::sync::OnceLock` 提供了此类功能。

Summary

- Atomic operations are indivisible; they have either fully completed, or they haven't happened yet.
- Atomic operations in Rust are done through the atomic types in `std::sync::atomic`, such as `AtomicI32`.
- Not all atomic types are available on all platforms.
- The relative ordering of atomic operations is tricky when multiple variables are involved. More in [Chapter 3](#).
- Simple loads and stores are nice for very basic inter-thread communication, like stop flags and status reporting.
- Lazy initialization can be done as a *race*, without causing a *data race*.
- Fetch-and-modify operations allow for a small set of basic atomic modifications that are especially useful when multiple threads are modifying the same atomic variable.
- Atomic addition and subtraction silently wrap around on overflow.
- Compare-and-exchange operations are the most flexible and general, and a building block for making any other atomic operation.
- A *weak* compare-and-exchange operation can be slightly more efficient.

总结

- 原子操作是不可分割的；它们要么已经完全完成，要么尚未发生。
- Rust 中的原子操作是通过 `std::sync::atomic` 中的原子类型（例如 `AtomicI32`）来完成的。
- 并非所有的原子类型在所有平台上都可用。
- 当涉及多个变量时，原子操作的相对内存顺序非常复杂。详情请参阅[第 3 章](#)。
- 简单的加载和存储适用于非常基础的线程间通信，例如停止标志和状态报告。
- 延迟初始化可以通过一次竞态来完成，而不会导致数据竞态。
- `fetch-and-modify` 操作允许进行一组简单的基础原子的修改，当多个线程在修改同一个原子变量时，这些操作尤其有用。
- 原子加法和减法在溢出时会静默地回绕。
- 比较并交换操作是最灵活且最通用的，它是构建任何其他原子操作的基础。
- 弱比较并交换（`weak compare-and-exchange`）操作的性能可能略高一些。

Memory Ordering

In [Chapter 2](#), we briefly touched upon the concept of *memory ordering*. In this chapter, we'll dive into this topic and explore all the available memory ordering options, and, most importantly, when to use which one.

Reordering and Optimizations

Processors and compilers perform all sorts of tricks to make your programs run as fast as possible. A processor might determine that two particular consecutive instructions in your program will not affect each other, and execute them *out of order*, if that is faster, for example. While one instruction is briefly blocked on fetching some data from main memory, several of the following instructions might be executed and finished before the first instruction finishes, as long as that wouldn't change the behavior of your program. Similarly, a compiler might decide to reorder or rewrite parts of your program if it has reason to believe it might result in faster execution. But, again, only if that wouldn't change the behavior of your program.

Let's take a look at the following function as an example:

```
fn f(a: &mut i32, b: &mut i32) {  
    *a += 1;  
    *b += 1;  
    *a += 1;  
}
```

Here, the compiler will most certainly understand that the order of these operations does not matter, since nothing happens between these three addition operations that depends on the value of `*a` or `*b`. (Assuming overflow checking is disabled.) Because of that, it might reorder the second and third operations, and then merge the first two into a single addition:

内存顺序

在[第 2 章](#)中，我们简要地触及了内存顺序的概念。在本章中，我们将深入探讨这个主题，研究所有可用的内存顺序选项，以及最重要的一点：在何时使用哪一个。

重排序与优化

处理器和编译器会采取各种手段，以尽可能地提高程序的运行速度。例如，处理器可能会判定程序中两个特定的连续指令互不影响，那么如果这样更快的话，它可能会乱序执行它们。当一条指令在从主内存获取某些数据而被短暂阻塞时，只要不改变程序的行为，随后的几条指令可能会在第一条指令完成之前就执行完毕。同样地，如果编译器有理由相信某种做法可以提高执行速度，它可能会决定对程序的某些部分进行重排序或重写。但同样地，前提是这不会改变程序的行为。

让我们以接下来的函数为例：

```
fn f(a: &mut i32, b: &mut i32) {  
    *a += 1;  
    *b += 1;  
    *a += 1;  
}
```

在这里，编译器几乎肯定会意识到这些操作的顺序并不重要，因为在这三个加法操作之间没有依赖于 `*a` 或 `*b` 值的操作。（假设禁用了溢出检查。）因此，它可能会重新排列第二和第三个操作，然后将前两个操作合并为一个加法：

```
fn f(a: &mut i32, b: &mut i32) {  
    *a += 2;  
    *b += 1;  
}
```

Later, while executing this function of the optimized compiled program, a processor might for a variety of reasons end up executing the second addition before the first addition, possibly because `*b` was available in a cache, while `*a` had to be fetched from the main memory.

Regardless of these optimizations, the result stays the same: `*a` is incremented by two and `*b` is incremented by one. The order in which they were incremented is entirely invisible to the rest of your program.

The logic for verifying that a specific reordering or other optimization won't affect the behavior of your program does not take other threads into account. In our example above, that's perfectly fine, as the unique references (`&mut i32`) guarantee that nothing else can possibly access the values, making other threads irrelevant. The only situation where this is a problem is when mutating data that's shared between threads. Or, in other words, when working with atomics. This is why we have to explicitly tell the compiler and processor what they can and can't do with our atomic operations, since their usual logic ignores interactions between threads and might allow for optimizations that do change the result of your program.

The interesting question is *how* we tell them. If we wanted to precisely spell out exactly what is and isn't acceptable, concurrent programming might become exceedingly verbose and error prone, and maybe even architecture-specific:

```
let x = a.fetch_add(1,  
    Dear compiler and processor,  
    Feel free to reorder this with operations on b,  
    but if there's another thread concurrently executing f,  
    please don't reorder this with operations on c!  
    Also, processor, don't forget to flush your store buffer!  
    If b is zero, though, it doesn't matter.  
    In that case, feel free to do whatever is fastest.  
    Thanks~ <3  
);
```

Instead, we can only pick from a small set of options, represented by the `std::sync::atomic::Ordering` enum, which every atomic operation takes as an argument. The set of available options is very limited, but has been carefully picked to fit most use cases well. The orderings are very abstract and do not directly reflect the actual compiler and processor mechanisms involved, such as instruction reordering. This makes it possible for your concurrent code to be architecture-independent and future-proof. It allows for verification without knowing the details of every single current and future processor and compiler version.

```
fn f(a: &mut i32, b: &mut i32) {  
    *a += 2;  
    *b += 1;  
}
```

随后，在执行这个经过优化编译的程序函数时，处理器可能出于各种原因最终在执行第一个加法之前先执行第二个加法，这可能是因为 `*b` 已经在缓存中，而 `*a` 则必须从主内存中获取。

无论这些优化如何，结果保持不变：`*a` 增加了 2，`*b` 增加了 1。它们增加的顺序对于程序的其余部分来说是完全不可见的。

验证特定的重排序或其他优化不会影响程序行为的逻辑并没有将其他线程考虑在内。在上面的示例中，这完全没问题，因为唯一引用 (`&mut i32`) 保证了没有其他任何东西能访问这些值，使得其他线程变得无关紧要。唯一会出现问题的情况是，在修改线程间共享的数据时。换句话说，就是在使用原子操作时。这就是为什么我们必须显式地告诉编译器和处理器，它们在处理我们的原子操作时可以做什么以及不能做什么，因为它们通常的逻辑会忽略线程间的交互，并可能会允许那些会改变程序结果的优化。

有趣的问题在于我们如何区分它们。如果我们想要精确地阐明哪些行为是可接受的、哪些是不可接受的，那么并发编程可能会变得极其冗长、容易出错，甚至可能与具体架构密切相关：

```
let x = a.fetch_add(1,  
    Dear compiler and processor,  
    Feel free to reorder this with operations on b,  
    but if there's another thread concurrently executing f,  
    please don't reorder this with operations on c!  
    Also, processor, don't forget to flush your store buffer!  
    If b is zero, though, it doesn't matter.  
    In that case, feel free to do whatever is fastest.  
    Thanks~ <3  
);
```

相反，我们只能从 `std::sync::atomic::Ordering` 枚举所代表的一组少量选项中进行选择，每个原子操作都将其作为一个参数。可用选项的集合非常有限，但经过精心挑选，以很好地适应大多数使用场景。这些内存顺序非常抽象，并不直接反映涉及的实际编译器和处理器机制（例如指令重排）。这使得您的并发代码能够独立于架构并具有前瞻性。它允许在无需了解每一个当前及未来处理器和编译器版本细节的情况下进行验证。

The available orderings in Rust are:

- Relaxed ordering: `Ordering::Relaxed`
- Release and acquire ordering: `Ordering::{Release, Acquire, AcqRel}`
- Sequentially consistent ordering: `Ordering::SeqCst`

In C++, there is also something called *consume ordering*, which has been purposely omitted from Rust, but is nonetheless interesting to discuss as well.

The Memory Model

The different memory ordering options have a strict formal definition to make sure we know exactly what we’re allowed to assume, and for compiler writers to know exactly what guarantees they need to provide to us. To decouple this from the details of specific processor architectures, memory ordering is defined in terms of an abstract *memory model*.

Rust’s memory model, which is mostly copied from C++, doesn’t match any existing processor architecture, but instead is an abstract model with a strict set of rules that attempt to represent the greatest common denominator of all current and future architectures, while also giving the compiler enough freedom to make useful assumptions while analyzing and optimizing programs.

We’ve already seen a part of the memory model in action in “[Borrowing and Data Races](#)” on [page 11](#), where we talked about how data races result in undefined behavior. Rust’s memory model allows for concurrent atomic stores, but considers concurrent non-atomic stores to the same variable to be a data race, resulting in undefined behavior.

On most processor architectures, however, there is actually no difference between an atomic store and a regular non-atomic store, as we’ll see in [Chapter 7](#). One could argue that the memory model is more restrictive than necessary, but these strict rules make it easier to reason about a program, both for the compiler and the programmer, and they leave space for future developments.

Happens-Before Relationship

The memory model defines the order in which operations happen in terms of *happens-before relationships*. This means that as an abstract model, it doesn’t talk about machine instructions, caches, buffers, timing, instruction reordering, compiler optimizations, and so on, but instead only defines situations where one thing is guaranteed to happen before another thing, and leaves the order of everything else undefined.

Rust 中可用的内存顺序包括:

- 宽松顺序: `Ordering::Relaxed`
- 释放-获取顺序: `Ordering::{Release, Acquire, AcqRel}`
- 顺序一致性顺序: `Ordering::SeqCst`

在 C 语言++中, 还有一种被称为 消费顺序 的机制, 它被特意地从 Rust 中剔除了, 但仍然值得讨论。

内存模型

不同的内存排序选项都有严格的正式定义, 以确保我们确切地知道可以假设什么, 并让编译器写者确切地知道他们需要为我们提供什么样的保证。为了与此与具体处理器架构的细节解耦, 内存排序是通过一个抽象的 内存模型 来定义的。

Rust 内存模型在很大程度上参考了 C 语言++, 它并不与任何现有的处理器架构完全一致, 而是一个抽象模型。该模型拥有一套严格的规则, 旨在代表所有当前和未来架构的最大公约数, 同时给予编译器足够的自由, 使其在分析和优化程序时能做出有用的假设。

我们已经在[第 11 页](#)的“[借用与数据竞态](#)”中看到过内存模型的一部分实际应用, 我们在那里讨论了数据竞争如何导致未定义行为。Rust 内存模型允许并发的原子存储, 但将对同一个变量的并发非原子存储视为数据竞态, 从而导致未定义行为。

然而, 在大多数处理器架构中, 原子存储和普通的非原子存储实际上没有区别, 我们将在[第 7 章](#)中看到这一点。有人可能会认为该内存模型比必要的限制更多, 但这些严格的规则使得编译器和程序员都能更容易地对程序进行推理, 并为未来的发展留下了空间。

先行发生关系

内存模型定义了操作发生的顺序, 具体表现为先行发生关系。这意味着作为一个抽象模型, 它并不讨论机器指令、缓存、缓冲区、时序、指令重排、编译器优化等, 而仅定义了某些情况下的操作保证在另一些操作之前发生, 而其他所有情况的顺序则是不确定的。

The basic happens-before rule is that everything that happens within the same thread happens in order. If a thread is executing `f()`; `g()`;, then `f()` *happens-before* `g()`.

Between threads, however, happens-before relationships only occur in a few specific cases, such as when spawning and joining a thread, unlocking and locking a mutex, and through atomic operations that use non-relaxed memory ordering. Relaxed memory ordering is the most basic (and most performant) memory ordering that, by itself, never results in any cross-thread happens-before relationships.

To explore what that means, let's take a look the following example where we assume `a` and `b` are concurrently executed by different threads:

```
static X: AtomicI32 = AtomicI32::new(0);
static Y: AtomicI32 = AtomicI32::new(0);

fn a() {
    X.store(10, Relaxed); ❶
    Y.store(20, Relaxed); ❷
}

fn b() {
    let y = Y.load(Relaxed); ❸
    let x = X.load(Relaxed); ❹
    println!("{x} {y}");
}
```

As mentioned above, the basic happens-before rule is that everything that happens within the same thread happens in order. In this case: ❶ happens-before ❷, and ❸ happens-before ❹, as shown in Figure 3-1. Since we use relaxed memory ordering, there are no other happens-before relationships in our example.

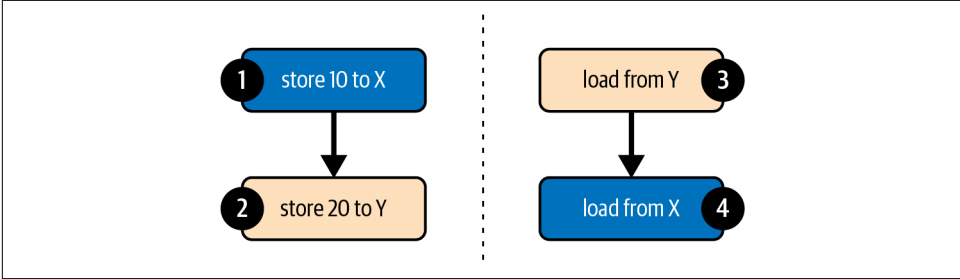


Figure 3-1. The happens-before relationships between atomic operations in the example code

If either of `a` or `b` completes before the other starts, the output will be `0 0` or `10 20`. If `a` and `b` run concurrently, it's easy to see how the output can be `10 0`. One way this can happen is if the operations run in this order: ❸ ❶ ❷ ❹.

基础的先行发生规则是：在同一个线程内发生的所有事情都是按顺序发生的。如果一个线程正在执行 `f()`; `g()`;, 那么 `f()` 先行发生于 `g()`。

然而，在线程之间，先行发生关系仅在少数特定情况下发生，例如 `spawn` 和合并线程、解锁和锁定互斥锁，以及通过使用非宽松内存顺序的原子操作。Relaxed 宽松内存顺序是最基础（且性能最高）的内存顺序，其本身绝不会导致任何跨线程的先行发生关系。

为了探讨这意味着什么，让我们来看下面的示例，我们假设 `a` and `b` 由不同的线程并发执行：

```
static X: AtomicI32 = AtomicI32::new(0);
static Y: AtomicI32 = AtomicI32::new(0);

fn a() {
    X.store(10, Relaxed); ❶
    Y.store(20, Relaxed); ❷
}

fn b() {
    let y = Y.load(Relaxed); ❸
    let x = X.load(Relaxed); ❹
    println!("{x} {y}");
}
```

如上所述，基本的先行发生规则是：在同一个线程内发生的所有事情都是按顺序发生的。在这种情况下：先行发生，且先行发生，如图 3-1 所示。由于我们使用了宽松内存顺序，在本示例中没有其他先行发生关系。

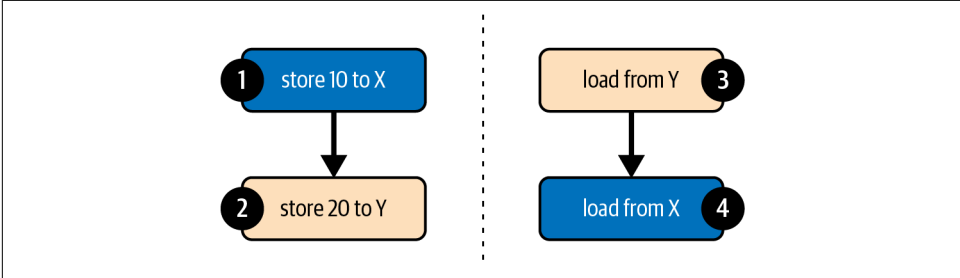


图 3-1. 示例代码中原子操作之间的先行发生关系

如果 `a` 或 `b` 在另一个开始之前完成，输出将是 `0 0` 或 `10 20`。如果 `a` 和 `b` 并发运行，很容易看出输出为什么可能是 `10 0`。这种情况发生的一种方式操作按以下顺序运行：❸ ❶ ❷ ❹。

More interestingly, the output can also be 0 20, even though there is no possible globally consistent order of the four operations that would result in this outcome. When ③ is executed, there is no happens-before relationship with ②, which means it could load either 0 or 20. When ④ is executed, there is no happens-before relationship with ①, which means it could load either 0 or 10. Given this, the output 0 20 is a valid outcome.

The important and counter-intuitive thing to understand is that operation ③ loading the value 20 does *not* result in a happens-before relationship with ②, even though that value is the one stored by ②. Our intuitive understanding of the concept of “before” breaks down when things don’t necessarily happen in a globally consistent order, such as when instruction reordering is involved.

A more practical and intuitive, but less formal, understanding is that from the perspective of the thread executing b, operations ① and ② might appear to happen in the opposite order.

Spawning and Joining

Spawning a thread creates a happens-before relationship between what happened before the spawn() call, and the new thread. Similarly, joining a thread creates a happens-before relationship between the joined thread and what happens after the join() call.

To demonstrate, the assertion in the following example cannot fail:

```
static X: AtomicI32 = AtomicI32::new(0);

fn main() {
    X.store(1, Relaxed);
    let t = thread::spawn(f);
    X.store(2, Relaxed);
    t.join().unwrap();
    X.store(3, Relaxed);
}

fn f() {
    let x = X.load(Relaxed);
    assert!(x == 1 || x == 2);
}
```

Because of the happens-before relationships formed by the join and spawn operations, we know for sure that the load from X happens after the first store, but before the last store, as visualized in Figure 3-2. However, whether it observes the value before or after the second store is unpredictable. In other words, it could load either 1 or 2, but not 0 or 3.

更有趣的是，输出也可以是 0 20，尽管这四个操作没有任何可能的全局一致顺序会导致这个结果。当执行③时，与②之间没有先行发生关系，这意味着它可能会加载 0 或 20。当执行④时，与①之间没有先行发生关系，这意味着它可能会加载 0 或 10。基于此，输出 0 20 是一个有效的结果。

一个重要且违反直觉的理解是，加载数值 20 的操作③并不会与②产生先行发生关系，即使该数值是由②存储的。当我们面对的事情并不一定按全局一致顺序发生时（例如涉及指令重排时），我们对“之前”这一概念的直观理解就会失效。

一个更实用、更直观但不够正式的理解是：从执行 b 的线程角度来看，操作①和②看起来可能像是按相反的顺序发生的。

创建线程 和 合并线程

创建线程在 spawn() 调用之前发生的事情与新线程之间建立了一种先行发生关系。同样，合并线程在被合并的线程与 join() 调用之后发生的事情之间建立了一种先行发生关系。

为了演示这一点，以下示例中的断言不可能失败：

```
static X: AtomicI32 = AtomicI32::new(0);

fn main() {
    X.store(1, Relaxed);
    let t = thread::spawn(f);
    X.store(2, Relaxed);
    t.join().unwrap();
    X.store(3, Relaxed);
}

fn f() {
    let x = X.load(Relaxed);
    assert!(x == 1 || x == 2);
}
```

由于 join 和 spawn 操作形成的先行发生关系，我们可以肯定地知道，从 x 加载的操作发生在第一次存储之后，但在最后一次存储之前，如图 3-2 所示。然而，它是在第二次存储之前还是之后观察到该值是不可预测的。换句话说，它加载到的值可能是 1 或 2，但不是 0 或 3。

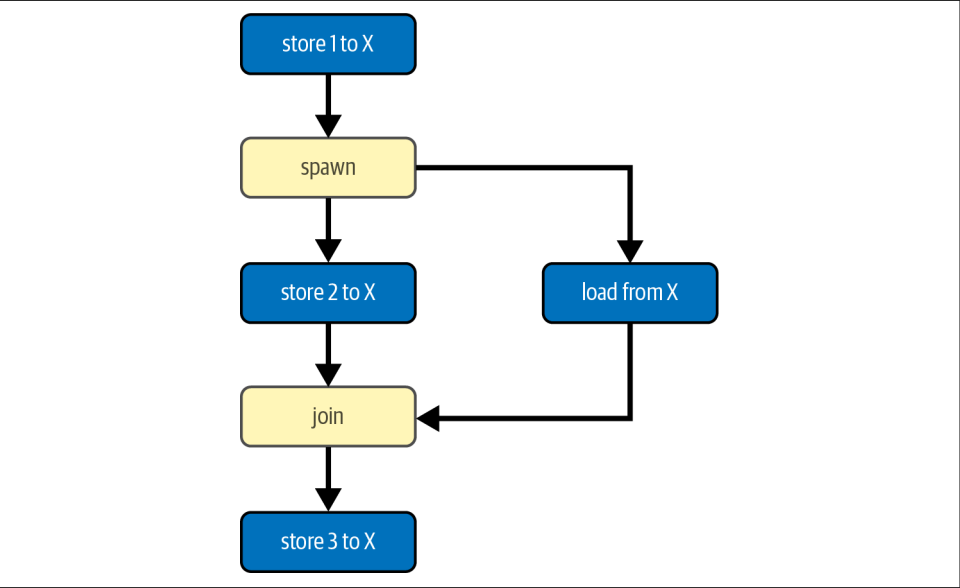


Figure 3-2. The happens-before relationships between the *spawn*, *join*, *store*, and *load* operations in the example code

```
static X: AtomicI32 = AtomicI32::new(0);

fn main() {
    X.store(1, Relaxed);
    let t = thread::spawn(f);
    X.store(2, Relaxed);
    t.join().unwrap();
    X.store(3, Relaxed);
}

fn f() {
    let x = X.load(Relaxed);
    assert!(x == 1 || x == 2);
}
```

Relaxed Ordering

While atomic operations using relaxed memory ordering do not provide any happens-before relationship, they do guarantee a *total modification order* of each individual atomic variable. This means that all modifications of the same atomic variable happen in an order that is the same from the perspective of every single thread.

To demonstrate what that means, consider the following example where we assume a and b are concurrently executed by different threads:

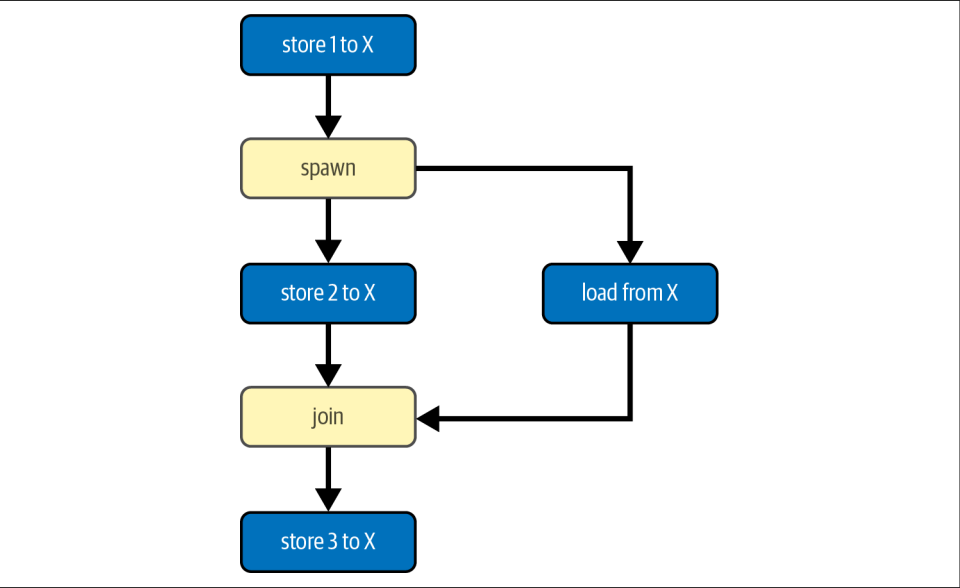


图 3-2. 示例代码中 *spawn*、*join*、存储和加载操作之间的先行发生关系

```
static X: AtomicI32 = AtomicI32::new(0);

fn main() {
    X.store(1, Relaxed);
    let t = thread::spawn(f);
    X.store(2, Relaxed);
    t.join().unwrap();
    X.store(3, Relaxed);
}

fn f() {
    let x = X.load(Relaxed);
    assert!(x == 1 || x == 2);
}
```

宽松顺序

虽然使用宽松内存顺序的原子操作不提供任何先行发生关系，但它们确实保证了每个独立原子变量的全修改顺序。这意味着，从每一个线程的角度来看，同一个原子变量的所有修改都发生在相同的顺序中。

为了演示这意味着什么，请考虑以下示例，我们假设 a 和 b 由不同的线程并发执行：

```
static X: AtomicI32 = AtomicI32::new(0);

fn a() {
    X.fetch_add(5, Relaxed);
    X.fetch_add(10, Relaxed);
}

fn b() {
    let a = X.load(Relaxed);
    let b = X.load(Relaxed);
    let c = X.load(Relaxed);
    let d = X.load(Relaxed);
    println!("{a} {b} {c} {d}");
}
```

In this example, only one thread modifies `X`, which makes it easy to see that there's only one possible order of modification of `X`: $0 \rightarrow 5 \rightarrow 15$. It starts at zero, then becomes five, and is finally changed to fifteen. Threads cannot observe any values from `X` that are inconsistent with this total modification order. This means that `0 0 0 0`, `0 0 5`, `15`, and `0 15 15 15` are some of the possible results from the print statement in the other thread, while an output of `0 5 0 15` or `0 0 10 15` is impossible.

Even if there's more than one possible order of modification for an atomic variable, all threads will agree on a single order.

Let's replace `a` by two separate functions, `a1` and `a2`, which we assume are each executed by a separate thread:

```
fn a1() {
    X.fetch_add(5, Relaxed);
}

fn a2() {
    X.fetch_add(10, Relaxed);
}
```

Assuming these are the only threads modifying `X`, there are now two possible modification orders: either $0 \rightarrow 5 \rightarrow 15$, or $0 \rightarrow 10 \rightarrow 15$, depending on which `fetch_add` operation executes first. Whichever happens, all threads observe the same order. So, even if we have hundreds of additional threads all running our `b()` function, we know that if one of them prints a `10`, the order must be $0 \rightarrow 10 \rightarrow 15$ and none of them can possibly print a `5`. And vice versa.

In [Chapter 2](#), we saw several examples of use cases where this total modification order guarantee for individual variables is enough, making relaxed memory ordering sufficient. However, if we try anything more advanced beyond those examples, we'll quickly see we need something stronger than relaxed memory ordering.

```
static X: AtomicI32 = AtomicI32::new(0);

fn a() {
    X.fetch_add(5, Relaxed);
    X.fetch_add(10, Relaxed);
}

fn b() {
    let a = X.load(Relaxed);
    let b = X.load(Relaxed);
    let c = X.load(Relaxed);
    let d = X.load(Relaxed);
    println!("{a} {b} {c} {d}");
}
```

在这个示例中，只有一个线程修改 `x`，因此很容易看出 `x` 只有一种可能的修改顺序： $0 \rightarrow 5 \rightarrow 15$ 。它从零开始，然后变为 5，最后被改为 15。线程无法观察到任何与此总修改顺序不一致的 `x` 值。这意味着 `0 0 0 0`、`0 0 5 15` 和 `0 15 15 15` 是另一个线程中打印语句可能产生的一些结果，而输出 `0 5 0 15` 或 `0 0 10 15` 是不可能的。

即使一个原子变量有多种可能的修改顺序，所有线程也会对其中一个单一顺序达成一致。

让我们用两个独立的函数 `a1` 和 `a2` 来替换 `a`，我们假设这两个函数分别由不同的线程执行：

```
fn a1() {
    X.fetch_add(5, Relaxed);
}

fn a2() {
    X.fetch_add(10, Relaxed);
}
```

假设这些是唯一修改 `x` 的线程，现在有两种可能的修改顺序： $0 \rightarrow 5 \rightarrow 15$ 或 $0 \rightarrow 10 \rightarrow 15$ ，具体取决于哪个 `fetch_add` 操作先执行。无论发生哪种情况，所有线程观察到的顺序都是相同的。因此，即使我们有数百个额外的线程都在运行 `b()` 函数，我们也知道，如果其中一个线程打印了 `10`，那么顺序一定是 $0 \rightarrow 10 \rightarrow 15$ ，且没有一个线程可能打印出 `5`。反之亦然。

在[第 2 章](#)中，我们看到了几个关于单个变量的总修改顺序保证就足够的用例，这使得松散内存顺序足以满足需求。然而，如果我们尝试在这些示例之外进行更高级的操作，很快就会发现我们需要比松散内存顺序更强的保证。

Out-of-Thin-Air Values

The lack of ordering guarantees around relaxed memory ordering can lead to some theoretical complications when operations depend on each other in a cyclic way.

To demonstrate, here's a contrived example where two threads load a value from one atomic and store it in the other:

```
static X: AtomicI32 = AtomicI32::new(0);
static Y: AtomicI32 = AtomicI32::new(0);

fn main() {
    let a = thread::spawn(|| {
        let x = X.load(Relaxed);
        Y.store(x, Relaxed);
    });
    let b = thread::spawn(|| {
        let y = Y.load(Relaxed);
        X.store(y, Relaxed);
    });
    a.join().unwrap();
    b.join().unwrap();
    assert_eq!(X.load(Relaxed), 0); // Might fail?
    assert_eq!(Y.load(Relaxed), 0); // Might fail?
}
```

It seems easy to conclude that the values of X and Y will never be anything other than zero, since the store operations only store values that were loaded from these same atomics, which are only ever zero.

If we strictly follow the theoretical memory model, however, we have to come to terms with our cyclic reasoning, and come to the scary conclusion that we might be wrong. In fact, the memory model technically allows for an outcome where both X and Y are 37 in the end, or any other value, making the assertions fail.

Due to the lack of ordering guarantees, the load operations of these two threads might *both* see the result of the store operation of the other thread, allowing for a cycle in the order of operations: we store 37 in Y because we loaded 37 from X, which was stored to X because we loaded 37 from Y, which is the value we stored in Y.

Fortunately, the possibility of such *out-of-thin-air* values is universally considered to be a bug in the theoretical model, and not something you need to take into account in practice. The theoretical problem of how to formalize relaxed memory ordering without the model allowing for such anomalies is an unsolved one. While this is an eyesore for formal verification that keeps many theoreticians up at night, the rest of us can relax in blissful ignorance knowing that this does not happen in practice.

凭空产生的值

当操作之间存在循环依赖时，松散内存顺序缺乏内存顺序保证，可能会导致一些理论上的复杂问题。

为了演示，这里有一个人为设计的例子，其中两个线程从一个原子操作变量中加载一个值，并将其存储到另一个原子操作变量中：

```
static X: AtomicI32 = AtomicI32::new(0);
static Y: AtomicI32 = AtomicI32::new(0);

fn main() {
    let a = thread::spawn(|| {
        let x = X.load(Relaxed);
        Y.store(x, Relaxed);
    });
    let b = thread::spawn(|| {
        let y = Y.load(Relaxed);
        X.store(y, Relaxed);
    });
    a.join().unwrap();
    b.join().unwrap();
    assert_eq!(X.load(Relaxed), 0); // Might fail?
    assert_eq!(Y.load(Relaxed), 0); // Might fail?
}
```

人们很容易得出结论，x 和 y 的值永远不会是零以外的任何值，因为存储操作仅存储从这些相同的原子操作变量中加载的值，而这些值始终为零。

然而，如果我们严格遵守理论上的内存模型，我们就必须面对我们的循环推理，并得出一个可怕的结论：我们可能是错的。事实上，从技术上讲，内存模型允许一种结果，即 x 和 y 最终都为 37 或任何其他值，从而导致断言失败。

由于缺乏内存顺序保证，这两个线程的加载操作可能同时看到另一个线程的存储操作结果，从而导致操作顺序中出现循环：我们将 37 存储在 Y 中，是因为我们从 x 中加载了 37，而 x 是因为我们从 Y 中加载了 37 而存储到 x 中的，而 Y 正是我们存储在 Y 中的值。

幸运的是，这种凭空产生 (*out-of-thin-air*) 值的可能性被普遍认为是理论模型中的一个 bug，而不是你在实践中需要考虑的问题。关于如何在不允许此类异常的情况下对松散内存顺序进行形式化定义，这一理论问题目前尚未解决。虽然这对从事形式化验证的人来说是一个令他们夜不能寐的痛点，但我们其他人可以在欣慰的无知中放松，因为这种情况在实践中并不会发生。

Release and Acquire Ordering

Release and *acquire* memory ordering are used in a pair to form a happens-before relationship between threads. Release memory ordering applies to store operations, while Acquire memory ordering applies to load operations.

A happens-before relationship is formed when an acquire-load operation observes the result of a release-store operation. In this case, the store and everything before it, happened before the load and everything after it.

When using Acquire for a fetch-and-modify or compare-and-exchange operation, it applies only to the part of the operation that loads the value. Similarly, Release applies only to the store part of an operation. AcqRel is used to represent the combination of Acquire and Release, which causes both the load to use acquire ordering, and the store to use release ordering.

Let's go over an example to see how that's used in practice. In the following example, we send an 64-bit integer from a spawned thread to the main thread. We use an extra atomic boolean to indicate to the main thread that the integer has been stored and is ready to be read.

```
use std::sync::atomic::Ordering::{Acquire, Release};

static DATA: AtomicU64 = AtomicU64::new(0);
static READY: AtomicBool = AtomicBool::new(false);

fn main() {
    thread::spawn(|| {
        DATA.store(123, Relaxed);
        READY.store(true, Release); // Everything from before this store ..
    });
    while !READY.load(Acquire) { // .. is visible after this loads `true`.
        thread::sleep(Duration::from_millis(100));
        println!("waiting...");
    }
    println!("{}", DATA.load(Relaxed));
}
```

When the spawned thread is done storing the data, it uses a release-store to set the READY flag to true. When the main thread observes this through its acquire-load operation, a happens-before relationship is established between those two operations, as shown in **Figure 3-3**. At that point, we know for sure that everything that happened before the release-store to READY is visible to everything that happens after the acquire-load. Specifically, when the main thread loads from DATA, we know for sure it will load the value stored by the background thread. There's only one possible outcome this program can print on its last line: 123.

释放-获取(Release and Acquire)排序

释放与获取内存顺序成对使用，以在线程之间建立先行发生关系。Release 内存顺序适用于存储操作，而Acquire 内存顺序适用于加载操作。

当 acquire-load 操作观察到 release-store 操作的结果时，就会形成一种先行发生关系。在这种情况下，存储操作及其之前的所有内容，都先行发生于加载操作及其之后的所有内容。

当在获取并修改或比较并交换操作中使用 Acquire 时，它仅适用于该操作中加载值的部分。同样，Release 仅适用于操作中的存储部分。AcqRel 用于表示 Acquire 和 Release 的结合，它会使加载操作使用获取顺序，同时使存储操作使用释放顺序。

让我们通过一个示例来看看这在实践中是如何使用的。在接下来的示例中，我们从一个 spawn 的线程向主线程发送一个 64 位整数。我们使用一个额外的原子布尔值来向主线程指示该整数已被存储且可以读取。

```
use std::sync::atomic::Ordering::{Acquire, Release};

static DATA: AtomicU64 = AtomicU64::new(0);
static READY: AtomicBool = AtomicBool::new(false);

fn main() {
    thread::spawn(|| {
        DATA.store(123, Relaxed);
        READY.store(true, Release); // Everything from before this store ..
    });
    while !READY.load(Acquire) { // .. is visible after this loads `true`.
        thread::sleep(Duration::from_millis(100));
        println!("waiting...");
    }
    println!("{}", DATA.load(Relaxed));
}
```

当生成的线程完成数据存储后，它会使用释放-存储 (release-store) 将 READY 标志设置为 true。当主线程通过其获取-加载 (acquire-load) 操作观察到这一变化时，这两个操作之间便建立起了先行发生 (happens-before) 关系，如图 3-3 所示。此时，我们可以确信，在释放-存储到 READY 之前发生的所有事情，对于在获取-加载之后发生的所有事情都是可见的。具体而言，当主线程从 DATA 中加载数据时，我们确信它将加载到后台线程所存储的值。因此，该程序最后一行打印的结果只可能是一个值：123。

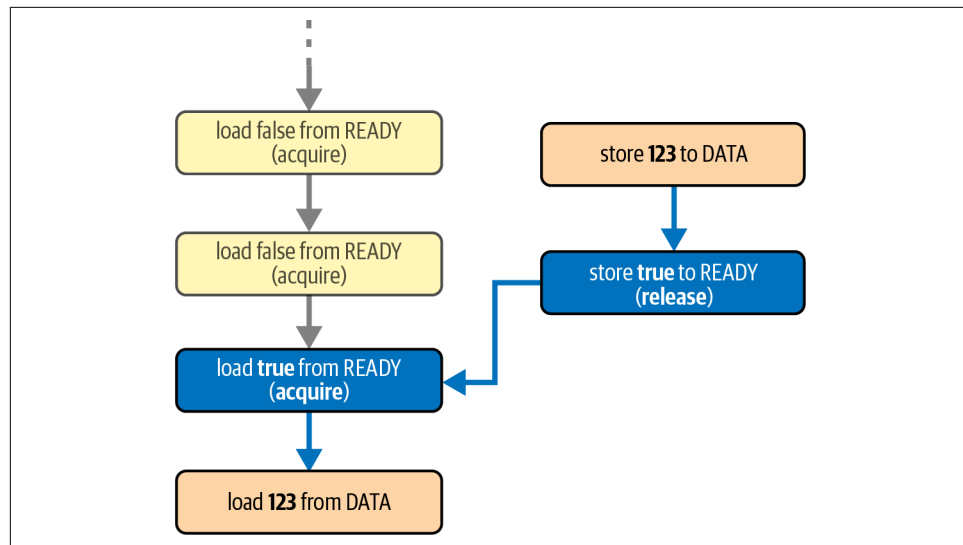


Figure 3-3. The happens-before relationships between atomic operations in the example code, showing the cross-thread relationship formed by the acquire and release operations

If we had used relaxed memory ordering for all operations in this example, the main thread could have seen `READY` flip to `true`, while still loading a zero from `DATA` afterwards.



The names “release” and “acquire” are based on their most basic use case: one thread *releases* data by atomically storing some value to an atomic variable, and another thread *acquires* it by atomically loading that value. This is exactly what happens when we unlock (release) a mutex and subsequently lock (acquire) it on another thread.

In our example, the happens-before relationship from the `READY` flag guarantees that the store and load operations of `DATA` cannot happen concurrently. This means that we don’t actually need those operations to be atomic.

However, if we simply try to use a regular non-atomic type for our data variable, the compiler will refuse our program, since Rust’s type system doesn’t allow us to mutate those from one thread when another thread is also borrowing them. The type system does not magically understand the happens-before relationship we’ve created here. Some `unsafe` code is necessary to promise to the compiler that we’ve thought about this carefully and we’re sure we’re not breaking any rules, as follows:

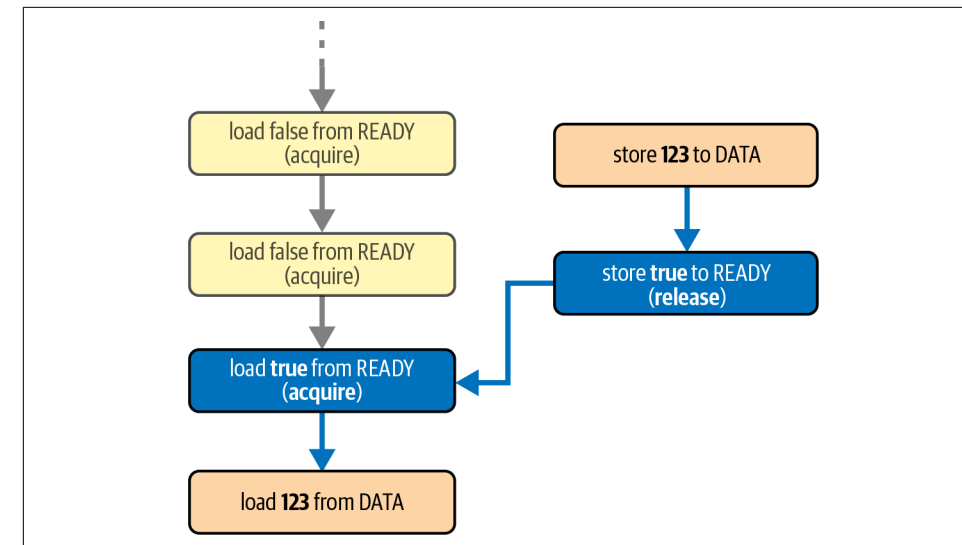


图 3-3。示例代码中原子操作之间的先行发生关系，展示了由获取内存顺序和释放语义操作形成的跨线程关系

如果在本例的所有操作中我们都使用了松散内存顺序，那么主线程可能会看到 `READY` 翻转为 `true`，但随后从 `DATA` 加载到的仍然是零。



“释放语义 (release)”和“获取内存顺序 (acquire)”这两个名称基于它们最基本的用例：一个线程通过原子地将某个值存储到原子变量中来释放数据，而另一个线程通过原子地加载该值来获取它。这正是我们在一个线程上解锁（释放）互斥锁，随后在另一个线程上锁定（获取）它时所发生的情况。

在我们的示例中，来自 `READY` flag 的先行发生关系保证了 `DATA` 的存储和加载操作不能并发发生。这意味着我们实际上并不需要这些操作是原子的。

然而，如果我们尝试为数据变量简单地使用一个常规的非原子的类型，编译器将会拒绝我们的程序，因为 Rust 的类型系统不允许我们在另一个线程地借用这些变量时，在其中一个线程中对其进行修改。类型系统无法神奇地理解我们在这里创建的先行发生关系。我们需要一些 `unsafe` 代码来向编译器保证，我们已经仔细考虑过这一点，并且确信自己没有违反任何规则，如下所示：

```
static mut DATA: u64 = 0;
static READY: AtomicBool = AtomicBool::new(false);

fn main() {
    thread::spawn(|| {
        // Safety: Nothing else is accessing DATA,
        // because we haven't set the READY flag yet.
        unsafe { DATA = 123 };
        READY.store(true, Release); // Everything from before this store ..
    });
    while !READY.load(Acquire) { // .. is visible after this loads `true`.
        thread::sleep(Duration::from_millis(100));
        println!("waiting...");
    }
    // Safety: Nothing is mutating DATA, because READY is set.
    println!("{}", unsafe { DATA });
}
```

More Formally

A happens-before relationship is formed when an acquire-load operation observes the result of a release-store operation. But what does that mean?

Imagine that two threads both release-store a seven into the same atomic variable, and a third thread loads a seven from that variable. Does the third thread now have a happens-before relationship with the first thread or the second one? That depends on “which seven” it loaded: the one from thread one or the one from thread two. (Or perhaps an unrelated seven.) This leads us to the conclusion that even though seven equals seven, there is something different about the two sevens from the two threads.

The way to think about this is in terms of the *total modification order* that we talked about in “Relaxed Ordering” on page 54: the ordered list of all modifications that happen to an atomic variable. Even if the same value is written to the same variable more than once, each of these operations represents a separate event in the total modification order of that variable. When we load a value, the value loaded matches a specific point on this per-variable “timeline,” which tells us which operation we might be synchronizing with.

For example, if the total modification order of the atomic is

- 1. Initialized at 0
- 2. **Release**-store 7 (from thread two)
- 3. Relaxed-store 6
- 4. **Release**-store 7 (from thread one)

then acquire-loading a 7 would synchronize with either the release-store from the second event, or the release-store from the last event. However, if we have previously

```
static mut DATA: u64 = 0;
static READY: AtomicBool = AtomicBool::new(false);

fn main() {
    thread::spawn(|| {
        // Safety: Nothing else is accessing DATA,
        // because we haven't set the READY flag yet.
        unsafe { DATA = 123 };
        READY.store(true, Release); // Everything from before this store ..
    });
    while !READY.load(Acquire) { // .. is visible after this loads `true`.
        thread::sleep(Duration::from_millis(100));
        println!("waiting...");
    }
    // Safety: Nothing is mutating DATA, because READY is set.
    println!("{}", unsafe { DATA });
}
```

更正式地定义

当一次 acquire-load 操作观察到一次 release-store 操作的结果时，就形成了一个先行发生关系。但这意味着什么呢？

想象一下，有两个线程都将数字 7 通过 release-store 存储到同一个原子变量中，而第三个线程从该变量中加载了数字 7。那么，第三个线程现在与第一个线程还是第二个线程具有先行发生关系？这取决于它加载的是“哪个 7”：是来自线程一的，还是来自线程二的。（或者可能是一个无关的 7。）这让我们得出结论：尽管 7 等于 7，但来自两个线程的这两个 7 之间存在某种差异。

思考这个问题的切入点是我们在 第 54 页的“宽松顺序”中讨论过的 总修改顺序：即对一个原子变量进行的所有修改的有序列表。即使同一个值被多次写入同一个变量，这些操作中的每一次在那个变量的总修改顺序中都代表一个独立的事件。当我们加载一个值时，加载的值对应于这个每个变量特有的“时间线”上的一个特定点，这告诉了我们我们可能在与哪个操作进行同步。

例如，如果该原子的总修改顺序为

- 1. 初始化为0
- 2. **释放语义**-存储7 （来自线程二）
- 3. 宽松存储 6
- 4. **释放语义**-存储 7（来自线程一）

那么获取加载一个 7 将会与第二个事件的释放存储，或者最后一个事件的释放存储同步。然而，如果我们之前

(in terms of happens-before relationships) seen a 6, we know we’re seeing the last 7, not the first one, meaning we now have a happens-before relationship with thread one, and not with thread two.

There is one extra detail, which is that a release-stored value may be modified by any number of fetch-and-modify and compare-and-exchange operations, while still resulting in a happens-before relationship with an acquire-load that reads the final result.

For example, imagine an atomic variable with the following total modification order:

1. Initialized at 0
2. **Release**-store 7
3. Relaxed-fetch-and-add 1, changing 7 to 8
4. **Release**-fetch-and-add 1, changing 8 to 9
5. **Release**-store 7
6. Relaxed-swap 10, changing 7 to 10

Now, if we acquire-load a 9 from this variable, we not only establish a happens-before relationship with the fourth operation (that stored this value), but also with the second operation (which stored a 7), even though the third operation used relaxed memory ordering.

Similarly, if we acquire-load a 10 from this variable, which was written by a relaxed operation, we still establish a happens-before relationship with the fifth operation (which stored a 7). Because that was just a regular store operation (not a fetch-and-modify or compare-and-exchange operation), it breaks the chain: we don’t establish a happens-before relationship with any of the other operations.

Example: Locking

Mutexes are the most common use case for release and acquire ordering (see “**Locking: Mutexes and RwLocks**” on page 18). When locking, they use an atomic operation to check if it was unlocked, using acquire ordering, while also (atomically) changing the state to “locked.” When unlocking, they set the state back to “unlocked” using release ordering. This means that there will be a happens-before relationship between unlocking a mutex and subsequently locking it.

（就先行发生关系而言）已经看到了 6，我们就知道我们看到的是最后一个 7 而非第一个，这意味着我们现在与线程 1 具有先行发生关系，而不是与线程 2。

还有一个额外的细节，即 release-store 存储值可以被任意数量的获取并修改 (fetch-and-modify) 和比较并交换 (compare-and-exchange) 操作所修改，而仍然能与读取最终结果的获取加载 (acquire-load) 建立先行发生 (happens-before) 关系。

例如，假设一个原子变量具有以下总修改顺序：

1. 初始化为0 .
2. **释放语义**存储7 .
3. Relaxed-fetch-and-add 1, 将 7 更改为8
4. **释放语义**获取并增加 1, 将 8 更改为9
5. **释放语义**存储 7
6. Relaxed-swap 10, 将 7 更改为 10 .

现在，如果从这个变量中获取加载 (acquire-load 一个 9，我们不仅与第四个操作（存储该值的操作）建立了先行发生关系，而且还与第二个操作（存储 7 的操作）建立了关系，尽管第三个操作使用的是松散内存顺序。

同样地，如果从这个变量中获取加载（acquire-load）了一个 10，而这个值是由一个宽松（relaxed）操作写入的，我们仍然能与第五个操作（存储了 7 的那个）建立先行发生（happens-before）关系。但因为那只是一个普通的存储操作（而非 fetch-and-modify 或比较并交换操作），它切断了链条：我们无法与任何其他操作建立先行发生关系。

示例：加锁

互斥锁是使用释放与获取内存顺序最常见的场景（参见第 18 页的“**加锁：互斥锁与读写锁**”）。在加锁时，它们会使用获取顺序通过一个原子操作来检查是否已解锁，同时（原子地）将状态更改为“已锁定”。在解锁时，它们会使用释放顺序将状态重新设为“已解锁”。这意味着，在解锁一个互斥锁与随后对其加锁之间将存在一个先行发生关系。

Here's a demonstration of this pattern:

```
static mut DATA: String = String::new();
static LOCKED: AtomicBool = AtomicBool::new(false);

fn f() {
    if LOCKED.compare_exchange(false, true, Acquire, Relaxed).is_ok() {
        // Safety: We hold the exclusive lock, so nothing else is accessing DATA.
        unsafe { DATA.push('!') };
        LOCKED.store(false, Release);
    }
}

fn main() {
    thread::scope(|s| {
        for _ in 0..100 {
            s.spawn(f);
        }
    });
}
```

As we've briefly seen in “Compare-and-Exchange Operations” on page 42, compare-and-exchange operations take two memory ordering arguments: one for the case where the comparison succeeded and the store happened, and one for the case where the comparison failed and the store did not happen. In `f`, we attempt to change `LOCKED` from `false` to `true`, and only access `DATA` if that succeeds. So, we only care about the success memory ordering. If the `compare_exchange` operation fails, that must be because `LOCKED` was already set to `true`, in which case `f` doesn't do anything. This matches the `try_lock` operation on a regular mutex.



An observant reader might have already noticed that the compare-and-exchange operation could also have been a swap operation, since swapping `true` for `true` when already locked doesn't change the correctness of the code:

```
// This also works.
if LOCKED.swap(true, Acquire) == false {
    ...
}
```

Thanks to the acquire and release memory ordering, we know for sure that no two threads can concurrently access `DATA`. As visualized in Figure 3-4, any previous access to `DATA` happened-before the subsequent release-store of `false` to `LOCKED`, which in turn happened-before the next acquire-compare-and-exchange (or acquire-swap) operation that changed that `false` to `true`, which happened-before the next access to `DATA`.

下面是对该模式的演示：

```
static mut DATA: String = String::new();
static LOCKED: AtomicBool = AtomicBool::new(false);

fn f() {
    if LOCKED.compare_exchange(false, true, Acquire, Relaxed).is_ok() {
        // Safety: We hold the exclusive lock, so nothing else is accessing DATA.
        unsafe { DATA.push('!') };
        LOCKED.store(false, Release);
    }
}

fn main() {
    thread::scope(|s| {
        for _ in 0..100 {
            s.spawn(f);
        }
    });
}
```

正如我们在第 42 页的“比较并交换操作”中简要看到的，比较并交换操作接受两个内存排序参数：一个用于比较成功且执行了存储的情况，另一个用于比较失败且未执行存储的情况。在 `f` 中，我们尝试将 `LOCKED` 从 `false` 更改为 `true`，并且只有在成功时才访问 `DATA`。因此，我们只关心成功的内存排序。如果 `compare_exchange` 操作失败，那一定是由于 `LOCKED` 已被设置为 `true`，在这种情况下，`f` 不执行任何操作。这与在普通互斥锁上的 `try_lock` 操作一致。



细心的读者可能已经注意到，比较并交换操作也可以是一个 swap 操作，因为在已经锁定时将 `true` swap 为 `true` 并不会改变代码的正确性：

```
// This also works.
if LOCKED.swap(true, Acquire) == false {
    ...
}
```

得益于获取内存顺序和释放内存顺序，我们可以肯定没有两个线程能同时访问 `DATA`。正如图 3-4 所示，任何对 `DATA` 的先前访问都先行发生于随后对 `false` 到 `LOCKED` 的释放存储操作，而该操作又先行发生于下一个将 `false` 更改为 `true` 的获取比较并交换（或获取 swap）操作，而后者又先行发生于下一次对 `DATA` 的访问。

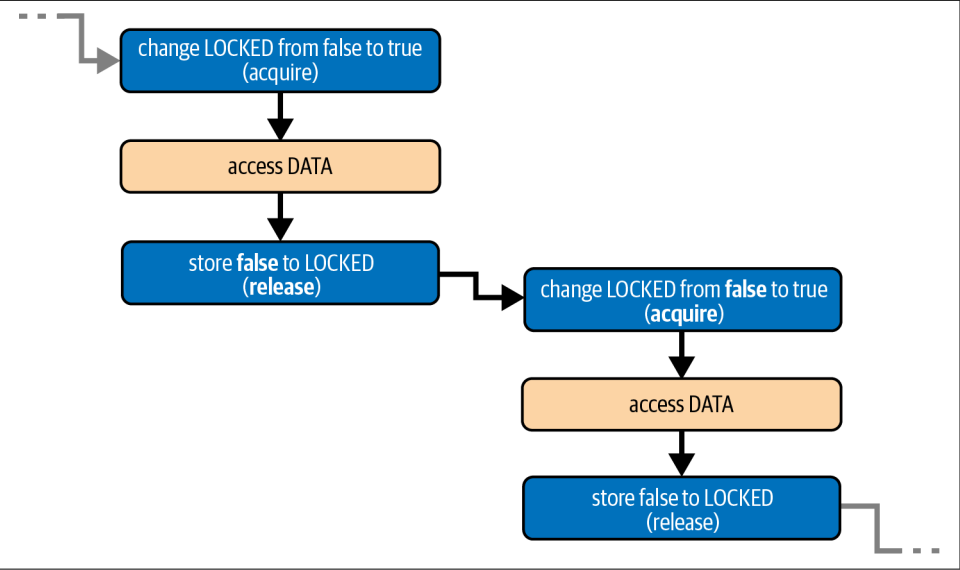


Figure 3-4. The happens-before relationships between atomic operations in the locking example, showing two threads locking and unlocking in sequence

In [Chapter 4](#) we'll turn this concept into a reusable type: a spin lock.

Example: Lazy Initialization with Indirection

In “[Example: Lazy One-Time Initialization](#)” on [page 45](#), we implemented lazy initialization of a global variable, using a compare-and-exchange operation to handle situations where multiple threads race to initialize the value concurrently. Because the value was a nonzero 64-bit integer, we were able to use an `AtomicU64` to store it, using zero as the placeholder before initializing it.

To do the same for a much larger data type that does not fit in a single atomic variable, we need to look for an alternative.

For this example, let's say we want to maintain the non-blocking behavior, so that threads never wait for another thread, but instead race and take the value from the first thread to complete initialization. This means we still need to be able to go from “uninitialized” to “fully initialized” in a single atomic operation.

As the fundamental theorem of software engineering tells us, every problem in computer science can be solved by adding another layer of indirection, and this problem is no different. Since we can't fit the data into a single atomic variable, we can instead use an atomic variable to store a *pointer* to the data.

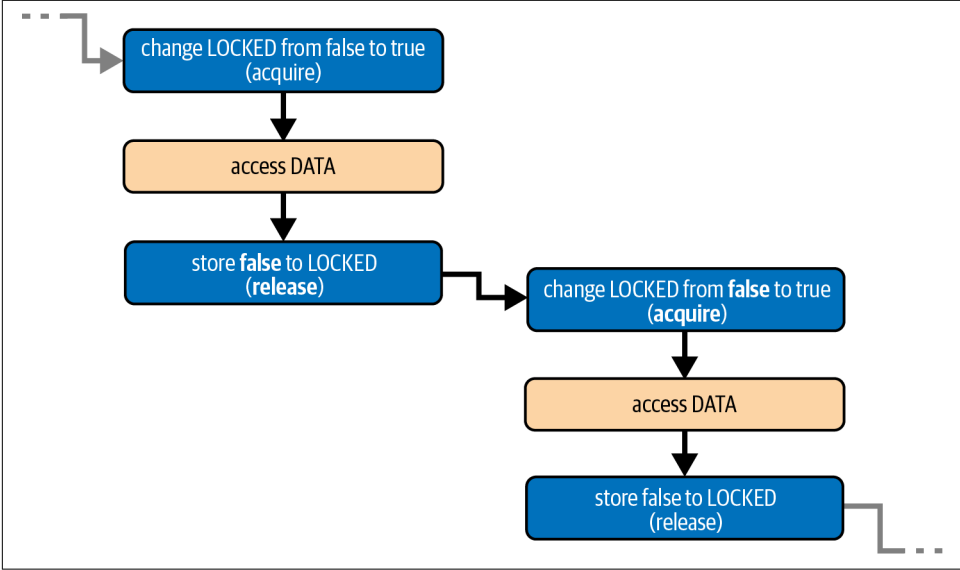


图 3-4. 加锁示例中原子操作之间的先行发生关系，展示了两个线程按顺序进行加锁和解锁的过程

在 [第 4 章](#) 中，我们将把这个概念转化为一个可复用的类型：自旋锁。

示例：使用间接层的延迟初始化

在 [第 45 页](#) 的“[示例：延迟一次性初始化](#)”中，我们实现了全局变量的延迟初始化，并使用比较并交换操作来处理多个线程竞态地并发初始化该值的情况。由于该值是一个非零的 64 位整数，我们能够使用 `AtomicU64` 来存储它，并在初始化之前使用零作为占位符。

对于无法放入单个原子变量的更大数据类型，如果想实现同样的功能，我们需要寻找另一种替代方案。

在这个示例中，假设我们希望保持无锁行为，使得线程永远不需要等待另一个线程，而是通过竞态并从第一个完成初始化的线程中获取该值。这意味着我们仍然需要能够在单个原子操作中从“未初始化”状态转变为“完全初始化”状态。

正如软件工程基本定理告诉我们的，计算机科学中的任何问题都可以通过增加一层间接层来解决，这个问题也不例外。既然我们无法将数据放入单个原子变量中，那么我们可以改用一个原子变量来存储一个指向该数据的指针。

An `AtomicPtr<T>` is the atomic version of a `*mut T`: a pointer to `T`. We can use a null pointer as the placeholder for the initial state, and use a compare-and-exchange operation to atomically replace it with a pointer to a newly allocated, fully initialized `T`, which can then be read by the other threads.

Since we're not only sharing the atomic variable containing the pointer, but also the data it points to, we can no longer use relaxed memory ordering like in [Chapter 2](#). We need to make sure that allocating and initializing the data does not race with reading it. In other words, we need to use release and acquire ordering on the store and load operations, to make sure the compiler and processor won't break our code by—for example—reordering the store of the pointer and the initialization of the data itself.

This leads us to the following implementation, for some arbitrary data type called `Data`:

```
use std::sync::atomic::AtomicPtr;

fn get_data() -> &'static Data {
    static PTR: AtomicPtr<Data> = AtomicPtr::new(std::ptr::null_mut());

    let mut p = PTR.load(Acquire);

    if p.is_null() {
        p = Box::into_raw(Box::new(generate_data()));
        if let Err(e) = PTR.compare_exchange(
            std::ptr::null_mut(), p, Release, Acquire
        ) {
            // Safety: p comes from Box::into_raw right above,
            // and wasn't shared with any other thread.
            drop(unsafe { Box::from_raw(p) });
            p = e;
        }
    }

    // Safety: p is not null and points to a properly initialized value.
    unsafe { &*p }
}
```

If the pointer we acquire-load from `PTR` is non-null, we assume it points to the already initialized data, and construct a reference to that data.

If it's still null, however, we generate new data and store it in a new allocation using `Box::new`. We then turn this `Box` into a raw pointer using `Box::into_raw`, so we can attempt to store it into `PTR` using a compare-and-exchange operation. If another thread wins the initialization race, `compare_exchange` fails as `PTR` is no longer null. If that happens, we turn our raw pointer back into a `Box` to deallocate it using `drop`, avoiding a memory leak, and continue with the pointer that the other thread stored in `PTR`.

`AtomicPtr<T>` 是 `*mut T` 的原子版本：一个指向 `T` 的指针。我们可以使用空指针作为初始状态的占位符，并使用比较并交换操作将其原子地替换为指向一个新分配且完全初始化好的 `T` 的指针，随后其他线程就可以读取该指针。

由于我们不仅在共享包含指针的原子变量，还在共享它所指向的数据，因此我们不能像[第 2 章](#)那样使用松散内存顺序。我们需要确保数据的分配和初始化不会与读取操作产生竞态。换句话说，我们需要在存储和加载操作上使用释放与获取排序，以确保编译器和处理器不会通过（例如）对指针的存储和数据本身的初始化进行重排序来破坏我们的代码。

这引导我们得到了针对名为 `Data` 的某种任意数据类型的以下实现：

```
use std::sync::atomic::AtomicPtr;

fn get_data() -> &'static Data {
    static PTR: AtomicPtr<Data> = AtomicPtr::new(std::ptr::null_mut());

    let mut p = PTR.load(Acquire);

    if p.is_null() {
        p = Box::into_raw(Box::new(generate_data()));
        if let Err(e) = PTR.compare_exchange(
            std::ptr::null_mut(), p, Release, Acquire
        ) {
            // Safety: p comes from Box::into_raw right above,
            // and wasn't shared with any other thread.
            drop(unsafe { Box::from_raw(p) });
            p = e;
        }
    }

    // Safety: p is not null and points to a properly initialized value.
    unsafe { &*p }
}
```

如果从 `PTR` 进行获取加载得到的指针是非空的，我们就假设它指向已经初始化好的数据，并构造该数据的引用。

然而，如果它仍然为 null，我们会生成新数据，并使用 `Box::new` 将其存储在新的内存分配中。然后，我们使用 `Box::into_raw` 将这个 `Box` 转换为原始指针，以便尝试使用比较并交换操作将其存储到 `PTR` 中。如果另一个线程赢得了初始化竞态，由于 `PTR` 不再为 null，`compare_exchange` 将会失败。如果发生这种情况，我们会将原始指针转回 `Box`，以便使用 `drop` 对其进行内存释放，从而避免内存泄漏，并继续使用另一个线程存储在 `PTR` 中的指针。

The safety comment on the final `unsafe` block states our assumption that the data it points to has already been initialized. Note how this includes an assumption about the order in which things happened. To make sure our assumption holds, we use release and acquire memory ordering to make sure initializing the data has actually *happened-before* creating a reference to it.

We load a potentially non-null (i.e., initialized) pointer in two places: through the load operation and through the `compare_exchange` operation when it fails. So, as explained above, we need to use `Acquire` for both the load memory ordering and the `compare_exchange` failure memory ordering, to be able to synchronize with the operation that stores the pointer. This store happens when the `compare_exchange` operation succeeds, so we must use `Release` as its success ordering.

Figure 3-5 shows a visualization of the operations and happens-before relationships for a situation in which three threads call `get_data()`. In this situation, thread A and B both observe a null pointer and both attempt to initialize the atomic pointer. Thread A wins that race, causing thread B's `compare_exchange` operation to fail. Thread C only observes the atomic pointer after it has been initialized by thread A. The end result is that all three threads end up using the box that was allocated by thread A.

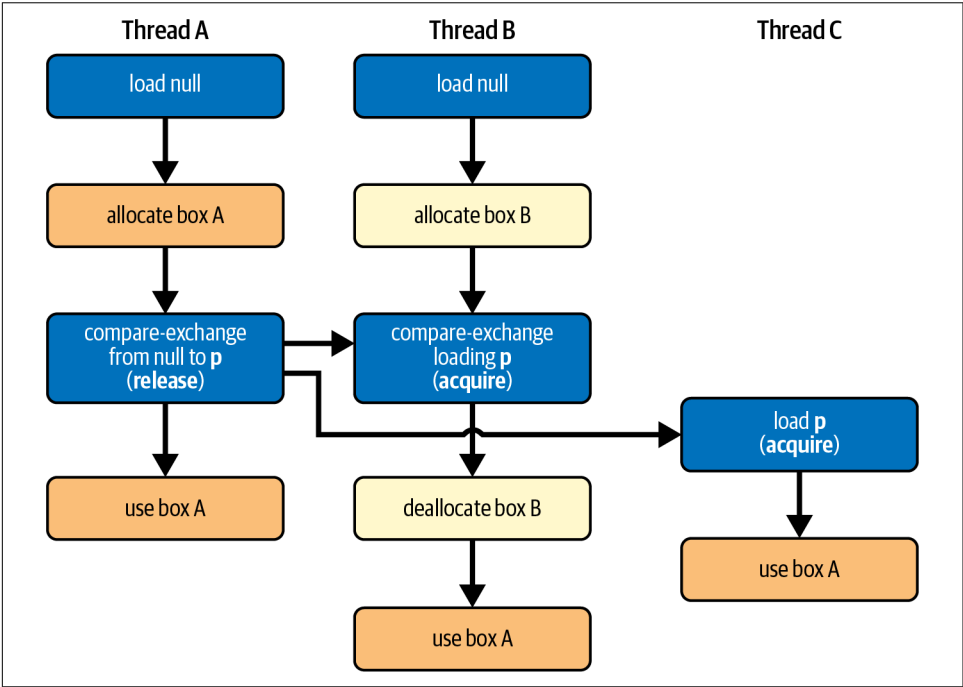


Figure 3-5. The operations and happens-before relationships among three threads calling `get_data()`

最后一个 `unsafe` 代码块中的安全注释阐述了我们的假设，即它所指向的数据已经完成了初始化。请注意，这其中包含了一个关于事件发生顺序的假设。为了确保我们的假设成立，我们使用释放-获取内存顺序，以确保初始化数据实际上先行发生于创建指向该数据的引用。

我们在两个地方加载了一个可能非空（即已初始化）的指针：通过 `load` 操作，以及在 `compare_exchange` 操作失败时。因此，如上所述，我们需要在 `load` 内存顺序和 `compare_exchange` 失败内存顺序中都使用 `Acquire`，以便能够与存储该指针的操作同步。这次存储发生在 `compare_exchange` 操作成功时，因此我们必须使用 `Release` 作为其成功内存顺序。

图 3-5 展示了三个线程调用 `get_data()` 时，操作及其先行发生关系的视觉化呈现。在这种情况下，线程 A 和 B 都观察到了空指针，并都尝试初始化该原子指针。线程 A 在这场竞态中获胜，导致线程 B 的 `compare_exchange` 操作失败。线程 C 仅在线程 A 完成初始化后才观察到该原子指针。最终结果是，所有三个线程最终都使用了由线程 A 分配的 Box。

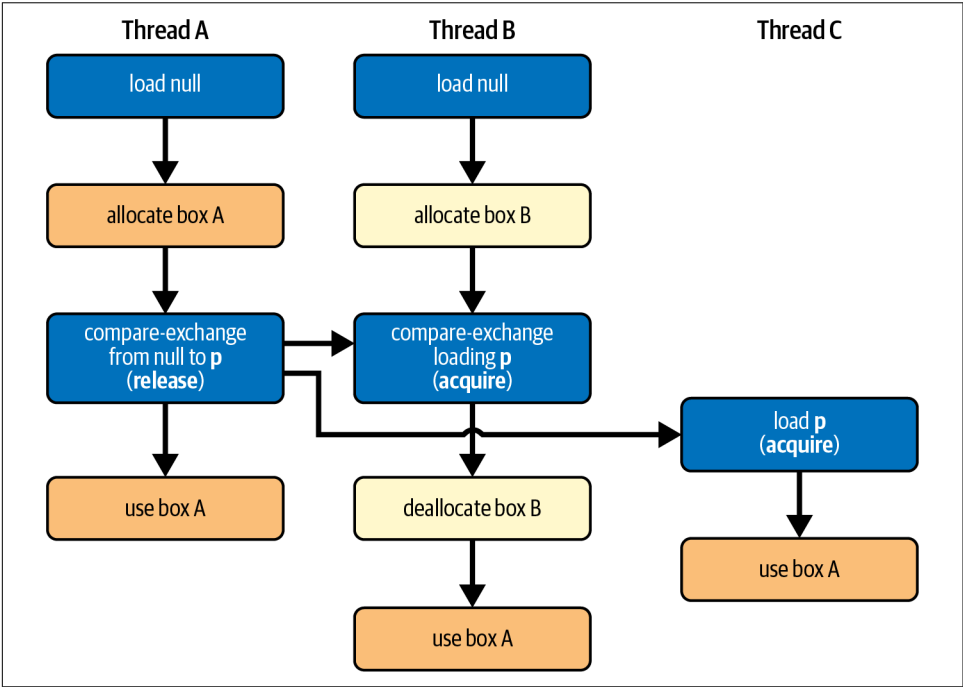


图 3-5. 三个线程调用 `get_data()` 时的操作及其之间的先行发生关系

Consume Ordering

Let’s take a closer look at the memory ordering in our last example. If we leave the strict memory model aside and think of it in more practical terms, we could say that the release ordering prevents the initialization of the data from being reordered with the store operation that shares the pointer with the other threads. This is important, since otherwise other threads might be able to see the data before it’s fully initialized.

Similarly, we could explain the acquire ordering as preventing reordering that would cause the data to be accessed before the pointer is loaded. One might reasonably wonder, however, if that makes any sense in practice. How could the data be accessed before its address is known? We might conclude that something weaker than acquire ordering might suffice. And we would be right: this weaker ordering is called *consume ordering*.

Consume ordering is basically a lightweight—more efficient—variant of acquire ordering, whose synchronizing effects are limited to things that *depend on* the loaded value.

What that means is that if you consume-load a release-stored value `x` from an atomic variable, then, basically, that store happened before the evaluation of *dependent* expressions like `*x`, `array[x]` or `table.lookup(x + 1)`, but *not* necessarily before independent operations, like reading another variable that we don’t need the value of `x` for.

Now there’s good news and bad news.

The good news is that—on all modern processor architectures—consume ordering is achieved with the exact same instructions as relaxed ordering. In other words, consume ordering can be “free,” which—at least on some platforms—is not the case for acquire ordering.

The bad news is that no compiler actually implements consume ordering.

As it turns out, not only is this concept of a “dependent” evaluation hard to define, it’s even harder to keep such dependencies intact while transforming and optimizing a program. For example, a compiler might be able to optimize `x + 2 - x` to just `2`, effectively dropping the dependency on `x`. More subtle variations of this issue can happen with more realistic expressions like `array[x]`, if the compiler is able to make any logical deductions about the possible values of `x` or the array’s elements. The issue gets even more complicated when taking control flow into account, like `if` statements or function calls.

Because of this, compilers upgrade consume ordering to acquire ordering, just to be safe. The C++20 standard even explicitly discourages the use of consume ordering,

消费顺序

让我们仔细分析一下上个示例中的内存顺序。如果我们暂时撇开严格的内存模型，用更实际的方式来思考，可以说释放顺序防止了数据的初始化与将指针共享给其他线程的存储操作之间发生重排序。这一点至关重要，否则其他线程可能会在数据完全初始化之前就看到它。

同样地，我们可以将获取顺序解释为防止了会导致在指针被加载之前就访问数据的重排序。然而，人们可能会合理地好奇，这在实践中是否有意义。在知道地址之前，怎么可能访问数据呢？我们可能会得出结论，某种比获取顺序更弱的顺序可能就足够了。而事实正是如此：这种更弱的顺序被称为 消费顺序。

消费顺序基本上是获取顺序的一种轻量级——更高效的——变体，其同步效果仅限于 依赖于加载值的内容。

这意味着，如果你从一个原子变量中消费加载(consume-load)一个release-stored值 `x`，那么，基本上，该存储操作发生在**依赖于**该值的表达式(如 `*x`、`array[x]` 或 `table.lookup(x + 1)`)的求值之前，但 不一定发生在独立操作（例如读取另一个我们不需要其值 `x` 的变量）之前。

现在，有个好消息和一个坏消息。

好消息是——在所有现代处理器架构中——消费顺序是通过与宽松顺序完全相同的指令实现的。换句话说，消费顺序可以是“免费”的，而这一点——至少在某些平台上——对于获取顺序来说并非如此。

糟糕的消息是，目前还没有任何编译器真正实现了 Consume 内存顺序。

事实证明，这种“依赖”评估的概念不仅难以定义，而且在对程序进行转换和优化时，更难以保持这些依赖关系的完整性。例如，编译器可能会将`x+2-x`优化为简单的 `2`，从而实际上 Drop 了对 `x` 的依赖。在面对像 `array[x]` 这样更真实的表达式时，如果编译器能够对 `x` 或数组元素的可能值做出任何逻辑推断，就会出现更微妙的变体问题。而当考虑到控制流（例如 `if` 语句或函数调用）时，问题会变得更加复杂。

因此，为了安全起见，编译器会将消费顺序升级为获取内存顺序。C语言++20标准甚至明确不鼓励使用消费顺序，并指出除了获取内存顺序之外的实现方式

noting that an implementation other than just acquire ordering turned out to be infeasible.

It's possible that a workable definition and implementation of consume ordering might be found in the future. Until that time arrives, however, Rust does not expose `Ordering::Consume`.

Sequentially Consistent Ordering

The strongest memory ordering is *sequentially consistent* ordering: `Ordering::SeqCst`. It includes all the guarantees of acquire ordering (for loads) and release ordering (for stores), and *also* guarantees a globally consistent order of operations.

This means that every single operation using SeqCst ordering within a program is part of a single total order that all threads agree on. This total order is consistent with the total modification order of each individual variable.

Since it is strictly stronger than acquire and release memory ordering, a sequentially consistent load or store can take the place of an acquire-load or release-store in a release-acquire pair to form a happens-before relationship. In other words, an acquire-load can not only form a happens-before relationship with a release-store, but also with a sequentially consistent store, and similarly the other way around.



Only when both sides of a happens-before relationship use SeqCst ordering is it guaranteed to be consistent with the single total order of SeqCst operations.

While it might seem like the easiest memory ordering to reason about, SeqCst ordering is almost never necessary in practice. In nearly all cases, regular acquire and release ordering suffice.

Here's an example that depends on sequentially consistent ordered operations:

```
use std::sync::atomic::Ordering::SeqCst;

static A: AtomicBool = AtomicBool::new(false);
static B: AtomicBool = AtomicBool::new(false);

static mut S: String = String::new();

fn main() {
    let a = thread::spawn(|| {
        A.store(true, SeqCst);
        if !B.load(SeqCst) {
            unsafe { S.push('!') };
        }
    })
}
```

被证明是不可行的。

未来可能会找到消费顺序的可行定义和实现。但在那时之前，Rust 并不公开 `Ordering::Consume`。

顺序一致性排序

最强的内存顺序是顺序一致性顺序: `Ordering::SeqCst`。它包含了获取顺序（用于加载）和释放顺序（用于存储）的所有保证，同时还保证了全局一致的操作顺序。

这意味着程序中每一个使用 SeqCst 内存顺序的操作都是一个所有线程都一致认同的单一全序的一部分。该全序与每个独立变量的总修改顺序是一致的。

由于它严格强于获取和释放内存顺序，因此顺序一致性的加载或存储操作可以在释放-获取配对中替代获取-加载或释放-存储，从而形成先行发生 (happens-before) 关系。换句话说，获取-加载不仅可与释放-存储形成先行发生关系，也可以与顺序一致性存储形成该关系；反之亦然。



只有当先行发生关系的双方都使用 SeqCst 内存顺序时，才能保证其与 SeqCst 操作的单一全序保持一致。

虽然 SeqCst 内存顺序看起来是最容易推导的内存排序，但在实践中几乎从未必要。在绝大多数情况下，常规的获取内存顺序 and 释放语义就足够了。

这是一个依赖于顺序一致内存顺序操作的示例：

```
use std::sync::atomic::Ordering::SeqCst;

static A: AtomicBool = AtomicBool::new(false);
static B: AtomicBool = AtomicBool::new(false);

static mut S: String = String::new();

fn main() {
    let a = thread::spawn(|| {
        A.store(true, SeqCst);
        if !B.load(SeqCst) {
            unsafe { S.push('!') };
        }
    })
}
```

```
});

let b = thread::spawn(|| {
    B.store(true, SeqCst);
    if !A.load(SeqCst) {
        unsafe { S.push('!') };
    }
});

a.join().unwrap();
b.join().unwrap();
}
```

Both threads first set their own atomic boolean to `true` to warn the other thread that they are about to access `S`, and then check the other's atomic boolean to see if they can safely access `S` without causing a data race.

If both store operations happen before either of the load operations, it's possible that neither thread ends up accessing `S`. However, it's impossible for *both* threads to access `S` and cause undefined behavior, since the sequentially consistent ordering guarantees only one of them can win the race. In every possible single total order, the first operation will be a store operation, which prevents the other thread from accessing `S`.

Virtually all real-world uses of `SeqCst` involve a similar pattern of a store that must be globally visible before a subsequent load on the same thread. For these situations, a potentially more efficient alternative is to instead use relaxed operations in combination with a `SeqCst fence`, which we'll explore next.

Fences

In addition to operations on atomic variables, there is one more thing that we can apply a memory ordering to: atomic fences.

The `std::sync::atomic::fence` function represents an *atomic fence* and is either a *release fence* (Release), an *acquire fence* (Acquire), or both (AcqRel or SeqCst). A `SeqCst` fence additionally also takes part in the sequentially consistent total order.

An atomic fence allows you to separate the memory ordering from the atomic operation. This can be useful if you want to apply a memory ordering to multiple operations, or if you only want to apply it conditionally.

```
});

let b = thread::spawn(|| {
    B.store(true, SeqCst);
    if !A.load(SeqCst) {
        unsafe { S.push('!') };
    }
});

a.join().unwrap();
b.join().unwrap();
}
```

两个线程首先将各自的原子布尔值设置为 `true`，以警告另一个线程它们即将访问 `s`，然后检查另一个线程的原子布尔值，以确认在不引起数据竞态的情况下是否可以安全地访问 `s`。

如果两个存储操作都发生在任何加载操作之前，那么可能会出现没有任何一个线程最终访问 `s` 的情况。然而，两个线程同时访问 `s` 并导致未定义行为是不可能的，因为顺序一致性排序保证了只有一个线程能够赢得竞态。在每一种可能的单一全序中，第一个操作将是一个存储操作，从而阻止另一个线程访问 `s`。

几乎所有现实世界中对 `SeqCst` 的使用都涉及类似的模式，即存储操作必须在同一线程随后的加载操作之前全局可见。对于这些情况，一个潜在的更高效的替代方案是结合使用松弛操作和 `SeqCst` 内存栅栏，我们接下来将对此进行探讨。

内存栅栏

除了对原子变量的操作之外，还有一项我们可以应用内存排序的内容：原子栅栏。

`std::sync::atomic::fence`函数表示一个**原子栅栏(atomic fence)**，它可以是释放屏障(Release)、获取屏障(Acquire)，或者两者兼有(AcqRel或SeqCst)。SeqCst栅栏还会额外参与顺序一致性全序。

原子栅栏允许您将内存排序与原子操作分离。如果您想将某种内存排序应用于多个操作，或者只想有条件地应用它，那么这将非常有用。

In essence, a release-store can be split into a release fence followed by a (relaxed) store, and an acquire-load can be split into a (relaxed) load followed by an acquire fence:

The store of a release-acquire relationship, <code>a.store(1, Release);</code>	The load of a release-acquire relationship, <code>a.load(Acquire);</code>
can be substituted by a release fence followed by a relaxed store: <code>fence(Release);</code> <code>a.store(1, Relaxed);</code>	can be substituted by a relaxed load followed by an acquire fence: <code>a.load(Relaxed);</code> <code>fence(Acquire);</code>

Using a separate fence can result in an extra processor instruction, though, which can be slightly less efficient.

More importantly, unlike a release-store or an acquire-load, a fence is not tied to any single atomic variable. This means that a single fence can be used for multiple variables at once.

Formally, a release fence can take the place of a release operation in a happens-before relationship if that release fence is followed (on the same thread) by any atomic operation that stores a value observed by the acquire operation we’re synchronizing with. Similarly, an acquire fence can take the place of any acquire operation if that acquire fence is preceded (on the same thread) by any atomic operation that loads a value stored by the release operation.

Putting this together, it means that a happens-before relationship is created between a release fence and an acquire fence if *any* store after the release fence is observed by *any* load before the acquire fence.

For example, suppose we have one thread executing a release fence followed by three atomic store operations to different variables, and another thread executing three load operations from those same variables followed by an acquire fence, as follows:

Thread 1:	Thread 2:
<code>fence(Release);</code>	<code>A.load(Relaxed);</code>
<code>A.store(1, Relaxed);</code>	<code>B.load(Relaxed);</code>
<code>B.store(2, Relaxed);</code>	<code>C.load(Relaxed);</code>
<code>C.store(3, Relaxed);</code>	<code>fence(Acquire);</code>

In this situation, if *any* of the load operations on thread 2 loads the value from the corresponding store operation of thread 1, the acquire fence on thread 2 happens-before the release fence on thread 1.

A fence does not have to directly precede or follow the atomic operations. Anything else can happen in between, including control flow. This can be used to make the

本质上，一个释放存储操作可以被分解为一个释放屏障紧跟一个（宽松）存储操作，而一个获取加载操作可以被分解为一个（宽松）加载操作紧跟一个获取屏障：

释放-获取关系中的存储操作， <code>a.store(1, Release);</code>	释放-获取关系中的加载操作， <code>a.load(Acquire);</code>
可以用一个释放屏障紧跟一个宽松存储来替代： <code>fence(Release);</code> <code>a.store(1, Relaxed);</code>	可以用一个松散加载紧跟一个获取屏障来替代： <code>a.load(Relaxed);</code> <code>fence(Acquire);</code>

不过，使用独立的屏障可能会导致产生额外的处理器指令，从而导致效率略微降低。

更重要的是，与释放存储或获取加载不同，屏障不绑定于任何单个原子变量。这意味着单个屏障可以同时用于多个变量。

正式地说，如果一个释放屏障之后（在同一线程上）紧跟任何原子操作，且该操作存储的值被我们正在同步的获取操作观察到，那么该释放屏障可以替代先行发生关系中的释放操作。同样地，如果一个获取屏障之前（在同一线程上）紧跟任何原子操作，且该操作加载了由释放操作存储的值，那么该获取屏障可以替代任何获取操作。

综合来看，这意味着如果在释放屏障之后的任何存储操作被获取屏障之前的任何加载操作观察到，那么在释放屏障和获取屏障之间就创建了一个先行发生关系。

例如，假设我们有一个线程执行一个释放屏障，随后是对不同变量执行三次原子存储操作；而另一个线程则从这些相同的变量中执行三次加载操作，随后执行一个获取屏障，如下所示：

线程 1:	线程 2:
<code>fence(Release);</code>	<code>A.load(Relaxed);</code>
<code>A.store(1, Relaxed);</code>	<code>B.load(Relaxed);</code>
<code>B.store(2, Relaxed);</code>	<code>C.load(Relaxed);</code>
<code>C.store(3, Relaxed);</code>	<code>fence(Acquire);</code>

在这种情况下，如果线程 2 上的任何加载操作加载了来自线程 1 对应存储操作的值，那么线程 2 上的获取屏障就发生在线程 1 上的释放屏障之后。

内存屏障不必直接地位于原子操作之前或之后。其间可以发生任何其他操作，包括控制流。这可以用来使内存屏障变为条件性的，类似于比较并交换操作

fence conditional, similar to how compare-and-swap operations have both a success and a failure ordering.

For example, if we load a pointer from an atomic variable using acquire memory ordering, we could use a fence to apply the acquire ordering only when the pointer is not null:

Using an acquire-load:

```
let p = PTR.load(Acquire);
if p.is_null() {
    println!("no data");
} else {
    println!("data = {}", unsafe { *p });
}
```

Using a conditional acquire fence:

```
let p = PTR.load(Relaxed);
if p.is_null() {
    println!("no data");
} else {
    fence(Acquire);
    println!("data = {}", unsafe { *p });
}
```

This can be beneficial if the pointer is often expected to be null, to avoid acquire memory ordering when not necessary.

Let's take a look at a more complicated use case of release and acquire fences:

```
use std::sync::atomic::fence;

static mut DATA: [u64; 10] = [0; 10];

const ATOMIC_FALSE: AtomicBool = AtomicBool::new(false);
static READY: [AtomicBool; 10] = [ATOMIC_FALSE; 10];

fn main() {
    for i in 0..10 {
        thread::spawn(move || {
            let data = some_calculation(i);
            unsafe { DATA[i] = data };
            READY[i].store(true, Release);
        });
    }
    thread::sleep(Duration::from_millis(500));
    let ready: [bool; 10] = std::array::from_fn(|i| READY[i].load(Relaxed));
    if ready.contains(&true) {
        fence(Acquire);
        for i in 0..10 {
            if ready[i] {
                println!("data[i] = {}", unsafe { DATA[i] });
            }
        }
    }
}
```

同时拥有成功和失败的内存顺序。

例如，如果我们要使用获取内存顺序从原子变量加载一个指针，我们可以使用内存栅栏来仅在指针不为 null 时应用获取顺序：

Using an acquire-load:

```
let p = PTR.load(Acquire);
if p.is_null() {
    println!("no data");
} else {
    println!("data = {}", unsafe { *p });
}
```

Using a conditional acquire fence:

```
let p = PTR.load(Relaxed);
if p.is_null() {
    println!("no data");
} else {
    fence(Acquire);
    println!("data = {}", unsafe { *p });
}
```

如果指针经常被预期为null，那么这样做是有益的，可以避免在不必要时使用获取内存顺序。

让我们来看一个更复杂的释放与获取内存栅栏的使用案例：

```
use std::sync::atomic::fence;

static mut DATA: [u64; 10] = [0; 10];

const ATOMIC_FALSE: AtomicBool = AtomicBool::new(false);
static READY: [AtomicBool; 10] = [ATOMIC_FALSE; 10];

fn main() {
    for i in 0..10 {
        thread::spawn(move || {
            let data = some_calculation(i);
            unsafe { DATA[i] = data };
            READY[i].store(true, Release);
        });
    }
    thread::sleep(Duration::from_millis(500));
    let ready: [bool; 10] = std::array::from_fn(|i| READY[i].load(Relaxed));
    if ready.contains(&true) {
        fence(Acquire);
        for i in 0..10 {
            if ready[i] {
                println!("data[i] = {}", unsafe { DATA[i] });
            }
        }
    }
}
```



`std::array::from_fn` is an easy way to execute something a certain number of times and collect the results into an array.

In this example, 10 threads do some calculations and store their results in a (non-atomic) shared variable. Each thread sets an atomic boolean to indicate that the data is ready to be read by the main thread, using a normal release-store. The main thread waits for half a second, checks all 10 booleans to see which threads are done, and prints whichever results are ready.

Instead of using 10 acquire-load operations to read the booleans, the main thread uses relaxed operations and a single acquire fence. It executes the fence before reading the data, but only if there is data to be read.

While in this particular example it might be completely unnecessary to put any effort into such optimization, this pattern for saving the overhead of additional acquire operations can be important when building highly efficient concurrent data structures.

A `SeqCst` fence is both a release fence and an acquire fence (just like `AcqRel`), but also part of the single total order of sequentially consistent operations. However, only the fence is part of the total order, but not necessarily the atomic operations before or after it. This means that unlike a release or acquire operation, a sequentially consistent operation cannot be split into a relaxed operation and a memory fence.

Compiler Fences

In addition to a regular atomic fence, the Rust standard library also provides a *compiler fence*: `std::sync::atomic::compiler_fence`. Its signature is identical to that of the regular `fence()` we discussed above, but its effects are restricted to just the compiler. Unlike a regular atomic fence, it does not prevent the processor from, for example, reordering instructions. In the vast majority of use cases for fences, a compiler fence does not suffice.

A potential use case might arise when implementing a Unix *signal handler*, or an *interrupt* on embedded systems. These are mechanisms that can suddenly interrupt a thread to temporarily execute an unrelated function on the same processor core. Because it happens on the same processor core, the usual ways in which the processor might affect memory ordering don't apply. (More on that in [Chapter 7](#).) In this case, a compiler fence might suffice, potentially saving an instruction and hopefully increasing performance.

Another use case involves *process-wide memory barriers*. This technique falls outside the scope of Rust's memory model and is only supported on some operating



`std::array::from_fn` 是一种简单的方法，可以执行某项操作特定次数，并将结果收集到一个数组中。

在这个例子中，10 个线程进行一些计算并将结果存储在一个（非原子的）共享变量中。每个线程使用一个普通的释放存储操作设置一个原子布尔值，以指示数据已准备好供主线程读取。主线程等待半秒钟，检查所有 10 个布尔值以查看哪些线程已完成，并打印所有已就绪的结果。

主线程没有使用 10 次 acquire-load 操作来读取布尔值，而是使用了松弛操作和一个单一的获取屏障。它在读取数据之前执行该屏障，但仅在有数据可读时才执行。

虽然在这个特定的例子中，投入精力进行此类优化可能完全没有必要，但在构建高效的并发数据结构时，这种节省额外获取操作开销的模式可能非常重要。

一个 `SeqCst` fence 既是释放屏障也是获取屏障（就像 `AcqRel` 一样），而且还是顺序一致性操作的单一全序的一部分。然而，只有该屏障本身是全序的一部分，而它之前或之后的原子操作则不一定。这意味着，与释放或获取操作不同，顺序一致性操作不能被拆分为一个松弛操作和一个内存栅栏。

编译器栅栏

除了常规的原子栅栏外，Rust 标准库还提供了一种编译器栅栏：

`std::sync::atomic::compiler_fence`。它的签名与我们上面讨论的常规 `fence()` 完全相同，但其效果仅限于编译器。与常规原子栅栏不同，它不能防止处理器进行（例如）指令重排。在绝大多数需要内存栅栏的使用场景中，编译器栅栏是不够的。

一个潜在的使用场景可能出现在实现 Unix 信号处理程序，或者在嵌入式系统上实现中断时。这些机制可以突然中断一个线程，以便在同一个处理器核心上临时执行一个无关的函数。由于这发生在同一个处理器核心上，处理器可能影响内存排序的常规方式不再适用。（详见 [第 7 章](#)。）在这种情况下，编译器栅栏可能就足够了，从而可能节省一条指令并有望提高性能。

另一个用例涉及 进程级内存屏障。这项技术超出了 Rust 内存模型的作用域，并且仅在某些操作系统上受支持：

systems: on Linux through the the `membarrier` syscall, and on Windows using the `FlushProcessWriteBuffers` function. It effectively allows a thread to forcefully inject a (sequentially consistent) atomic fence into all concurrently running threads. This allows us to replace two matching fences with a lightweight compiler fence on one side and a heavyweight process-wide barrier on the other side. If the code on the side of the lightweight fence is executed much more often, this can improve overall performance. (See the documentation of the `membarrier` crate on [crates.io](https://crates.io/crates/membarrier) for more details and a cross-platform way to use such barriers in Rust.)

A compiler fence can also be an interesting tool for exploring the effect of the processor on memory ordering. In “[An Experiment](#)” on [page 155](#), we’ll break our code on purpose by replacing a regular fence with a compiler fence. This will let us experience the subtle but potentially disastrous effects of the processor when using the wrong memory ordering.

Common Misconceptions

There are a lot of misconceptions around memory ordering. Before we end this chapter, let’s go over the most common ones.

Myth: *I need strong memory ordering to make sure changes are “immediately” visible.*

A common misunderstanding is that using a weak memory ordering like `Relaxed` means that changes to an atomic variable might never arrive at another thread, or only after a significant delay. The name “relaxed” might make it sound like nothing happens until something forces some part of the hardware to wake up and do what it should’ve done instead of relaxing.

The truth is that the memory model doesn’t say anything about timing at all. It only defines in which order certain things happen; not how long you might have to wait for them. A hypothetical computer in which it takes years to get data from one thread to another is quite unusable, but can perfectly satisfy the memory model.

In real life, memory ordering is about things like reordering instructions, which usually happen at nanosecond scale. Stronger memory ordering does not make your data travel faster; it might even slow your program down.

Myth: *Disabling optimization means I don’t need to care about memory ordering.*

Both the compiler and the processor play a role in making things happen in a different order than we might expect. Disabling compiler optimization does not disable every possible transformation in the compiler, and does not disable the processor features that result in instruction reordering and similar potentially problematic behavior.

在Linux上通过 `membarrier` 系统调用，在Windows上使用 `FlushProcessWriteBuffers` 函数。它有效地允许一个线程强制地将一个（顺序一致的）原子栅栏注入到所有并发运行的线程中。这使得我们能够用一侧的轻量级编译器栅栏和另一侧的重量级进程级屏障来替换两个匹配的内存栅栏。如果轻量级栅栏侧的代码被执行得更频繁，这可以提高整体性能。(详见 [crates.io](https://crates.io/crates/membarrier) 上 `membarrier` crate 的文档，以了解更多细节以及在 Rust 中使用此类 `barrier` 的跨平台方法。)

编译器栅栏也可以成为探索处理器对内存顺序影响的一种有趣工具。在[第 155 页的“一次实验”](#)中，我们将通过用编译器栅栏替换普通栅栏，故意破坏我们的代码。这将让我们体验到在使用错误的内存顺序时，处理器所带来的细微但可能具有毁灭性的影响。

常见误区

关于内存顺序存在很多误区。在结束本章之前，让我们来回顾一下最常见的一些。

误区：我需要强内存顺序来确保更改能“立即”可见。

一个常见的误解是，使用像 `Relaxed` 这样的弱内存顺序意味着对原子变量的更改可能永远无法到达另一个线程，或者需要经过很长时间的延迟。“relaxed(宽松)”这个名字可能会让人觉得，除非有某种力量强制硬件的一部分苏醒并执行它本应该执行的操作，否则它会一直处于“放松”状态而什么都不做。

事实是，内存模型完全没有提及时间问题。它只定义了某些事情发生的顺序，而不是你可能需要等待它们多久。一台假设的计算机，如果将数据从一个线程传输到另一个线程需要数年时间，虽然完全无法使用，但它可以完美地满足内存模型。

在现实生活中，内存排序涉及指令重排等问题，这些通常发生在纳秒级别。更强的内存排序并不会让你的数据传输得更快；它甚至可能会降低程序的运行速度。

误区：禁用优化意味着我不需要关心内存排序。

编译器和处理器在导致执行顺序与我们预期地不同方面都起到了作用。禁用编译器优化并不能禁用编译器中所有可能的转换，也无法禁用处理器中导致指令重排及类似潜在问题行为的特性。

Myth: Using a processor that doesn't reorder instructions means I don't need to care about memory ordering.

Some simple processors, such as those in small microcontrollers, have only one core and only ever execute one instruction at a time, all in order. However, while it's true that on such devices there's a significantly lower chance of an incorrect memory ordering resulting in actual issues, it's still possible for the compiler to make invalid assumptions based on incorrect memory ordering, breaking your code. Besides that, it's also important to realize that even when a processor does not execute instructions out of order, it might still have other features that can be relevant for memory ordering.

Myth: Relaxed operations are free.

Whether this is true depends on your definition of “free.” It's true that Relaxed is the most efficient memory ordering and that it can be significantly faster than the others. It's even true that on all modern platforms, relaxed load and store operations compile down to the same processor instructions as non-atomic reads and writes.

If an atomic variable is only used by a single thread, any difference in speed with a non-atomic variable will most likely be because of the compiler having more freedom and being more effective at optimizing non-atomic operations. (Compilers tend to avoid most types of optimizations for atomic variables.)

However, accessing the same memory from multiple threads is usually significantly slower than accessing it from a single thread. A thread that continuously writes to an atomic variable will likely experience a noticeable slowdown when other threads start repeatedly reading the variable, since the processor cores and their caches now have to start collaborating.

We'll explore this effect in [Chapter 7](#).

Myth: Sequentially consistent memory ordering is a great default and is always correct.

Putting aside performance concerns, sequentially consistent memory ordering is often seen as the perfect type of memory ordering to default to, because of its strong guarantees. It's true that *if* any other memory ordering is correct, SeqCst is also correct. This might make it sound like SeqCst is always correct. However, it's entirely possible that a concurrent algorithm is simply incorrect, regardless of memory ordering.

More importantly, when reading code, SeqCst basically tells the reader: “this operation depends on the total order of every single SeqCst operation in the program,” which is an incredibly far-reaching claim. The same code would likely be easier to review and verify if it used weaker memory ordering instead, if possible. For example, Release effectively tells the reader: “this relates to an acquire operation on the same

误区：使用一个不重新排列指令的处理器意味着我不需要关心内存排序。

一些简单的处理器（例如小型微控制器中的处理器）只有一个核心，且始终一次只按顺序执行一条指令。然而，虽然在这些设备上，错误的内存顺序导致实际问题的概率显著降低，但编译器仍有可能基于错误的内存顺序做出无效假设，从而导致代码崩溃。除此之外，同样重要的是要意识到，即使处理器不乱序执行指令，它可能仍具有其他与内存顺序相关的特性。

误区：松弛操作是免费的。

这是否属实取决于你对“免费”的定义。Relaxed 确实是最高效的内存排序，且其速度可能明显快于其他排序方式。甚至可以说在所有现代平台上，宽松加载和存储操作编译后的处理器指令与非原子读取和写入完全相同。

如果一个原子变量仅被单线程使用，那么它与非原子变量在速度上的任何差异，极有可能是因为编译器在优化非原子操作时拥有更大的自由度和更高的效率。（编译器倾向于对原子变量避免大多数类型的优化。）

然而，从多个线程访问同一块内存通常比从单线程访问要慢得多。一个持续向原子变量写入的线程，在其他线程开始重复读取该变量时，可能会感受到明显的减速，因为此时处理器核心及其缓存必须开始协同工作。

我们将在[第 7 章](#)中探讨这一影响。

误区：顺序一致性内存顺序是一个极佳的默认选项，且永远是正确的。

撇开性能顾虑不谈，由于其强大的保证，顺序一致性内存顺序通常被视为最完美的默认内存顺序类型。诚然，如果任何其他内存顺序是正确的，那么 SeqCst 也是正确的。这听起来似乎意味着 SeqCst 永远正确。然而，一个并发算法可能本身就是错误的，无论使用什么样的内存排序。

更重要的是，在阅读代码时，SeqCst基本上在告诉读者：“此操作依赖于程序中每一个SeqCst操作的全序”，这是一个影响极其深远的断言。如果可能的话，若该代码使用较弱的内存排序，可能会更容易地进行审查和验证。例如，Release实际上在告诉读者：“这与同一个变量上的获取操作相关”，这在理解代码时

variable,” which involves far fewer considerations when forming an understanding of the code.

It is advisable to see SeqCst as a warning sign. Seeing it in the wild often means that either something complicated is going on, or simply that the author did not take the time to analyze their memory ordering related assumptions, both of which are reasons for extra scrutiny.

Myth: *Sequentially consistent memory ordering can be used for a “release-load” or an “acquire-store.”*

While SeqCst can stand in for Acquire or Release, it is not a way to somehow create an acquire-store or release-load. Those remain nonexistent. Release only applies to store operations, and acquire only to load operations.

For example, a Release-store does not form any release-acquire relationship with a SeqCst-store. If you need them to be part of a globally consistent order, both operations will have to use SeqCst.

Summary

- There might not be a global consistent order of all atomic operations, as things can appear to happen in a different order from different threads.
- However, each individual atomic variable has its own *total modification order*, regardless of memory ordering, which all threads agree on.
- The order of operations is formally defined through *happens-before* relationships.
- Within a single thread, there is a happens-before relationship between every single operation.
- Spawning a thread happens-before everything the spawned thread does.
- Everything a thread does happens-before joining that thread.
- Unlocking a mutex happens-before locking that mutex again.
- Acquire-loading the value from a release-store establishes a happens-before relationship. This value may be modified by any number of fetch-and-modify and compare-and-exchange operations.
- A consume-load would be a lightweight version of an acquire-load, if it existed.
- Sequentially consistent ordering results in a globally consistent order of operations, but is almost never necessary and can make code review more complicated.
- Fences allow you to combine the memory ordering of multiple operations or apply a memory ordering conditionally.

考虑因素要少得多。

建议将 SeqCst 视为一个警告信号。在实际代码中看到它通常意味着，要么是正在进行某种复杂的操作，要么仅仅是作者没有花时间分析其与内存顺序相关的假设，这两者都是需要额外审查的原因。

误区：顺序一致性内存顺序可以用于“释放-加载”或“获取-存储”。

虽然 SeqCst 可以替代 Acquire 或 Release，但它并不能以某种方式创建获取-存储或释放-加载。这两者依然是不存在的。释放语义仅适用于存储操作，而获取内存顺序仅适用于加载操作。

例如，一个 Release-存储操作不会与 SeqCst-存储操作形成任何释放-获取关系。如果你需要它们成为全局一致顺序的一部分，那么这两个操作都必须使用 SeqCst。

总结

- 所有原子操作可能并不存在一个全局一致顺序，因为从不同的线程来看，事情发生的顺序可能会有所不同。
- 然而，无论内存排序如何，每个独立的原子变量都有其所有线程都认同的总修改顺序。
- 操作的顺序是通过先行发生关系正式定义的。
- 在单线程内部，每一步操作之间都存在先行发生关系。
- 创建线程先行发生于该被创建线程所执行的所有操作。
- 线程所执行的所有操作都先行发生于该线程的汇合。
- 解锁互斥锁先行发生于再次锁定该互斥锁。
- 从一个释放存储中获取加载值会建立一个先行发生关系。该值可以被任意数量的获取并修改和比较并交换操作所修改。
- 如果消费加载存在的话，它将是获取加载的一个轻量级版本。
- 顺序一致性排序会导致操作具有全局一致顺序，但几乎从未必要，且会让代码审查变得更加复杂。
- 内存栅栏允许您结合多个操作的内存排序，或有条件地应用内存排序。

